

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng hệ thống thương mại điện tử theo kiến trúc
Microservices**

NGUYỄN ĐỨC BẢO HOÀNG

hoang.ndb227116@sis.hust.edu.vn

Chương trình đào tạo: Toán - Tin

Giảng viên hướng dẫn: TS. Vũ Khánh Quý

Chuyên ngành: Toán - Tin

Khoa: Toán - Tin

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng hệ thống thương mại điện tử theo kiến trúc
Microservices**

NGUYỄN ĐỨC BẢO HOÀNG

hoang.ndb227116@sis.hust.edu.vn

Chương trình đào tạo: Toán - Tin

Giảng viên hướng dẫn: TS. Vũ Khánh Quý

Chữ kí GVHD

Chuyên ngành: Toán - Tin

Khoa: Toán - Tin

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc đến các thầy cô trong Khoa Khoa học và Kỹ thuật Máy tính, Trường Công nghệ Thông tin và Truyền thông, Đại học Bách khoa Hà Nội đã tận tình giảng dạy, truyền đạt kiến thức và định hướng cho em trong suốt quá trình học tập tại trường.

Đặc biệt, em xin gửi lời cảm ơn chân thành nhất đến **TS. [Tên giảng viên hướng dẫn]**, người đã trực tiếp hướng dẫn, định hướng nghiên cứu và dành nhiều thời gian quý báu để góp ý, chỉnh sửa cho đề án này. Những lời khuyên và phản biện của thầy/cô đã giúp em nhìn nhận vấn đề một cách hệ thống và khoa học hơn, từ đó hoàn thiện đề án một cách tốt nhất.

Em cũng xin cảm ơn gia đình và bạn bè đã luôn động viên, ủng hộ và tạo điều kiện thuận lợi để em tập trung vào việc nghiên cứu và hoàn thành đề án đúng thời hạn.

Trong quá trình thực hiện, mặc dù đã cố gắng hết sức nhưng do giới hạn về thời gian và kiến thức, đề án không tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp quý báu từ các thầy cô để đề án được hoàn thiện hơn.

Hà Nội, tháng 6 năm 2026

Sinh viên thực hiện

Nguyễn Đức Bảo Hoàng

TÓM TẮT NỘI DUNG ĐỒ ÁN

Đồ án trình bày toàn bộ quá trình phân tích, thiết kế và triển khai một hệ thống thương mại điện tử theo kiến trúc microservices, sử dụng Spring Boot 3.5, Spring Cloud 2025 và Java 21. Hệ thống được phân rã thành 13 microservice độc lập dựa trên nguyên tắc Domain-Driven Design (DDD) và Bounded Context, mỗi service sở hữu cơ sở dữ liệu và vòng đời triển khai riêng biệt.

Về giao tiếp dịch vụ, hệ thống kết hợp REST + OpenFeign (đồng bộ) với Apache Kafka KRaft (bất đồng bộ). Bốn distributed pattern cốt lõi được áp dụng đầy đủ: Saga Orchestration với order-service đóng vai orchestrator quản lý distributed transaction; Transactional Outbox đảm bảo at-least-once event publishing; Idempotent Consumer đảm bảo exactly-once processing effect; CQRS-lite với Elastic-search làm read model phục vụ full-text search.

Hai tính năng kỹ thuật nổi bật của hệ thống là: (1) flash sale chống over-sell bằng Redis Lua atomic script, buyer set, cơ chế bù trừ và reconciliation; (2) tích hợp cổng thanh toán VNPAY sandbox với xác thực HMAC-SHA512, Return URL, IPN callback và timeout scheduler. Bảo mật được xử lý tập trung qua Keycloak 26 (OAuth 2.0/OIDC) với JWT RS256, API Gateway đóng vai Resource Server theo Trusted Subsystem Pattern — các service nội bộ nhận thông tin định danh qua header mà không cần gọi lại Keycloak ở mỗi request.

Ngoài backend, đồ án triển khai Next.js storefront và Admin Panel phục vụ các luồng người mua và quản trị. Hệ thống được đóng gói bằng Docker Compose, triển khai production-like trên một máy ảo AWS EC2 với Caddy reverse proxy/HTTPS và GitHub Actions/GHCR cho quy trình build/push image. Observability stack gồm Prometheus, Grafana và Zipkin cho phép giám sát metrics, trace và debug các luồng nghiệp vụ end-to-end. Kết quả thực nghiệm gồm smoke test backend 9/9 nhóm đạt, k6 catalog/checkout/flash-sale và các kịch bản resilience cho order-service, Kafka, Redis và inventory compensation.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)
Nguyễn Đức Bảo Hoàng

ABSTRACT

This thesis presents the complete analysis, design, and implementation of an e-commerce system based on a microservices architecture, built with Spring Boot 3.5, Spring Cloud 2025, and Java 21. The system is decomposed into 13 independent microservices following Domain-Driven Design (DDD) and Bounded Context principles, each owning its own database and deployment lifecycle.

Service communication combines synchronous REST + OpenFeign with asynchronous Apache Kafka (KRaft mode). Four core distributed patterns are fully applied: Saga Orchestration with order-service acting as orchestrator for distributed transaction management; Transactional Outbox guaranteeing at-least-once event publishing; Idempotent Consumer ensuring exactly-once processing effect; and CQRS-lite with Elasticsearch as the read model for full-text search.

Two notable technical highlights are: (1) a flash sale over-sell prevention mechanism using Redis Lua atomic scripts, buyer set, compensation, and reconciliation; (2) VNPAY payment gateway integration with HMAC-SHA512 verification, Return URL, IPN callback handling, and timeout scheduler. Security is centralized via Keycloak 26 (OAuth 2.0/OIDC) with JWT RS256, where the API Gateway acts as Resource Server applying the Trusted Subsystem Pattern — internal services receive identity information via request headers without needing to call Keycloak on every request.

In addition to the backend, the thesis implements a Next.js storefront and Admin Panel for customer and administrator workflows. The system is packaged with Docker Compose and deployed in a production-like single-host environment on AWS EC2, using Caddy for reverse proxy/HTTPS and GitHub Actions/GHCR for image build and publishing. The observability stack includes Prometheus, Grafana, and Zipkin, enabling metrics monitoring, distributed tracing, and end-to-end debugging. Experimental results include 9/9 backend readiness suites passing, k6 catalog/checkout/flash-sale tests, and resilience scenarios covering order-service, Kafka, Redis, and inventory compensation.

MỤC LỤC

DANH MỤC TỪ VIẾT TẮT	ix
DANH MỤC THUẬT NGỮ	x
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu đề tài	2
1.3 Phạm vi đề tài	3
1.4 Khảo sát hệ thống tương tự	4
1.5 Định hướng giải pháp.....	6
1.6 Bố cục báo cáo	8
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT.....	10
2.1 Kiến trúc Microservices	10
2.1.1 Định nghĩa và đặc trưng cốt lõi.....	10
2.1.2 So sánh với Monolith và SOA	10
2.1.3 Tám ngọ nhận trong tính toán phân tán.....	11
2.2 Domain-Driven Design và phân rã hệ thống	11
2.3 Các mô hình giao tiếp trong Microservices	12
2.3.1 Giao tiếp đồng bộ và bất đồng bộ	12
2.3.2 Mức độ đảm bảo phân phối thông điệp.....	13
2.4 Giao dịch phân tán và tính nhất quán dữ liệu	13
2.4.1 Định lý CAP và nguyên lý PACELC	13
2.4.2 Saga Pattern	14
2.4.3 Transactional Outbox Pattern	15
2.4.4 Idempotent Consumer	16
2.4.5 CQRS — Phân tách trách nhiệm Lệnh và Truy vấn	17

2.5 Các mẫu thiết kế đảm bảo độ tin cậy	18
2.5.1 Circuit Breaker	18
2.5.2 Rate Limiting	18
2.6 Bảo mật trong hệ thống phân tán	19
2.6.1 OAuth 2.0 — Khung ủy quyền tiêu chuẩn	19
2.6.2 OpenID Connect — Lớp xác thực định danh	19
2.6.3 JSON Web Token (JWT)	20
2.6.4 Trusted Subsystem Pattern	20
2.7 API Gateway và Service Discovery	21
2.7.1 API Gateway Pattern	21
2.7.2 Service Discovery	21
2.8 Khả năng quan sát trong hệ thống phân tán	22
CHƯƠNG 3. GIẢI PHÁP CÔNG NGHỆ	24
3.1 Tổng quan giải pháp công nghệ	24
3.2 Ngôn ngữ và nền tảng phát triển	24
3.2.1 Java 21	24
3.2.2 Maven Multi-Module	25
3.2.3 Spring Boot 3.5	25
3.2.4 Thư viện hỗ trợ	26
3.3 Hệ sinh thái Spring Cloud	26
3.3.1 Spring Cloud Gateway	26
3.3.2 Netflix Eureka — Service Discovery	27
3.3.3 Spring Cloud Config Server	27
3.3.4 OpenFeign — Giao tiếp đồng bộ giữa các dịch vụ	28
3.4 Hạ tầng dữ liệu	28
3.4.1 PostgreSQL 17	28

3.4.2 Redis 8.....	29
3.4.3 Elasticsearch 8.....	29
3.5 Apache Kafka — Message Broker.....	30
3.6 Keycloak — Bảo mật và quản lý định danh.....	30
3.7 VNPAY — Cổng thanh toán trực tuyến.....	31
3.8 Hệ thống quan sát — Observability Stack.....	32
3.8.1 Prometheus và Grafana — Giám sát số liệu.....	32
3.8.2 Zipkin — Truy vết phân tán.....	33
3.8.3 Mailpit — Kiểm thử email.....	33
3.9 Next.js, AWS EC2 và CI/CD.....	33
3.9.1 Next.js — Storefront và Admin Panel.....	33
3.9.2 AWS EC2, Caddy và GitHub Actions.....	34
3.10 Docker Compose — Đóng gói và triển khai.....	34
3.11 Công cụ kiểm thử.....	35
3.11.1 JUnit 5 và Testcontainers.....	35
3.11.2 k6 — Kiểm thử hiệu năng.....	35
CHƯƠNG 4. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	36
4.1 Phân tích yêu cầu hệ thống.....	36
4.1.1 Yêu cầu chức năng.....	36
4.1.2 Yêu cầu phi chức năng.....	37
4.2 Mô hình hóa yêu cầu — Biểu đồ Use Case.....	38
4.3 Kiến trúc tổng thể.....	39
4.4 Phân rã miền nghiệp vụ và sở hữu dịch vụ.....	40
4.5 Thiết kế cơ sở dữ liệu.....	41
4.5.1 Mô hình quan hệ — PostgreSQL.....	41
4.5.2 Mô hình tài liệu — Elasticsearch.....	42

4.5.3 Mô hình khóa — Redis	43
4.6 Thiết kế API và giao tiếp đồng bộ	43
4.6.1 Định tuyến API Gateway	43
4.6.2 Giao tiếp Feign giữa các dịch vụ	44
4.7 Thiết kế luồng sự kiện — Kafka Topics	44
4.8 Thiết kế bảo mật	46
4.9 Thiết kế máy trạng thái	47
4.9.1 Máy trạng thái đơn hàng	47
4.9.2 Máy trạng thái thanh toán	47
4.9.3 Máy trạng thái chiến dịch Flash Sale	47
4.10 Thiết kế luồng nghiệp vụ chính	48
4.10.1 Luồng đăng ký người dùng	48
4.10.2 Luồng đặt hàng COD — Saga Orchestration	48
4.10.3 Luồng đặt hàng VNPAY	50
4.10.4 Luồng Flash Sale	50
4.10.5 Luồng tìm kiếm sản phẩm (CQRS-lite)	51
4.11 Thiết kế tác vụ nền — Scheduled Jobs	52
CHƯƠNG 5. CÀI ĐẶT, TRIỂN KHAI, KIỂM THỬ VÀ ĐÁNH GIÁ	54
5.1 Môi trường phát triển và cấu trúc dự án	54
5.2 Cài đặt các thành phần backend cốt lõi	55
5.2.1 Spring Cloud layer	55
5.2.2 Database per Service và Flyway	55
5.2.3 Saga Orchestration, Outbox và Idempotent Consumer	55
5.2.4 Thanh toán VNPAY	56
5.2.5 Flash sale high-concurrency	56
5.3 Cài đặt bảo mật và phân quyền	56

5.4 Cài đặt frontend Storefront và Admin Panel	57
5.5 Docker Compose và vận hành local	58
5.6 Triển khai production-like trên AWS EC2	59
5.7 Chiến lược kiểm thử.....	59
5.8 Kết quả kiểm thử hiệu năng	60
5.9 Kết quả kiểm thử khả năng chịu lỗi	61
5.10 Đánh giá tổng thể.....	62
5.10.1 Đối chiếu với mục tiêu đề tài	62
5.10.2 Hạn chế	62
TÀI LIỆU THAM KHẢO.....	65
PHỤ LỤC.....	66
A. HƯỚNG DẪN CÀI ĐẶT VÀ VẬN HÀNH.....	66
B. ĐẶC TẢ USE CASE.....	69

DANH MỤC HÌNH VẼ

Hình 1.1	Sơ đồ tổng quan sáu trụ cột kỹ thuật của hệ thống	6
Hình 2.1	Saga Orchestration Pattern	15
Hình 2.2	Saga Choreography Pattern	15
Hình 2.3	Luồng hoạt động của Transactional Outbox Pattern	16
Hình 2.4	Sơ đồ kiến trúc CQRS với mô hình ghi và mô hình đọc riêng biệt	17
Hình 2.5	Sơ đồ trạng thái của Circuit Breaker Pattern	18
Hình 2.6	Kiến trúc ứng dụng sử dụng API Gateway	21
Hình 4.1	Biểu đồ Use Case tổng thể hệ thống	39
Hình 4.2	Sơ đồ thành phần bốn tầng của hệ thống	40
Hình 4.3	Mô hình khóa Redis: Cart, Cache, Flash Sale	43
Hình 4.4	Sơ đồ luồng sự kiện Kafka giữa các dịch vụ	46
Hình 4.5	Luồng xác thực và phân quyền qua API Gateway	47
Hình 4.6	Sơ đồ máy trạng thái thanh toán và chiến dịch Flash Sale . . .	48
Hình 4.7	Biểu đồ tuần tự luồng đăng kí và đăng nhập người dùng	49
Hình 4.8	Biểu đồ tuần tự luồng đặt hàng COD — Saga Orchestration . .	50
Hình 4.9	Biểu đồ tuần tự luồng đặt hàng VNPAY với xử lý IPN callback	51
Hình 4.10	Biểu đồ tuần tự luồng mua hàng Flash Sale với cơ chế ba lớp bảo vệ	52
Hình 4.11	Biểu đồ tuần tự luồng tìm kiếm sản phẩm theo mô hình CQRS-lite	52

DANH MỤC BẢNG BIỂU

Bảng 1.1	Các thành phần trong phạm vi đề tài	3
Bảng 1.2	Khảo sát kiến trúc các sàn thương mại điện tử lớn	5
Bảng 2.1	So sánh ba mô hình kiến trúc phần mềm	11
Bảng 2.2	So sánh hai mô hình giao tiếp trong microservices	13
Bảng 2.3	So sánh Saga Choreography và Saga Orchestration	14
Bảng 3.1	Các thư viện và công cụ phát triển chính	26
Bảng 4.1	Yêu cầu chức năng theo miền nghiệp vụ	37
Bảng 4.2	Yêu cầu phi chức năng và chiến lược đáp ứng	38
Bảng 4.3	Sở hữu dịch vụ và giao tiếp	41
Bảng 4.4	Định tuyến API Gateway	44
Bảng 4.5	Bản đồ Kafka Topics	45
Bảng 4.6	Tác vụ nền trong hệ thống	53
Bảng 5.1	Môi trường và công cụ phát triển	54
Bảng 5.2	Map service với lớp lưu trữ chính	55
Bảng 5.3	Chính sách phân quyền tại API Gateway	57
Bảng 5.4	Các module chính của Admin Panel	58
Bảng 5.5	Thành phần triển khai production-like	59
Bảng 5.6	Kết quả backend readiness smoke test	60
Bảng 5.7	Tổng hợp kết quả kiểm thử hiệu năng bằng k6	60
Bảng 5.8	Tổng hợp kết quả kiểm thử khả năng chịu lỗi	61
Bảng 5.9	Đối chiếu kết quả với mục tiêu ban đầu	62
Bảng A.1	Các URL truy cập hệ thống trên môi trường local	67

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Tên tiếng Anh	Tên tiếng Việt
API	Application Programming Interface	Giao diện lập trình ứng dụng
ACID	Atomicity, Consistency, Isolation, Durability	Tính nguyên tử, nhất quán, cô lập, bền vững
CAP	Consistency, Availability, Partition Tolerance	Định lý nhất quán, khả dụng, chịu phân mảnh
CI/CD	Continuous Integration / Continuous Deployment	Tích hợp và triển khai liên tục
CNTT		Công nghệ thông tin
COD	Cash on Delivery	Thanh toán khi nhận hàng
CORS	Cross-Origin Resource Sharing	Chia sẻ tài nguyên giữa các nguồn gốc khác nhau
CQRS	Command Query Responsibility Segregation	Tách biệt trách nhiệm lệnh và truy vấn
ĐATN		Đồ án tốt nghiệp
DDD	Domain-Driven Design	Thiết kế hướng domain
DTO	Data Transfer Object	Đối tượng truyền dữ liệu
HMAC	Hash-based Message Authentication Code	Mã xác thực thông điệp dựa trên hàm băm
HTTP	HyperText Transfer Protocol	Giao thức truyền siêu văn bản
IPN	Instant Payment Notification	Thông báo thanh toán tức thì
JWT	JSON Web Token	Token web định dạng JSON
OIDC	OpenID Connect	Giao thức xác thực định danh mở
RBAC	Role-Based Access Control	Kiểm soát truy cập dựa trên vai trò
REST	Representational State Transfer	Chuyển trạng thái đại diện
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc
TMĐT		Thương mại điện tử
TTL	Time To Live	Thời gian sống của bản ghi
UUID	Universally Unique Identifier	Định danh duy nhất toàn cục

DANH MỤC THUẬT NGỮ

Thuật ngữ	Giải nghĩa
Bounded Context	Ranh giới rõ ràng trong DDD, bên trong đó mọi khái niệm domain có nghĩa nhất quán và duy nhất
Circuit Breaker	Pattern ngăn cascading failure: tự động ngắt mạch khi service downstream liên tục lỗi
Eventual Consistency	Mô hình nhất quán trong hệ phân tán: các bản sao dữ liệu sẽ hội tụ về cùng giá trị sau một khoảng thời gian
Flash Sale	Sự kiện bán hàng số lượng giới hạn trong thời gian ngắn, tạo tải đồng thời cực cao
Idempotent Consumer	Pattern đảm bảo xử lý cùng một message nhiều lần không gây side effect khác so với xử lý một lần
Microservice	Service nhỏ, độc lập, xây dựng xung quanh một Bounded Context, có thể triển khai và scale riêng biệt
Outbox Pattern	Pattern lưu event vào bảng outbox cùng transaction với business data, đảm bảo at-least-once delivery
Over-sell	Tình trạng bán nhiều hơn số lượng hàng thực có, xảy ra do race condition trong hệ thống tải cao
Rate Limiting	Kỹ thuật giới hạn số request một client gửi trong một khoảng thời gian nhất định
Saga Pattern	Pattern xử lý distributed transaction bằng chuỗi local transaction; khi lỗi thực thi compensating transaction
Service Discovery	Cơ chế tự động phát hiện địa chỉ IP:Port của service trong môi trường container động
Trusted Subsystem	Pattern: chỉ API Gateway validate JWT, forward thông tin user qua header tin cậy cho backend

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong thập kỷ qua, thương mại điện tử (TMĐT) tại Việt Nam đã chứng kiến tốc độ tăng trưởng vượt bậc. Theo báo cáo của Hiệp hội Thương mại điện tử Việt Nam (VECOM) năm 2024, doanh thu TMĐT B2C ước đạt 25 tỷ USD, tăng trưởng bình quân trên 25%/năm trong giai đoạn 2020–2024. Các sàn giao dịch lớn như Shopee, Lazada, Tiki đang phục vụ hàng chục triệu người dùng đồng thời, đặc biệt trong các sự kiện Flash Sale và Black Friday — nơi lưu lượng giao dịch có thể tăng đột biến gấp 10–50 lần so với ngày thường.

Thực tế vận hành cho thấy các hệ thống TMĐT xây dựng theo kiến trúc Monolith truyền thống gặp nhiều hạn chế nghiêm trọng khi đối mặt với bài toán quy mô lớn. Thứ nhất, toàn bộ hệ thống phải được mở rộng đồng thời dù chỉ một module đang chịu tải cao, dẫn đến lãng phí tài nguyên và chi phí vận hành. Thứ hai, mọi thay đổi dù nhỏ đều yêu cầu triển khai lại toàn bộ ứng dụng, làm chậm tốc độ phát hành tính năng mới và tăng rủi ro khi cập nhật. Thứ ba, một lỗi cục bộ có thể kéo sập toàn bộ hệ thống do thiếu cơ chế cách ly sự cố (fault isolation). Đối với các nghiệp vụ đặc thù như flash sale — nơi hàng chục nghìn người dùng đồng thời cạnh tranh một số lượng hàng giới hạn trong vài phút — Monolith không thể đáp ứng được yêu cầu về khả năng mở rộng chọn lọc và xử lý đồng thời cao.

Kiến trúc microservices ra đời như một giải pháp cho những hạn chế nêu trên. Tuy nhiên, việc thiết kế và triển khai một hệ thống microservices hoàn chỉnh đòi hỏi nắm vững nhiều mẫu thiết kế phức tạp: từ Saga Pattern cho giao dịch phân tán, Transactional Outbox cho truyền thông điệp tin cậy, đến Idempotent Consumer cho xử lý phân phối at-least-once. Đây là những vấn đề được đề cập rộng rãi trong tài liệu học thuật nhưng ít dự án thực tế tại Việt Nam triển khai một cách đầy đủ và có hệ thống. Bên cạnh đó, việc tích hợp các thành phần hạ tầng như service discovery, API gateway, message broker, distributed caching, full-text search engine và identity provider vào một hệ thống vận hành thống nhất cũng là một thách thức kỹ thuật đáng kể.

Xuất phát từ thực tiễn trên, đề tài đặt ra câu hỏi nghiên cứu: *Làm thế nào để thiết kế và triển khai một hệ thống thương mại điện tử hoàn chỉnh theo kiến trúc microservices, có khả năng mở rộng độc lập theo từng miền nghiệp vụ, chịu lỗi cục bộ, đảm bảo tính nhất quán dữ liệu trong môi trường phân tán, và đặc biệt hỗ trợ tính năng flash sale chống over-sell trong điều kiện tải cao?*

1.2 Mục tiêu đề tài

Mục tiêu tổng quát

Thiết kế, triển khai và đánh giá một hệ thống thương mại điện tử đầy đủ chức năng theo kiến trúc microservices, sử dụng Spring Boot 3.5, Spring Cloud 2025 và Java 21, qua đó minh chứng tính khả thi và hiệu quả của kiến trúc này trong việc giải quyết các bài toán đặc thù của TMĐT: khả năng mở rộng chọn lọc, xử lý đồng thời cao, tính nhất quán dữ liệu phân tán và khả năng quan sát hệ thống từ đầu đến cuối.

Mục tiêu cụ thể

Đề tài đặt ra sáu mục tiêu cụ thể, mỗi mục tiêu đều có thể đo lường và kiểm chứng thông qua mã nguồn, cấu hình hệ thống và kết quả kiểm thử:

1. **Phân rã hệ thống theo Domain-Driven Design:** Phân tích và thiết kế 13 bounded context độc lập, mỗi context tương ứng với một microservice sở hữu cơ sở dữ liệu, Dockerfile và vòng đời triển khai riêng biệt. Các service bao phủ đầy đủ các miền nghiệp vụ cốt lõi của TMĐT: định danh, người dùng, sản phẩm, tồn kho, giỏ hàng, đơn hàng, thanh toán, khuyến mãi, thông báo, đánh giá, tìm kiếm, nội dung và flash sale.
2. **Áp dụng các mẫu thiết kế phân tán cốt lõi:** Triển khai bốn mẫu thiết kế quan trọng trong môi trường phân tán — Saga Orchestration (với order-service làm bộ điều phối giao dịch phân tán trong luồng đặt hàng); Transactional Outbox (đảm bảo phát hành sự kiện at-least-once ngay cả khi service gặp sự cố); Idempotent Consumer (đảm bảo hiệu ứng xử lý exactly-once ở phía nhận); và CQRS-lite (tách biệt mô hình đọc với Elasticsearch phục vụ tìm kiếm toàn văn).
3. **Tích hợp bảo mật tập trung với Keycloak:** Sử dụng Keycloak 26 làm nhà cung cấp định danh, cấu hình OAuth 2.0/OpenID Connect với JWT ký thuật toán RS256 (khóa bất đối xứng). API Gateway đóng vai trò Resource Server duy nhất theo mẫu Trusted Subsystem — xác thực tập trung, chuyển tiếp thông tin định danh qua header tin cậy cho các service backend, loại bỏ việc mỗi service phải gọi lại Keycloak để xác minh token.
4. **Tích hợp cổng thanh toán VNPAY:** Kết nối với môi trường sandbox của VNPAY, sinh URL thanh toán với chữ ký xác thực HMAC-SHA512, xử lý Return URL và IPN callback, đồng thời triển khai cơ chế tự động hủy thanh toán khi quá thời hạn 30 phút.
5. **Xây dựng cơ chế Flash Sale chống over-sell:** Thiết kế giải pháp chống tranh

chấp tài nguyên bằng Redis Lua atomic script, buyer set chống mua trùng, cơ chế bù trừ khi tạo đơn thất bại và reconciliation scheduler đối chiếu lại số đơn thực tế. Mục tiêu: trong điều kiện nhiều người dùng đồng thời, số đơn hàng thành công không vượt quá số lượng slot của chiến dịch.

6. **Xây dựng hệ thống quan sát hoàn chỉnh:** Triển khai Prometheus và Grafana cho thu thập và trực quan hóa số liệu (JVM heap, request rate, latency, error rate); Zipkin cho truy vết phân tán theo chuẩn W3C Trace Context; structured logging với khả năng liên kết traceId/spanId, cho phép giám sát và gỡ lỗi hệ thống từ đầu đến cuối.

1.3 Phạm vi đề tài

Phạm vi thực hiện

Hệ thống được xây dựng bao gồm 13 business microservice, ba Spring Cloud infrastructure service, các hệ thống hạ tầng hỗ trợ, Next.js storefront/Admin Panel và bộ kịch bản kiểm thử thực nghiệm. Toàn bộ backend và hạ tầng được đóng gói bằng Docker Compose; môi trường production-like bổ sung frontend container, Caddy reverse proxy và GitHub Actions/GHCR cho quy trình build/push image. Bảng 1.1 liệt kê chi tiết các thành phần trong phạm vi thực hiện của đề tài.

Bảng 1.1: Các thành phần trong phạm vi đề tài

Nhóm	Thành phần	Công nghệ
Spring Cloud	API Gateway	Spring Cloud Gateway
	Discovery Server	Netflix Eureka
	Config Server	Spring Cloud Config
Business Services	identity-service, user-service	Keycloak, PostgreSQL, Kafka
	product-service	PostgreSQL, Redis, Kafka
	inventory-service	PostgreSQL, Kafka
	cart-service	Redis
	order-service, payment-service	PostgreSQL, Kafka, VNPAY
	voucher-service, content-service	PostgreSQL
	notification-service	PostgreSQL, Kafka, Mailpit
	review-service	PostgreSQL, OpenFeign
	search-service	Elasticsearch, Kafka
	flash-sale-service	PostgreSQL, Redis, Kafka
Infrastructure	PostgreSQL 17, Redis 8	Dữ liệu & Cache
	Kafka KRaft 8.2	Message Broker
	Elasticsearch 8.18	Search Engine
	Keycloak 26.6, Mailpit	Xác thực & Email
	Prometheus, Grafana, Zipkin	Observability
Frontend	Storefront	Next.js App Router
	Admin Panel	Server Actions, ROLE_ADMIN guard
Kiểm thử & Triển khai	Unit/Integration Test	JUnit 5, Testcontainers
	Smoke/E2E Test	Bash scripts, Phase 14 demo
	Performance Test	k6
	Docker Compose/AWS	EC2 single-host, Caddy, GHCR

Ngoài phạm vi

Các hạng mục sau nằm ngoài phạm vi của đề tài, xuất phát từ giới hạn về thời gian, tài nguyên và định hướng nghiên cứu của đồ án tốt nghiệp:

- **Frontend cấp sản phẩm thương mại:** Đề tài đã triển khai Next.js storefront và Admin Panel phục vụ demo các luồng chính. Tuy nhiên, các năng lực production như tối ưu UX đa thiết bị, upload file thật qua object storage, workflow kiểm duyệt nâng cao và mobile app riêng chưa nằm trong phạm vi.
- **Kubernetes và Service Mesh:** Triển khai trên Kubernetes với Istio hoặc Linkerd là hướng mở rộng trong tương lai. Phạm vi hiện tại dừng ở Docker Compose single-host, phù hợp với quy mô của một đồ án tốt nghiệp.
- **CI/CD pipeline cấp sản xuất đầy đủ:** GitHub Actions đã hỗ trợ build/push image lên GitHub Container Registry và triển khai lên EC2 ở mức production-like. Phạm vi hiện tại chưa bao gồm canary/blue-green deployment, quét bảo mật bắt buộc, quản lý secret chuyên dụng và triển khai đa môi trường.
- **Thanh toán VNPAY thực tế:** Chỉ sử dụng môi trường sandbox của VNPAY để minh họa luồng thanh toán. Chưa đăng ký tài khoản merchant phục vụ giao dịch thật.
- **Logistics và hoàn tiền:** Các nghiệp vụ vận chuyển, giao hàng và hoàn tiền phức tạp nằm ngoài phạm vi backend e-commerce cốt lõi mà đề tài hướng tới.
- **Triển khai đa vùng:** Hệ thống được triển khai trên một máy ảo EC2 duy nhất tại khu vực ap-southeast-1 (Singapore), chưa hỗ trợ chuyển đổi dự phòng đa vùng địa lý.

1.4 Khảo sát hệ thống tương tự

Để có cơ sở thực tiễn cho các quyết định thiết kế, đề tài tiến hành khảo sát kiến trúc của một số hệ thống TMĐT tiêu biểu trên thị trường, cũng như các dự án mã nguồn mở có liên quan trong cộng đồng phát triển.

Các sàn thương mại điện tử lớn

Bảng 1.2 tổng hợp kết quả khảo sát kiến trúc của ba sàn TMĐT hàng đầu tại Việt Nam và Đông Nam Á, tập trung vào các khía cạnh: mô hình kiến trúc, ngôn ngữ lập trình chính, hệ quản trị cơ sở dữ liệu, message queue, giải pháp tìm kiếm và chiến lược xử lý flash sale.

Bảng 1.2: Khảo sát kiến trúc các sàn thương mại điện tử lớn

Tiêu chí	Shopee	Tiki	Lazada
Kiến trúc	Microservices (hàng trăm service)	Microservices kết hợp Monolith	Microservices (hệ sinh thái Alibaba)
Ngôn ngữ chính	Go, Python, Java	Java (Spring), Go	Java
Cơ sở dữ liệu	MySQL sharding, Redis, TiDB	PostgreSQL, MySQL, Redis, ES	MySQL, Redis, HBase
Message Queue	Kafka, RabbitMQ	Kafka	RocketMQ, Kafka
Tìm kiếm	Elasticsearch	Elasticsearch	Elasticsearch
Flash Sale	Redis + Lua + DB constraint	Redis + Queue + DB lock	Alibaba Sentinel

Từ kết quả khảo sát, có thể rút ra năm điểm chung trong kiến trúc của các sàn TMĐT lớn, qua đó củng cố cho các quyết định công nghệ của đề tài: (1) kiến trúc microservices được sử dụng để phân rã nghiệp vụ phức tạp thành các dịch vụ độc lập; (2) chiến lược polyglot persistence — kết hợp nhiều loại cơ sở dữ liệu cho các mục đích khác nhau — là cách tiếp cận phổ biến; (3) message queue đóng vai trò trung tâm trong xử lý bất đồng bộ và giao tiếp giữa các dịch vụ; (4) Elasticsearch là lựa chọn thống nhất cho bài toán tìm kiếm toàn văn; và (5) Redis là thành phần then chốt trong kiến trúc xử lý flash sale, thường được kết hợp với các cơ chế bảo vệ ở tầng cơ sở dữ liệu.

Dự án tham khảo

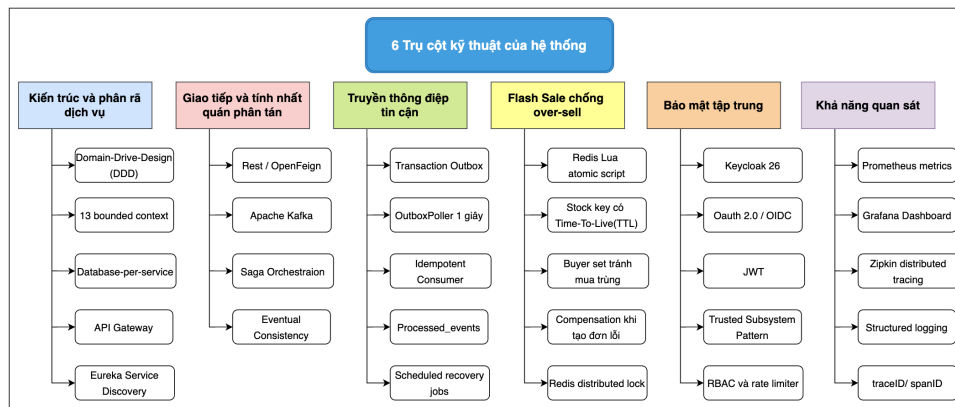
Bên cạnh các hệ thống thương mại thực tế, đề tài cũng tham khảo một số dự án mã nguồn mở triển khai microservices với Spring Boot:

- **Spring PetClinic Microservices:** Dự án chính thức của Spring Team minh họa quá trình tách một ứng dụng monolith thành các microservices sử dụng Spring Cloud. Đây là nguồn tham khảo chính về cấu trúc dự án Maven multi-module, cấu hình Spring Cloud và tổ chức Docker Compose cho môi trường phát triển cục bộ.
- **AxonBank (Axon Framework):** Dự án minh họa việc áp dụng CQRS và Event Sourcing với Axon Framework trên nền Spring Boot, cung cấp tài liệu tham khảo giá trị về thiết kế hướng sự kiện và cách tổ chức luồng xử lý command/query.
- **Các dự án E-Commerce Microservices trên GitHub:**
Nhiều dự án e-commerce mã nguồn mở trên GitHub cung cấp ý tưởng tham

khảo về cấu trúc thư mục dịch vụ, thiết kế REST API và dockerization. Tuy nhiên, hầu hết các dự án này chưa triển khai đầy đủ các mẫu thiết kế phân tán phức tạp như Saga, Transactional Outbox hay cơ chế atomic counter cho flash sale — những điểm mà đề tài này hướng tới giải quyết một cách trọn vẹn.

1.5 Định hướng giải pháp

Dựa trên phân tích bài toán, khảo sát thực tiễn và nghiên cứu lý thuyết, đề tài xác định sáu định hướng kỹ thuật then chốt, chi phối toàn bộ quá trình thiết kế và triển khai hệ thống.



Hình 1.1: Sơ đồ tổng quan sáu trụ cột kỹ thuật của hệ thống

Kiến trúc và phân rã dịch vụ. Áp dụng phương pháp Domain-Driven Design với Bounded Context làm đơn vị phân rã, hệ thống được chia thành 13 microservice tương ứng với 13 miền nghiệp vụ độc lập. Mỗi service sở hữu cơ sở dữ liệu riêng và không được phép truy cập trực tiếp vào dữ liệu của service khác — mọi tương tác dữ liệu phải thông qua API hoặc sự kiện. API Gateway đóng vai trò điểm vào duy nhất cho mọi yêu cầu từ phía client, kết hợp với Eureka Service Discovery để định tuyến động và Config Server để quản lý cấu hình tập trung, đảm bảo tính nhất quán về mặt vận hành trên toàn hệ thống.

Giao tiếp và tính nhất quán phân tán. Hệ thống kết hợp hai phương thức giao tiếp: REST + OpenFeign cho các truy vấn đồng bộ cần phản hồi tức thời (xác minh giá sản phẩm, kiểm tra tồn kho) và Apache Kafka KRaft cho các tương tác bất đồng bộ không yêu cầu phản hồi ngay (cập nhật trạng thái đơn hàng, gửi thông báo). Saga Orchestration Pattern được áp dụng để quản lý giao dịch phân tán trong luồng đặt hàng, với order-service đóng vai trò bộ điều phối trung tâm, đảm bảo tính nhất quán cuối cùng thay vì two-phase commit. Toàn bộ 13 Kafka topic được thiết kế rõ ràng về producer, consumer và cấu trúc payload.

Truyền thông điệp tin cậy. Transactional Outbox Pattern được triển khai tại order-service và payment-service để giải quyết bài toán dual-write: sự kiện được

ghi vào bảng `outbox` hoặc `payment_outbox` trong cùng một giao dịch cơ sở dữ liệu với dữ liệu nghiệp vụ. `OutboxPoller` (chu kỳ quét 1 giây) định kỳ đọc các sự kiện chưa được phát hành bằng truy vấn `FOR UPDATE SKIP LOCKED` và gửi lên Kafka, đảm bảo `at-least-once delivery` ngay cả khi service gặp sự cố giữa chừng. Phía nhận, `Idempotent Consumer` sử dụng bảng `processed_events` để phát hiện và loại bỏ sự kiện trùng lặp, đạt được hiệu ứng `exactly-once processing` ở tầng ứng dụng. Sáu `scheduled job` chạy nền đảm bảo hệ thống tự phục hồi sau các tình huống bất thường: `ReservationExpiryScheduler`, `PaymentTimeoutScheduler`, `OutboxPoller` cho `order` và `payment`, `CampaignScheduler` và `ReconciliationScheduler`.

Flash sale chống over-sell. Giải pháp được thiết kế theo mô hình phòng thủ đa lớp: lớp thứ nhất sử dụng Redis Lua atomic script để thực hiện thao tác kiểm tra `buyer set`, kiểm tra tồn kho và giảm slot trong bộ nhớ một cách nguyên tử; lớp thứ hai sử dụng logic tạo đơn và cơ chế bù trừ để hoàn slot khi quá trình tạo đơn thất bại; lớp thứ ba sử dụng `ReconciliationScheduler` để đối chiếu `soldCount` với số đơn flash-sale thực tế trong `order-service`. Cơ chế `distributed lock` dựa trên Redis đảm bảo chỉ một instance của `flash-sale-service` thực thi `CampaignScheduler` tại mỗi thời điểm, còn Redis `stock key` và `buyers key` được đặt TTL theo thời điểm kết thúc chiến dịch.

Bảo mật tập trung. Toàn bộ hoạt động xác thực và phân quyền được xử lý tại API Gateway thông qua Keycloak 26 theo mẫu `Trusted Subsystem Pattern`: Gateway là điểm duy nhất xác minh JWT (RS256), sau đó chuyển tiếp thông tin định danh đã được xác thực qua ba header tin cậy — `X-User-Id`, `X-User-Roles` và `X-User-Email` — cho các service backend. Các service nội bộ hoàn toàn tin tưởng các header này mà không cần gọi lại Keycloak, giúp giảm độ trễ và giảm tải cho Identity Provider. Phân quyền dựa trên mô hình RBAC với hai vai trò chính: `ROLE_USER` (khách hàng) và `ROLE_ADMIN` (quản trị viên). Các endpoint nhạy cảm được bảo vệ bổ sung bằng Redis token bucket rate limiter: 10 request/giây cho đặt hàng và 3 request/giây cho mua flash sale.

Khả năng quan sát. Hệ thống quan sát được xây dựng trên cả ba trụ cột. Về metrics: Prometheus định kỳ thu thập số liệu từ các endpoint `/actuator/prometheus`, Grafana trực quan hóa thông qua các dashboard được provision tự động cho JVM, Spring Boot service metrics và tổng quan Saga. Về distributed tracing: Zipkin tiếp nhận và hiển thị vết (trace) từ tất cả service theo chuẩn W3C Trace Context, mỗi request được gán một `traceId` duy nhất và mỗi đoạn xử lý trong một service là một span, cho phép tái tạo toàn bộ hành trình của request qua hệ thống. Về logging: structured logging với Logback pattern chứa `traceId` và `spanId` trong

mỗi dòng log, cho phép liên kết chéo giữa logs và traces khi cần điều tra sự cố.

1.6 Bố cục báo cáo

Báo cáo được tổ chức thành năm chương chính, một chương kết luận và hai phụ lục. Nội dung các chương được sắp xếp theo logic từ tổng quan đến chi tiết, từ lý thuyết đến thực hành, từ thiết kế đến kiểm chứng:

Chương 1 — Giới thiệu đề tài trình bày bối cảnh và câu hỏi nghiên cứu, mục tiêu, phạm vi, kết quả khảo sát hệ thống tương tự, định hướng giải pháp và bố cục của báo cáo.

Chương 2 — Cơ sở lý thuyết trình bày có hệ thống các khái niệm và nguyên lý nền tảng: kiến trúc microservices, Domain-Driven Design và Bounded Context, các mô hình giao tiếp đồng bộ và bất đồng bộ, giao dịch phân tán và Saga Pattern, Transactional Outbox và Idempotent Consumer, CQRS, Circuit Breaker, Rate Limiting, bảo mật phân tán (OAuth 2.0, OpenID Connect, JWT, Trusted Subsystem Pattern), API Gateway, Service Discovery và Observability.

Chương 3 — Giải pháp công nghệ phân tích và lý giải việc lựa chọn từng công nghệ trong hệ thống: Java 21, Maven multi-module, Spring Boot 3.5 và Spring Cloud 2025.0.0, Spring Cloud Gateway, Eureka, Config Server, OpenFeign, Apache Kafka KRaft, PostgreSQL, Redis, Elasticsearch, Keycloak và VNPAY. Chương này cũng trình bày Prometheus, Grafana, Zipkin, Mailpit, Docker Compose, Next.js, AWS EC2, Caddy và GitHub Actions/GHCR.

Chương 4 — Phân tích và Thiết kế hệ thống trình bày toàn bộ kết quả phân tích và thiết kế: yêu cầu chức năng và phi chức năng, biểu đồ use case, kiến trúc tổng thể, bản đồ ngữ cảnh 13 bounded context, thiết kế cơ sở dữ liệu, thiết kế API và giao tiếp, thiết kế luồng sự kiện Kafka, thiết kế bảo mật, máy trạng thái và biểu đồ tuần tự cho các luồng nghiệp vụ chính.

Chương 5 — Cài đặt, Triển khai, Kiểm thử và Đánh giá trình bày chi tiết quá trình hiện thực hóa thiết kế: môi trường phát triển, cấu trúc dự án, cài đặt các dịch vụ cốt lõi, frontend/Admin Panel, cấu hình Docker Compose, triển khai production-like trên AWS EC2 và quy trình CI/CD. Phần cuối chương trình bày chiến lược và kết quả kiểm thử: backend readiness, kiểm thử chức năng, hiệu năng, khả năng chịu lỗi và đánh giá tổng thể.

Kết luận và Hướng phát triển tổng kết những kết quả đã đạt được, phân tích khách quan những hạn chế còn tồn tại và đề xuất các hướng phát triển tiềm năng trong tương lai.

Phụ lục A — Hướng dẫn cài đặt và vận hành cung cấp hướng dẫn từng bước

để thiết lập môi trường phát triển, biên dịch mã nguồn, khởi động hệ thống, kiểm tra trạng thái và truy cập các endpoint của hệ thống.

Phụ lục B — Đặc tả Use Case cung cấp đặc tả chi tiết cho các use case chính của hệ thống, bao gồm điều kiện tiên quyết, luồng sự kiện chính, luồng sự kiện thay thế và điều kiện hậu kiểm.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

Chương này trình bày các khái niệm lý thuyết và mẫu thiết kế kiến trúc làm nền tảng cho các quyết định thiết kế của hệ thống. Các nội dung được sắp xếp theo trình tự từ tổng quan đến chi tiết: bắt đầu với kiến trúc microservices và phương pháp phân rã hệ thống, tiếp đến các mô hình giao tiếp và xử lý giao dịch phân tán, sau cùng là các cơ chế đảm bảo độ tin cậy, bảo mật và khả năng quan sát. Phần áp dụng các lý thuyết này vào công nghệ cụ thể sẽ được trình bày ở Chương 3.

2.1 Kiến trúc Microservices

2.1.1 Định nghĩa và đặc trưng cốt lõi

Kiến trúc microservices là một phong cách kiến trúc phần mềm, trong đó ứng dụng được xây dựng như một tập hợp các dịch vụ nhỏ, độc lập, mỗi dịch vụ chạy trong tiến trình riêng và giao tiếp với nhau thông qua các giao thức nhẹ, điển hình là HTTP/REST hoặc message queue. Khác với kiến trúc Monolith truyền thống — nơi toàn bộ ứng dụng được đóng gói thành một khối thống nhất, microservices phân rã hệ thống theo ranh giới nghiệp vụ, cho phép mỗi dịch vụ được phát triển, triển khai và mở rộng một cách độc lập.

Một microservice được thiết kế đúng chuẩn cần thỏa mãn năm đặc trưng cốt lõi. Thứ nhất, dịch vụ phải được mô hình hóa xung quanh một miền nghiệp vụ cụ thể, ranh giới của dịch vụ được xác định bởi nghiệp vụ chứ không phải bởi tầng kỹ thuật. Thứ hai, mỗi dịch vụ sở hữu cơ sở dữ liệu riêng và không chia sẻ trực tiếp dữ liệu với dịch vụ khác — mọi truy cập dữ liệu chéo phải thông qua API hoặc sự kiện được công bố công khai. Thứ ba, mỗi dịch vụ có thể được triển khai độc lập mà không ảnh hưởng đến hoạt động của các dịch vụ còn lại. Thứ tư, hệ thống được phân quyền — không tồn tại điểm kiểm soát tập trung duy nhất, logic nghiệp vụ và dữ liệu được phân tán trên nhiều dịch vụ. Thứ năm, tổ chức nhóm phát triển cần phản ánh cấu trúc hệ thống, trong đó mỗi nhóm sở hữu và chịu trách nhiệm với một hoặc một vài dịch vụ.

2.1.2 So sánh với Monolith và SOA

Để hiểu rõ vị trí của microservices trong phổ kiến trúc phần mềm, Bảng 2.1 so sánh ba mô hình kiến trúc chính trên các tiêu chí thiết kế quan trọng.

Bảng 2.1: So sánh ba mô hình kiến trúc phần mềm

Tiêu chí	Monolith	SOA	Microservices
Cơ sở dữ liệu	Dùng chung	Thường dùng chung	Cơ sở dữ liệu riêng
Giao tiếp	Lời gọi nội bộ	Enterprise Service Bus	REST / Message Queue
Triển khai	Toàn bộ ứng dụng	Từng dịch vụ	Độc lập từng dịch vụ
Mở rộng	Theo chiều dọc	Theo dịch vụ	Theo chiều ngang, từng dịch vụ
Cách ly sự cố	Không có	Một phần	Tốt (circuit breaker, bulkhead)
Độ phức tạp	Thấp ban đầu	Cao (phức tạp ESB)	Cao (hệ thống phân tán)
Phù hợp khi	Nhóm nhỏ, sản phẩm MVP	Doanh nghiệp lớn, kế thừa	Nhóm lớn, cần mở rộng độc lập

Kiến trúc Monolith phù hợp với giai đoạn đầu của sản phẩm khi quy mô nhỏ và tốc độ phát triển được ưu tiên. Tuy nhiên, khi hệ thống phát triển, Monolith bộc lộ các hạn chế về khả năng mở rộng chọn lọc, tốc độ triển khai và cách ly sự cố. SOA ra đời nhằm giải quyết một phần các hạn chế này thông qua việc tách hệ thống thành các dịch vụ, nhưng phụ thuộc nặng vào Enterprise Service Bus (ESB) — một thành phần trung tâm dễ trở thành điểm nghẽn và điểm lỗi đơn. Microservices khắc phục nhược điểm của SOA bằng cách loại bỏ ESB, chuyển trí thông minh từ đường ống truyền dẫn sang các điểm đầu cuối, đồng thời trao cho mỗi dịch vụ toàn quyền sở hữu dữ liệu của mình.

2.1.3 Tám ngộ nhận trong tính toán phân tán

Khi chuyển từ Monolith sang microservices, hệ thống trở thành một hệ phân tán và lập tức phải đối mặt với tám ngộ nhận nguy hiểm đã được tổng kết từ thực tiễn công nghiệp: (1) mạng luôn đáng tin cậy; (2) độ trễ bằng không; (3) băng thông vô hạn; (4) mạng an toàn tuyệt đối; (5) topology mạng không thay đổi; (6) chỉ có một quản trị viên duy nhất; (7) chi phí truyền tải bằng không; (8) mạng đồng nhất. Mỗi ngộ nhận này đặt ra một yêu cầu thiết kế cụ thể cho hệ thống microservices: cần cơ chế thử lại và ngắt mạch để đối phó với mạng không đáng tin cậy; cần khám phá dịch vụ động để thích ứng với topology thay đổi; cần quản lý cấu hình tập trung và hệ thống quan sát để vận hành hiệu quả môi trường đa dịch vụ.

2.2 Domain-Driven Design và phân rã hệ thống

Domain-Driven Design (DDD) là phương pháp thiết kế phần mềm lấy lĩnh vực nghiệp vụ làm trung tâm, đặc biệt phù hợp cho việc xác định ranh giới giữa các microservice trong một hệ thống phức tạp. DDD cung cấp một bộ công cụ khái niệm giúp đội ngũ phát triển và chuyên gia nghiệp vụ cùng xây dựng một mô hình

chung, nhất quán về hệ thống.

Khái niệm trung tâm của DDD là **Bounded Context** — một ranh giới tường minh bên trong đó một mô hình nghiệp vụ cụ thể được định nghĩa và áp dụng một cách nhất quán. Trong mỗi Bounded Context, mọi thuật ngữ đều có một ý nghĩa duy nhất, rõ ràng, được thống nhất giữa đội ngũ kỹ thuật và đội ngũ nghiệp vụ thông qua **Ubiquitous Language** (ngôn ngữ chung). Ví dụ, khái niệm "Đơn hàng" (Order) trong ngữ cảnh đặt hàng mang ý nghĩa về thông tin khách hàng, danh sách sản phẩm và tổng giá trị; trong khi đó, "Đơn hàng" trong ngữ cảnh kho hàng lại mang ý nghĩa về danh sách mặt hàng cần picking, packing và shipping. Hai cách hiểu này thuộc về hai Bounded Context khác nhau và do đó nên được triển khai trong hai microservice riêng biệt.

DDD cũng định nghĩa **Aggregate** là một nhóm các thực thể (entity) và đối tượng giá trị (value object) có liên quan chặt chẽ với nhau, được xử lý như một đơn vị nhất quán duy nhất. Mỗi Aggregate có một **Aggregate Root** — thực thể gốc đóng vai trò là điểm truy cập duy nhất từ bên ngoài vào Aggregate. Mọi thao tác thay đổi trạng thái bên trong Aggregate đều phải đi qua Aggregate Root, đảm bảo các ràng buộc nhất quán (invariant) không bị vi phạm. Trong thiết kế microservices, ranh giới của một Aggregate thường cũng chính là ranh giới của một giao dịch cơ sở dữ liệu — một transaction chỉ nên thao tác trên một Aggregate duy nhất.

Context Map là biểu đồ thể hiện mối quan hệ giữa các Bounded Context trong toàn hệ thống, bao gồm các mẫu quan hệ như: Shared Kernel (chia sẻ một phần mô hình), Customer-Supplier (một bên cung cấp, một bên tiêu thụ), Conformist (một bên tuân theo mô hình của bên kia), Anti-corruption Layer (lớp chống nhiễu, bảo vệ mô hình nội bộ khỏi ảnh hưởng từ bên ngoài) và Open Host Service (dịch vụ công khai cho nhiều đối tượng sử dụng). Context Map là công cụ quan trọng giúp hình dung toàn bộ bức tranh tích hợp giữa các dịch vụ.

2.3 Các mô hình giao tiếp trong Microservices

2.3.1 Giao tiếp đồng bộ và bất đồng bộ

Trong hệ thống microservices, giao tiếp giữa các dịch vụ được phân thành hai mô hình chính với những đặc điểm và đánh đổi khác biệt. Việc lựa chọn mô hình giao tiếp phù hợp cho từng tương tác cụ thể là một trong những quyết định thiết kế quan trọng nhất.

Bảng 2.2: So sánh hai mô hình giao tiếp trong microservices

Tiêu chí	Đồng bộ (REST/gRPC)	Bất đồng bộ (Message Queue)
Tính kết dính	Chặt về mặt thời gian — bên gọi phải chờ phản hồi	Lỏng — bên gửi không cần biết bên nhận
Xử lý lỗi	Bên gọi bị ảnh hưởng trực tiếp nếu bên được gọi lỗi	Bên gửi không bị ảnh hưởng trực tiếp
Độ trễ	Thấp cho từng yêu cầu đơn lẻ	Có thêm độ trễ do đi qua trung gian message queue
Tình huống sử dụng	Truy vấn cần kết quả ngay (kiểm tra giá, tồn kho)	Tác động phụ không cần kết quả ngay (gửi email, index)
Khả năng mở rộng	Bị giới hạn bởi bên được gọi	Có thể mở rộng độc lập cả bên gửi và bên nhận

Giao tiếp đồng bộ phù hợp với các tình huống client cần phản hồi ngay lập tức để tiếp tục xử lý, ví dụ như kiểm tra tính hợp lệ của sản phẩm trước khi thêm vào giỏ hàng, hoặc xác minh số dư voucher trước khi áp dụng. Tuy nhiên, giao tiếp đồng bộ tạo ra sự phụ thuộc về mặt thời gian giữa các dịch vụ — nếu dịch vụ được gọi gặp sự cố hoặc chậm, dịch vụ gọi cũng bị ảnh hưởng theo hiệu ứng dây chuyền.

Giao tiếp bất đồng bộ thông qua message queue giải quyết vấn đề này bằng cách tách rời bên gửi và bên nhận về mặt thời gian. Bên gửi công bố sự kiện lên message broker và tiếp tục xử lý mà không cần chờ phản hồi; bên nhận tiêu thụ sự kiện khi có thể. Mô hình này đặc biệt phù hợp cho các tác động phụ như gửi email xác nhận đơn hàng, cập nhật chỉ mục tìm kiếm, hoặc đồng bộ dữ liệu giữa các dịch vụ.

2.3.2 Mức độ đảm bảo phân phối thông điệp

Trong giao tiếp bất đồng bộ, mức độ đảm bảo phân phối thông điệp là một yếu tố thiết kế quan trọng, được phân thành ba cấp độ. **At-most-once** nghĩa là thông điệp có thể bị mất nhưng không bao giờ bị trùng lặp — phù hợp với các tác vụ không quan trọng như cập nhật số liệu thống kê. **At-least-once** đảm bảo thông điệp luôn được xử lý ít nhất một lần, nhưng có thể bị trùng lặp trong trường hợp có sự cố mạng hoặc broker — đây là mức đảm bảo phổ biến nhất. **Exactly-once** là mức lý tưởng, đảm bảo mỗi thông điệp được xử lý đúng một lần duy nhất. Trong thực tế, exactly-once thường không được cung cấp bởi bản thân message broker mà được hiện thực ở tầng ứng dụng bằng cách kết hợp cơ chế at-least-once delivery với Idempotent Consumer ở phía nhận.

2.4 Giao dịch phân tán và tính nhất quán dữ liệu

2.4.1 Định lý CAP và nguyên lý PACELC

Trong môi trường cơ sở dữ liệu phân tán, Định lý CAP phát biểu rằng một hệ thống không thể đồng thời đảm bảo cả ba thuộc tính: Tính nhất quán (Consistency)

— mọi thao tác đọc đều trả về kết quả của thao tác ghi gần nhất; Tính khả dụng (Availability) — mọi yêu cầu đều nhận được phản hồi (thành công hoặc thất bại); và Khả năng chịu phân mảnh (Partition Tolerance) — hệ thống tiếp tục hoạt động khi mạng bị phân mảnh. Vì phân mảnh mạng là điều không thể tránh khỏi trong thực tế, mọi hệ thống phân tán buộc phải chọn giữa CP (ưu tiên nhất quán, hy sinh khả dụng khi có phân mảnh) và AP (ưu tiên khả dụng, chấp nhận nhất quán cuối cùng). Hệ thống thương mại điện tử thường lựa chọn mô hình AP với eventual consistency — dữ liệu có thể tạm thời không nhất quán giữa các bản sao, nhưng sẽ hội tụ về cùng một giá trị sau một khoảng thời gian nhất định.

Nguyên lý PACELC mở rộng CAP bằng cách chỉ ra rằng ngay cả khi không có phân mảnh mạng, hệ thống vẫn phải đối mặt với sự đánh đổi giữa Độ trễ (Latency) và Tính nhất quán (Consistency). Điều này có ý nghĩa thực tiễn quan trọng: trong điều kiện vận hành bình thường, nhà thiết kế vẫn phải quyết định ưu tiên tốc độ phản hồi hay độ chính xác tuyệt đối của dữ liệu.

2.4.2 Saga Pattern

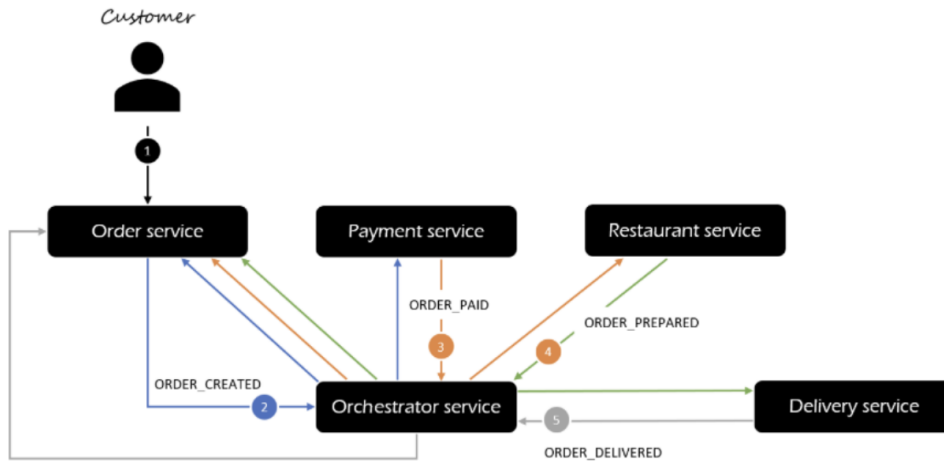
Trong kiến trúc microservices với nguyên tắc mỗi dịch vụ sở hữu cơ sở dữ liệu riêng, không thể sử dụng giao dịch ACID truyền thống để đảm bảo tính nhất quán xuyên suốt nhiều dịch vụ. Saga Pattern giải quyết bài toán này bằng cách phân rã một giao dịch phân tán lớn thành một chuỗi các giao dịch cục bộ tuần tự $T_1, T_2, \dots, T_n$, mỗi giao dịch được thực thi trong phạm vi một dịch vụ duy nhất và có thể commit hoặc rollback độc lập. Điểm mấu chốt của Saga là cơ chế bù trừ (compensation): nếu một giao dịch T_i trong chuỗi thất bại, hệ thống sẽ thực thi tuần tự các giao dịch bù trừ $C_{i-1}, \dots, C_1$ theo thứ tự ngược để hoàn tác các thay đổi đã được commit trước đó, đưa hệ thống về trạng thái nhất quán.

Saga có hai biến thể triển khai với những đặc điểm khác biệt rõ rệt:

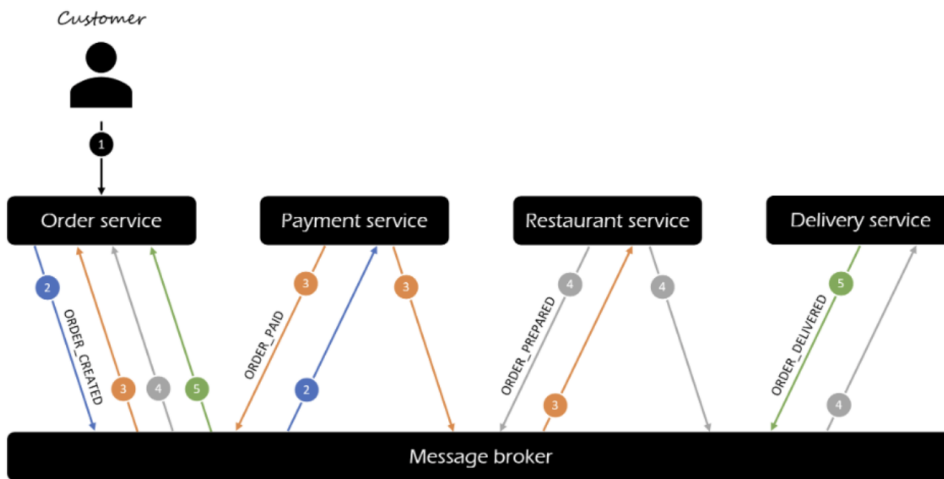
Bảng 2.3: So sánh Saga Choreography và Saga Orchestration

Đặc điểm	Choreography	Orchestration
Cơ chế	Mỗi dịch vụ tự quyết định hành động dựa trên sự kiện	Bộ điều phối trung tâm ra lệnh tuần tự
Tính kết dính	Lỏng — các dịch vụ không biết về nhau	Chặt hơn — bộ điều phối nắm toàn bộ luồng
Khả năng theo dõi	Khó — luồng xử lý phân tán khắp các dịch vụ	Dễ — trạng thái tập trung tại bộ điều phối
Xử lý bù trừ	Phức tạp — khó xác định ai kích hoạt bù trừ	Bộ điều phối tự xử lý bù trừ
Thêm bước mới	Cần sửa đổi nhiều dịch vụ	Chỉ cần sửa đổi bộ điều phối
Phù hợp khi	Luồng đơn giản, ít bước	Luồng phức tạp, nhiều bước, cần kiểm soát tập trung

Trong Choreography, mỗi dịch vụ tự lắng nghe sự kiện từ dịch vụ trước đó và quyết định hành động tiếp theo — giống như một vũ công tự ứng biến theo nhạc. Trong Orchestration, một bộ điều phối trung tâm (orchestrator) chịu trách nhiệm điều phối toàn bộ luồng, gửi lệnh tuần tự đến từng dịch vụ và xử lý kết quả trả về — giống như một nhạc trưởng chỉ huy dàn nhạc. Đối với các luồng nghiệp vụ phức tạp, nhiều bước và yêu cầu khả năng theo dõi, kiểm soát tập trung như luồng đặt hàng trong thương mại điện tử, Orchestration thường là lựa chọn phù hợp hơn.



Hình 2.1: Saga Orchestration Pattern



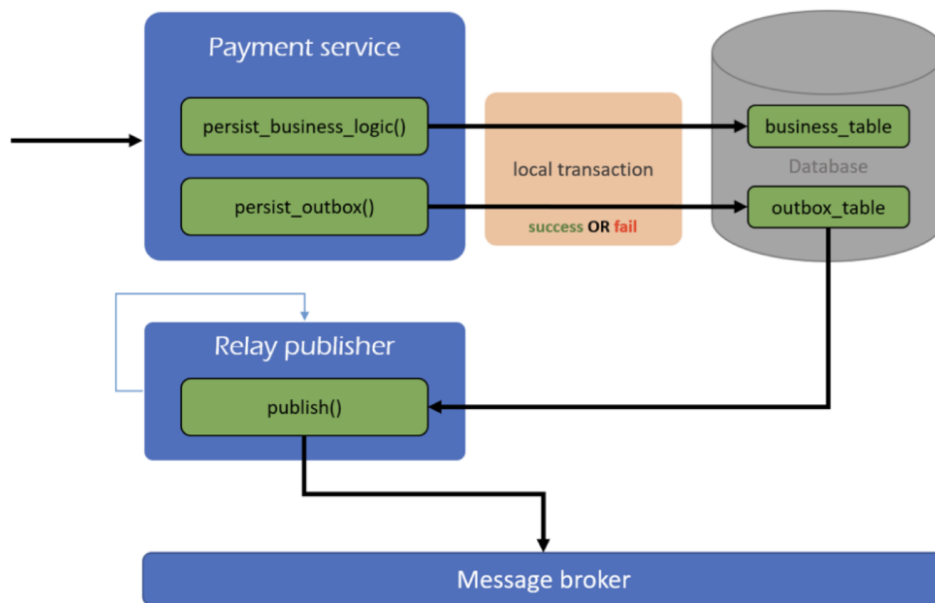
Hình 2.2: Saga Choreography Pattern

2.4.3 Transactional Outbox Pattern

Một trong những thách thức kỹ thuật quan trọng nhất khi kết hợp cơ sở dữ liệu quan hệ với message broker là bài toán dual-write: làm thế nào để đảm bảo thao tác ghi dữ liệu nghiệp vụ vào cơ sở dữ liệu và thao tác phát hành sự kiện lên message broker là nguyên tử — cả hai cùng thành công hoặc cùng thất bại. Trong cách tiếp cận thông thường, nếu dịch vụ ghi dữ liệu thành công nhưng gặp sự cố trước khi kịp phát hành sự kiện, các dịch vụ khác sẽ không bao giờ nhận được thông báo về thay đổi, dẫn đến mất đồng bộ dữ liệu.

Transactional Outbox Pattern giải quyết bài toán này bằng một ý tưởng đơn giản nhưng hiệu quả: thay vì phát hành sự kiện trực tiếp lên message broker, dịch vụ ghi bản ghi sự kiện vào một bảng `outbox_events` trong cùng một giao dịch cơ sở dữ liệu với dữ liệu nghiệp vụ. Vì cả hai thao tác ghi đều nằm trong cùng một giao dịch ACID, chúng được đảm bảo là nguyên tử — hoặc cùng thành công, hoặc cùng thất bại. Một tiến trình nền độc lập (Outbox Poller) định kỳ quét bảng outbox, đọc các sự kiện chưa được phát hành và gửi chúng lên message broker. Sau khi phát hành thành công, bản ghi được đánh dấu là đã gửi.

Cơ chế này đảm bảo at-least-once delivery: nếu dịch vụ gặp sự cố trước khi Outbox Poller kịp phát hành sự kiện, sau khi khởi động lại, Poller sẽ quét lại bảng outbox và phát hành các sự kiện còn tồn đọng. Trong trường hợp Poller phát hành thành công nhưng gặp sự cố trước khi kịp đánh dấu đã gửi, sự kiện có thể bị phát hành trùng lặp — và đây chính là lý do cần đến Idempotent Consumer ở phía nhận.



Hình 2.3: Luồng hoạt động của Transactional Outbox Pattern

2.4.4 Idempotent Consumer

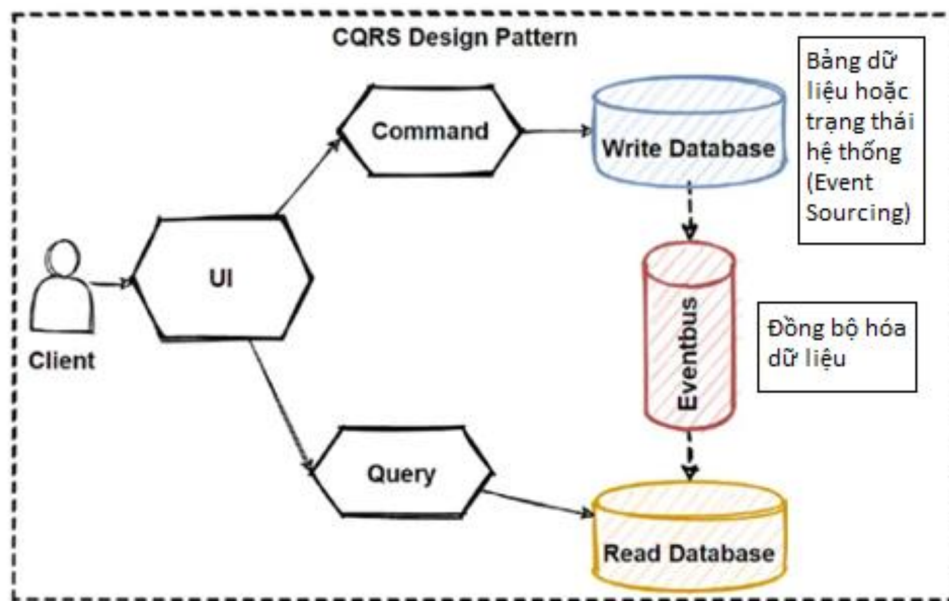
Với cơ chế at-least-once delivery, message broker có thể phân phối cùng một thông điệp nhiều lần cho consumer — điều này xảy ra khi consumer xử lý thông điệp thành công nhưng không kịp gửi tín hiệu xác nhận (acknowledgment) về broker trước khi gặp sự cố, hoặc khi Outbox Poller phát hành trùng lặp. Idempotent Consumer là mẫu thiết kế giải quyết vấn đề này: đảm bảo rằng việc xử lý cùng một thông điệp nhiều lần không tạo ra tác động phụ khác biệt so với việc chỉ xử lý một lần.

Cơ chế triển khai phổ biến là sử dụng bảng `processed_events` trong cơ sở

dữ liệu của consumer. Trước khi xử lý một thông điệp, consumer kiểm tra xem mã định danh của thông điệp (`event_id`) đã tồn tại trong bảng này hay chưa. Nếu đã tồn tại, thông điệp bị bỏ qua. Nếu chưa, consumer thực thi nghiệp vụ và ghi `event_id` vào bảng `processed_events` trong cùng một giao dịch, đảm bảo tính nguyên tử giữa việc xử lý nghiệp vụ và việc ghi nhận đã xử lý. Sự kết hợp giữa Transactional Outbox ở phía producer và Idempotent Consumer ở phía consumer tạo ra hiệu ứng exactly-once processing ở tầng ứng dụng, ngay cả khi hạ tầng bên dưới chỉ đảm bảo at-least-once.

2.4.5 CQRS — Phân tách trách nhiệm Lệnh và Truy vấn

CQRS (Command Query Responsibility Segregation) là mẫu thiết kế tách biệt hoàn toàn mô hình ghi (Command) và mô hình đọc (Query) của hệ thống. Mô hình ghi được tối ưu cho tính nhất quán và toàn vẹn dữ liệu, thường sử dụng cơ sở dữ liệu quan hệ với các ràng buộc chặt chẽ. Mô hình đọc được tối ưu cho hiệu năng truy vấn, có thể được phi chuẩn hóa (denormalize) hoặc lưu trữ trên một loại cơ sở dữ liệu hoàn toàn khác, phù hợp hơn với nhu cầu tìm kiếm và hiển thị.



Hình 2.4: Sơ đồ kiến trúc CQRS với mô hình ghi và mô hình đọc riêng biệt

Hai mô hình được đồng bộ thông qua luồng sự kiện: khi có thay đổi ở phía ghi, một sự kiện được phát hành và phía đọc cập nhật mô hình của mình một cách bất đồng bộ. Điều này đồng nghĩa với việc chấp nhận eventual consistency — dữ liệu phía đọc có thể tạm thời chưa phản ánh thay đổi mới nhất, nhưng sẽ hội tụ trong thời gian ngắn. Trong phạm vi hẹp hơn, CQRS-lite áp dụng nguyên tắc này cho một chức năng cụ thể thay vì toàn bộ hệ thống — ví dụ, sử dụng Elasticsearch làm mô hình đọc riêng cho chức năng tìm kiếm toàn văn, trong khi các chức năng khác vẫn dùng chung mô hình đọc-ghi truyền thống.

2.5 Các mẫu thiết kế đảm bảo độ tin cậy

2.5.1 Circuit Breaker

Trong kiến trúc microservices, một dịch vụ thường phụ thuộc vào nhiều dịch vụ khác để hoàn thành chức năng của mình. Khi một dịch vụ phụ thuộc gặp sự cố hoặc phản hồi chậm, nếu không có cơ chế bảo vệ, dịch vụ gọi sẽ tiếp tục gửi yêu cầu và chờ đợi trong vô vọng, dẫn đến cạn kiệt tài nguyên (thread pool, connection pool) và lan truyền sự cố sang các dịch vụ khác — hiện tượng này được gọi là cascading failure (sự cố dây chuyền).

Circuit Breaker Pattern hoạt động tương tự như cầu dao điện trong mạch điện: nó giám sát tần suất lỗi của các lời gọi đến dịch vụ phụ thuộc và tự động "ngắt mạch" khi tỷ lệ lỗi vượt quá ngưỡng cho phép. Pattern vận hành qua ba trạng thái. Ở trạng thái **CLOSED**, mọi yêu cầu được chuyển tiếp bình thường và bộ đếm lỗi được duy trì. Khi tỷ lệ lỗi vượt ngưỡng cấu hình, mạch chuyển sang trạng thái **OPEN** — mọi yêu cầu bị từ chối ngay lập tức mà không gọi đến dịch vụ phụ thuộc, thay vào đó trả về kết quả dự phòng (fallback). Sau một khoảng thời gian chờ nhất định, mạch chuyển sang trạng thái **HALF_OPEN**: một số lượng giới hạn yêu cầu được phép gọi thực tế đến dịch vụ phụ thuộc để kiểm tra. Nếu các yêu cầu này thành công, mạch trở về CLOSED; nếu thất bại, mạch quay lại OPEN và tiếp tục chờ.



Hình 2.5: Sơ đồ trạng thái của Circuit Breaker Pattern

2.5.2 Rate Limiting

Rate Limiting là kỹ thuật giới hạn số lượng yêu cầu mà một client có thể gửi đến hệ thống trong một khoảng thời gian nhất định. Trong bối cảnh thương mại điện tử, rate limiting phục vụ ba mục đích chính: bảo vệ hệ thống khỏi các cuộc tấn công từ chối dịch vụ (DoS/DDoS); ngăn chặn hành vi lạm dụng như bot đặt hàng

tự động, quét giá sản phẩm; và đảm bảo công bằng giữa những người dùng trong các sự kiện giới hạn như flash sale.

Thuật toán Token Bucket là một trong những cơ chế rate limiting phổ biến và linh hoạt nhất. Mỗi client được gán một "thùng" (bucket) có sức chứa tối đa B token. Token được thêm vào thùng với tốc độ không đổi r token mỗi giây, cho đến khi thùng đầy. Mỗi yêu cầu đến tiêu thụ một token từ thùng; nếu thùng rỗng, yêu cầu bị từ chối. Cơ chế này cho phép xử lý các đợt bùng nổ ngắn hạn (burst) — client có thể gửi nhanh B yêu cầu liên tiếp miễn là đã tích lũy đủ token, nhưng không thể duy trì tốc độ cao hơn r trong dài hạn. Điều này đặc biệt phù hợp với các API thương mại điện tử, nơi người dùng thường thao tác nhanh trong thời gian ngắn rồi nghỉ.

2.6 Bảo mật trong hệ thống phân tán

2.6.1 OAuth 2.0 — Khung ủy quyền tiêu chuẩn

OAuth 2.0 là một khung ủy quyền (authorization framework) được tiêu chuẩn hóa, cho phép một ứng dụng bên thứ ba (Client) truy cập vào tài nguyên được bảo vệ trên một máy chủ tài nguyên (Resource Server) thay mặt cho người dùng (Resource Owner), mà không cần Client biết thông tin xác thực của người dùng. Mô hình này hoạt động thông qua bốn vai trò: Resource Owner (người dùng sở hữu dữ liệu), Client (ứng dụng muốn truy cập), Authorization Server (máy chủ cấp token ủy quyền) và Resource Server (máy chủ lưu trữ tài nguyên được bảo vệ).

Luồng hoạt động cơ bản: Client yêu cầu ủy quyền từ Resource Owner, nhận được một Grant (mã ủy quyền). Client sau đó đổi Grant này lấy Access Token từ Authorization Server. Access Token được gửi kèm trong mỗi yêu cầu đến Resource Server để chứng minh quyền truy cập. Các grant type chính bao gồm: Authorization Code (dành cho ứng dụng web có backend), Client Credentials (dành cho giao tiếp máy-với-máy, không có người dùng), Authorization Code với PKCE (dành cho ứng dụng di động và Single Page Application).

2.6.2 OpenID Connect — Lớp xác thực định danh

OpenID Connect (OIDC) được xây dựng như một lớp định danh trên nền OAuth 2.0, bổ sung khả năng xác thực người dùng (authentication) — điều mà bản thân OAuth 2.0 không giải quyết. Trong khi OAuth 2.0 trả lời câu hỏi "ứng dụng được phép làm gì?" (ủy quyền), OIDC bổ sung câu hỏi "người dùng là ai?" (xác thực). OIDC bổ sung một loại token mới gọi là ID Token — một JWT chứa các thông tin định danh của người dùng như mã định danh, địa chỉ email, tên hiển thị và các vai trò.

Sự phân biệt giữa Access Token và ID Token mang ý nghĩa thiết kế quan trọng: Access Token được gửi đến Resource Server để ủy quyền truy cập API; ID Token được gửi đến Client để xác định danh tính người dùng và thiết lập phiên làm việc. Trong thực tế, một ứng dụng web thường nhận cả hai loại token: Access Token để gọi API backend, ID Token để hiển thị thông tin người dùng trên giao diện.

2.6.3 JSON Web Token (JWT)

JWT là một tiêu chuẩn mở định nghĩa cách thức truyền thông tin an toàn giữa các bên dưới dạng một đối tượng JSON được ký số. JWT bao gồm ba phần, được mã hóa base64url và phân tách bởi dấu chấm: Header chứa thông tin về thuật toán ký; Payload chứa các claims — những khẳng định về thực thể (người dùng, ứng dụng) và các siêu dữ liệu như thời điểm phát hành, thời điểm hết hạn; Signature là chữ ký số được tạo ra bằng cách ký tổ hợp Header và Payload với một khóa bí mật hoặc khóa riêng.

JWT cho phép xác thực phi trạng thái (stateless authentication): server không cần duy trì session, chỉ cần xác minh chữ ký của token là có thể tin tưởng vào nội dung bên trong. Hai chế độ ký chính được sử dụng: ký đối xứng (HS256) — cùng một khóa bí mật được dùng để ký và xác minh, phù hợp với hệ thống đơn dịch vụ nhưng không phù hợp với môi trường đa dịch vụ vì tất cả dịch vụ phải chia sẻ cùng một khóa; ký bất đối xứng (RS256) — khóa riêng để ký, khóa công khai để xác minh, cho phép mỗi dịch vụ xác minh token độc lập mà không cần biết khóa riêng, rất phù hợp với kiến trúc microservices.

2.6.4 Trusted Subsystem Pattern

Trong kiến trúc microservices, có hai cách tiếp cận để các dịch vụ backend xác định danh tính người dùng từ một request. Cách thứ nhất: mỗi dịch vụ tự xác minh JWT với Authorization Server — cách này tiêu tốn tài nguyên do mỗi request phải thực hiện thêm một lời gọi mạng, đồng thời tạo ra sự phụ thuộc mạnh giữa tất cả dịch vụ và Authorization Server. Cách thứ hai — Trusted Subsystem Pattern: chỉ có API Gateway thực hiện việc xác minh JWT, sau đó chuyển tiếp thông tin định danh đã được xác thực (mã người dùng, vai trò, email) qua các HTTP header tin cậy như X-User-Id, X-User-Roles và X-User-Email đến các dịch vụ backend. Backend hoàn toàn tin tưởng các header này mà không cần xác minh lại JWT.

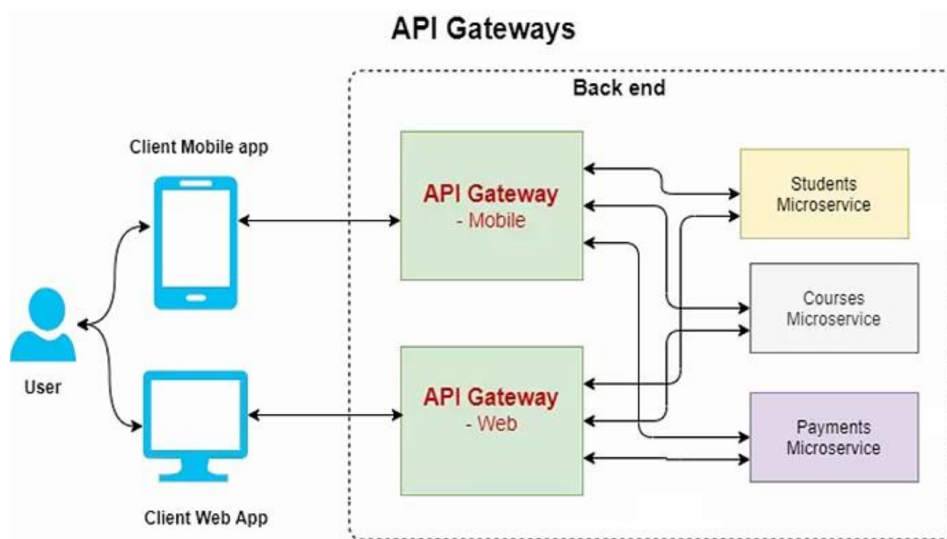
Pattern này hoạt động dựa trên một điều kiện tiên quyết quan trọng: các dịch vụ backend phải được bảo vệ ở tầng mạng, đảm bảo rằng chỉ có API Gateway mới có thể gọi trực tiếp vào các endpoint nội bộ của backend. Trong môi trường Docker Compose, điều kiện này được đảm bảo thông qua việc chỉ expose cổng của Gateway ra bên ngoài, trong khi tất cả các dịch vụ backend chỉ lắng nghe trên mạng

nội bộ của Docker.

2.7 API Gateway và Service Discovery

2.7.1 API Gateway Pattern

Trong kiến trúc microservices, mỗi dịch vụ có địa chỉ mạng và API riêng. Nếu client phải biết và giao tiếp trực tiếp với từng dịch vụ, độ phức tạp phía client sẽ tăng lên đáng kể, đặc biệt khi số lượng dịch vụ lớn và địa chỉ thay đổi liên tục trong môi trường container. Hơn nữa, các vấn đề cross-cutting như xác thực, phân quyền, rate limiting, CORS và biến đổi request/response sẽ phải được xử lý lặp lại ở mỗi dịch vụ hoặc đẩy về phía client.



Hình 2.6: Kiến trúc ứng dụng sử dụng API Gateway

API Gateway Pattern giải quyết những vấn đề này bằng cách cung cấp một điểm vào duy nhất cho tất cả client. Gateway đảm nhiệm tập trung các trách nhiệm chung: định tuyến yêu cầu đến đúng dịch vụ đích dựa trên đường dẫn, header hoặc tham số; xác thực và phân quyền tại một điểm duy nhất; áp dụng rate limiting để bảo vệ hệ thống; xử lý CORS cho các yêu cầu từ trình duyệt; và thực hiện biến đổi request/response khi cần thiết (thêm header, chuyển đổi định dạng). Mô hình này giúp giảm đáng kể độ phức tạp phía client, tạo một điểm kiểm soát bảo mật duy nhất và cho phép thay đổi kiến trúc nội bộ mà không ảnh hưởng đến client.

2.7.2 Service Discovery

Trong môi trường container động như Docker, địa chỉ IP và cổng của các dịch vụ thay đổi mỗi khi container được khởi động lại hoặc scale. Việc hardcode địa chỉ trong cấu hình là không khả thi. Service Discovery là cơ chế giải quyết vấn đề này: mỗi dịch vụ tự động đăng ký địa chỉ hiện tại của mình lên một registry trung tâm (service registry) khi khởi động, và hủy đăng ký khi dừng. Các dịch vụ khác —

bao gồm API Gateway — truy vấn registry để lấy địa chỉ thực tế của dịch vụ cần gọi.

Có hai mô hình Service Discovery chính. Client-side Discovery: client tự truy vấn registry và tự quyết định dịch vụ đích, phù hợp khi client có đủ thông tin để đưa ra quyết định. Server-side Discovery: client gửi yêu cầu đến một load balancer, load balancer truy vấn registry và chuyển tiếp yêu cầu đến một instance phù hợp — mô hình này che giấu hoàn toàn chi tiết discovery khỏi client. Trong hệ sinh thái Spring Cloud, Netflix Eureka đóng vai trò service registry, hỗ trợ cả hai mô hình và tích hợp chặt chẽ với Spring Cloud Gateway cho server-side discovery.

2.8 Khả năng quan sát trong hệ thống phân tán

Trong hệ thống microservices, một request từ client có thể đi qua hàng chục dịch vụ khác nhau trước khi hoàn thành. Khi xảy ra sự cố — một request bị chậm, thất bại hoặc trả về kết quả không mong đợi — việc xác định nguyên nhân gốc rễ trở nên cực kỳ khó khăn nếu chỉ dựa vào log của từng dịch vụ riêng lẻ. Khả năng quan sát (observability) là thuộc tính cho phép hiểu được trạng thái nội tại của hệ thống thông qua các tín hiệu đầu ra bên ngoài mà không cần biết trước câu hỏi cụ thể cần trả lời.

Observability trong hệ thống phân tán được xây dựng trên ba trụ cột bổ sung và hỗ trợ lẫn nhau:

Metrics (Số liệu) là các giá trị số được thu thập định kỳ theo thời gian, phản ánh tình trạng tổng thể của hệ thống: tỷ lệ yêu cầu (request rate), độ trễ (latency percentile), tỷ lệ lỗi (error rate), mức sử dụng CPU và bộ nhớ, số lượng kết nối cơ sở dữ liệu, độ trễ của message consumer (consumer lag). Metrics rất hiệu quả cho việc giám sát chủ động, thiết lập cảnh báo và lập kế hoạch năng lực, nhưng không cung cấp thông tin chi tiết về nguyên nhân của từng sự cố riêng lẻ.

Distributed Tracing (Truy vết phân tán) theo dõi hành trình hoàn chỉnh của một request khi nó đi qua nhiều dịch vụ. Mỗi request được gán một mã định danh duy nhất toàn cục (Trace ID), và mỗi đoạn xử lý trong một dịch vụ được ghi nhận như một Span với mã định danh riêng, thời gian bắt đầu, thời gian kết thúc và các thẻ mô tả (tags). Tập hợp tất cả Span của một Trace cho phép tái tạo chính xác đường đi của request qua hệ thống, xác định dịch vụ nào đang gây ra độ trễ và điểm nào trong luồng xử lý bị lỗi. Chuẩn W3C Trace Context đảm bảo khả năng tương tác giữa các thư viện tracing khác nhau.

Structured Logging (Ghi nhật ký có cấu trúc) ghi lại các sự kiện trong quá trình xử lý dưới dạng có cấu trúc (thường là JSON) thay vì văn bản thuần túy. Mỗi

dòng log chứa các trường được đánh dấu rõ ràng: thời gian, tên dịch vụ, mức độ nghiêm trọng, Trace ID, Span ID và nội dung thông điệp. Khi kết hợp với Trace ID từ Distributed Tracing, structured logging cho phép lọc và tổng hợp toàn bộ log của một request duy nhất xuyên suốt tất cả dịch vụ, tạo ra bức tranh hoàn chỉnh về những gì đã xảy ra trong suốt vòng đời của request.

CHƯƠNG 3. GIẢI PHÁP CÔNG NGHỆ

Chương này trình bày chi tiết các giải pháp công nghệ được lựa chọn để hiện thực hóa hệ thống. Trong khi Chương 2 đã trình bày các nguyên lý và mẫu thiết kế nền tảng, chương này tập trung trả lời câu hỏi: *với mỗi yêu cầu kỹ thuật đặt ra, công nghệ nào được chọn, vì sao và có những lựa chọn thay thế nào*. Mỗi công nghệ được phân tích theo bốn khía cạnh: vai trò trong hệ thống, lý do lựa chọn, so sánh với các giải pháp thay thế và những hạn chế cần lưu ý.

3.1 Tổng quan giải pháp công nghệ

Hệ thống được xây dựng dựa trên một tập hợp công nghệ được tuyển chọn để đáp ứng đồng thời các yêu cầu về kiến trúc microservices, xử lý phân tán, bảo mật, hiệu năng và khả năng vận hành. Ở tầng ngôn ngữ và nền tảng, Java 21 kết hợp với Spring Boot 3.5 và Maven multi-module tạo thành bộ khung phát triển chính. Ở tầng điều phối, Spring Cloud cung cấp ba thành phần cốt lõi: API Gateway (Spring Cloud Gateway), Service Registry (Netflix Eureka) và Config Server (Spring Cloud Config), cùng với OpenFeign cho giao tiếp đồng bộ giữa các dịch vụ. Ở tầng dữ liệu, hệ thống áp dụng chiến lược polyglot persistence với PostgreSQL cho dữ liệu quan hệ, Redis cho cache và thao tác nguyên tử, Elasticsearch cho tìm kiếm toàn văn. Apache Kafka đảm nhiệm vai trò message broker trung tâm cho toàn bộ giao tiếp bất đồng bộ. Keycloak cung cấp giải pháp bảo mật tập trung, VNPAY xử lý thanh toán trực tuyến. Ở tầng giao diện, Next.js được dùng để xây dựng storefront và Admin Panel. Toàn bộ hệ thống được đóng gói và vận hành thông qua Docker Compose; môi trường production-like chạy trên AWS EC2, dùng Caddy làm reverse proxy/HTTPS và GitHub Actions/GHCR cho quy trình build/-push image. Prometheus, Grafana và Zipkin tạo thành bộ công cụ quan sát toàn diện.

3.2 Ngôn ngữ và nền tảng phát triển

3.2.1 Java 21

Java 21 được chọn làm ngôn ngữ lập trình chính cho toàn bộ hệ thống. Đây là phiên bản Hỗ trợ Dài hạn (LTS) mới nhất tại thời điểm phát triển, được Oracle cam kết hỗ trợ đến năm 2031. Java 21 mang lại nhiều cải tiến đáng kể so với các phiên bản LTS trước đó: Virtual Threads (Project Loom) cho phép xử lý hàng triệu kết nối đồng thời với chi phí thấp hơn nhiều so với mô hình thread truyền thống; Pattern Matching cải thiện cú pháp làm việc với kiểu dữ liệu; Record Classes giảm đáng kể mã boilerplate cho các lớp DTO.

Đối chiếu với các ngôn ngữ JVM khác, Kotlin cung cấp cú pháp ngắn gọn hơn và null safety tốt hơn nhưng cộng đồng và hệ sinh thái nhỏ hơn đáng kể so với Java, đặc biệt trong mảng enterprise. Scala có hệ thống kiểu mạnh mẽ hơn nhưng đường cong học tập dốc và tốc độ biên dịch chậm. Trong bối cảnh đồ án tốt nghiệp với yêu cầu ổn định, tài liệu phong phú và khả năng tuyển dụng sau khi tốt nghiệp, Java 21 là lựa chọn tối ưu.

Hạn chế chính của Java trong bối cảnh microservices là thời gian khởi động tương đối chậm và mức tiêu thụ bộ nhớ cao hơn so với Go hoặc Rust. Điều này được khắc phục một phần thông qua cấu hình JVM memory cap trong Docker Compose production profile.

3.2.2 Maven Multi-Module

Toàn bộ mã nguồn được tổ chức theo cấu trúc Maven multi-module với một file `pom.xml` cha quản lý tập trung các phiên bản dependency, plugin và cấu hình build chung. Cấu trúc này mang lại ba lợi ích chính. Thứ nhất, đảm bảo tính nhất quán về phiên bản thư viện trên toàn bộ 17 module — tránh xung đột dependency khi các service sử dụng chung các thư viện như Spring Boot, Spring Cloud, Jackson và Lombok. Thứ hai, module `common` chứa các lớp dùng chung (`ApiResponse`, các lớp ngoại lệ chuẩn hóa, các DTO hợp đồng sự kiện) mà tất cả service đều phụ thuộc vào, giúp tránh trùng lặp mã nguồn và đảm bảo tính nhất quán trong giao tiếp. Thứ ba, Maven Wrapper (`mvnw`) được tích hợp vào dự án, cho phép biên dịch toàn bộ hệ thống mà không cần cài đặt Maven thủ công.

So với Gradle — một lựa chọn thay thế phổ biến với DSL linh hoạt hơn và tốc độ build nhanh hơn nhờ incremental build và build cache — Maven được chọn vì cấu trúc XML tường minh, dễ đọc và phổ biến rộng rãi trong các dự án Spring Boot truyền thống. Với quy mô 17 module của đồ án, sự khác biệt về tốc độ build giữa Maven và Gradle không đáng kể.

3.2.3 Spring Boot 3.5

Spring Boot 3.5 là nền tảng phát triển chính cho toàn bộ 16 dịch vụ Java trong hệ thống (13 business service và 3 infrastructure service). Spring Boot được chọn vì bốn lý do chính: (1) cơ chế auto-configuration loại bỏ phần lớn cấu hình thủ công, cho phép tập trung vào logic nghiệp vụ; (2) hệ sinh thái Spring phong phú cung cấp giải pháp cho hầu hết mọi nhu cầu — từ bảo mật (Spring Security), truy cập dữ liệu (Spring Data JPA, Spring Data Redis, Spring Data Elasticsearch), đến messaging (Spring Kafka) và observability (Micrometer Tracing); (3) Actuator tích hợp sẵn các endpoint giám sát và health check, là nền tảng cho toàn bộ hệ thống observability; (4) khả năng embedded server (Netty cho Gateway, Tomcat cho các

service khác) giúp đóng gói mỗi service thành một JAR tự chứa, dễ dàng container hóa.

Đối chiếu với các lựa chọn thay thế: Quarkus và Micronaut cung cấp thời gian khởi động nhanh hơn và mức tiêu thụ bộ nhớ thấp hơn nhờ cơ chế compile-time dependency injection thay vì runtime reflection, rất phù hợp cho môi trường serverless hoặc Kubernetes. Tuy nhiên, Spring Boot vượt trội về độ hoàn thiện của hệ sinh thái, chất lượng tài liệu và quy mô cộng đồng — những yếu tố quan trọng đối với một đồ án tốt nghiệp có thời gian giới hạn.

3.2.4 Thư viện hỗ trợ

Bên cạnh các framework chính, dự án sử dụng một số thư viện hỗ trợ quan trọng. Lombok giảm đáng kể mã boilerplate cho các lớp entity, DTO và configuration thông qua các annotation như `@Data`, `@Builder`, `@RequiredArgsConstructor`, cho phép tập trung vào logic nghiệp vụ thay vì viết getter, setter và constructor thủ công. Jackson đảm nhiệm việc tuần tự hóa và giải tuần tự hóa JSON trong toàn bộ hệ thống, từ request/response của REST API đến payload của Kafka message. Flyway quản lý phiên bản schema database thông qua các migration script SQL được đánh số thứ tự, đảm bảo schema database luôn đồng bộ với phiên bản mã nguồn hiện tại trên mọi môi trường.

Bảng 3.1: Các thư viện và công cụ phát triển chính

Mục đích	Công cụ / Thư viện	Phiên bản
Ngôn ngữ lập trình	Java (OpenJDK)	21 LTS
Build tool	Maven Wrapper	3.9+
Framework chính	Spring Boot	3.5.13
Spring Cloud	Spring Cloud Dependencies	2025.0.0
Giảm boilerplate	Lombok	1.18.36+
JSON serialization	Jackson	(qua Spring Boot)
Database migration	Flyway	11.7.0
API documentation	SpringDoc OpenAPI	2.8.11

3.3 Hệ sinh thái Spring Cloud

3.3.1 Spring Cloud Gateway

Spring Cloud Gateway đóng vai trò API Gateway — điểm vào duy nhất cho mọi yêu cầu từ client đến hệ thống. Gateway được xây dựng trên nền tảng Spring WebFlux và Netty, cung cấp mô hình xử lý bất đồng bộ, non-blocking, cho phép xử lý hàng nghìn kết nối đồng thời với số lượng thread hạn chế.

Gateway được cấu hình thông qua các route định tuyến, mỗi route ánh xạ một đường dẫn (`/api/products/**`) đến một dịch vụ đích (`lb://product-`

service) thông qua Eureka service discovery. Các filter được áp dụng trên từng route hoặc toàn cục để xử lý các trách nhiệm cross-cutting: AuthHeaderFilter trích xuất thông tin người dùng từ JWT và chèn vào header của request chuyển tiếp đến backend; GuestSessionFilter tạo và duy trì session cho người dùng chưa đăng nhập khi thao tác với giỏ hàng; RateLimiterFilter áp dụng Redis token bucket rate limiter cho các endpoint nhạy cảm. CORS được cấu hình tập trung tại Gateway thông qua biến môi trường `CORS_ALLOWED_ORIGINS`, cho phép linh hoạt thay đổi giữa môi trường phát triển và sản xuất.

So sánh với các giải pháp thay thế: Netflix Zuul (phiên bản 1.x dựa trên Servlet API blocking, hiệu năng thấp hơn đáng kể so với reactive Gateway; phiên bản 2.x chưa bao giờ được phát hành chính thức). Kong và Traefik là các API Gateway độc lập, mạnh mẽ và hiệu năng cao, nhưng yêu cầu vận hành riêng biệt và không tích hợp sẵn với Spring Security/OAuth2 như Spring Cloud Gateway. Với hệ thống toàn Spring, Gateway là lựa chọn tự nhiên và tối ưu.

3.3.2 Netflix Eureka — Service Discovery

Eureka Server đóng vai trò Service Registry — nơi tất cả dịch vụ đăng ký địa chỉ của mình khi khởi động. Mỗi service được cấu hình với `eureka.client.serviceUrl.defaultZone` trỏ về Eureka Server tại `http://eureka:8761/eureka`. Khi API Gateway cần định tuyến một request đến `lb://product-service`, nó truy vấn Eureka để lấy danh sách các instance hiện có của `product-service` và thực hiện client-side load balancing.

Eureka hoạt động theo mô hình AP (ưu tiên Availability và Partition Tolerance theo CAP Theorem), phù hợp với yêu cầu của hệ thống TMĐT — nơi tính sẵn sàng của service registry quan trọng hơn tính nhất quán tuyệt đối. Cơ chế self-preservation của Eureka ngăn việc xóa hàng loạt registration khi có sự cố mạng tạm thời, đảm bảo hệ thống tiếp tục hoạt động.

Lựa chọn thay thế: Consul (HashiCorp) cung cấp health check mạnh mẽ hơn và hỗ trợ key-value store có thể thay thế Config Server; Kubernetes Service + Ingress cung cấp service discovery tự nhiên trong môi trường Kubernetes nhưng không khả dụng trong môi trường Docker Compose mà đề tài hướng tới. Eureka được chọn vì tích hợp chặt chẽ với Spring Cloud, cấu hình tối thiểu và đáp ứng tốt yêu cầu single-host của đồ án.

3.3.3 Spring Cloud Config Server

Config Server cung cấp cơ chế quản lý cấu hình tập trung cho toàn bộ hệ thống. Thay vì mỗi service nhúng cấu hình trong file `application.yml` tĩnh hoặc phụ thuộc vào biến môi trường rải rác, tất cả cấu hình được lưu trữ tập trung trên

Config Server và được phục vụ qua HTTP API. Mỗi service khi khởi động sẽ gọi đến Config Server để lấy cấu hình tương ứng với tên service và profile đang chạy, cho phép thay đổi cấu hình tại một điểm duy nhất và áp dụng cho tất cả instance của service mà không cần rebuild image.

Trong đồ án, Config Server sử dụng native profile (đọc từ classpath) thay vì Git backend, phù hợp với quy mô đồ án và môi trường Docker. Cấu hình được tổ chức theo cấu trúc `{service-name}-{profile}.yaml`, cho phép mỗi service có cấu hình riêng cho môi trường docker và local. Hạn chế của cách tiếp cận này là cần rebuild Config Server image mỗi khi thay đổi cấu hình, không linh hoạt bằng Git backend hoặc các giải pháp quản lý cấu hình động như Vault.

3.3.4 OpenFeign — Giao tiếp đồng bộ giữa các dịch vụ

OpenFeign là thư viện HTTP client khai báo (declarative) cho phép giao tiếp giữa các dịch vụ một cách đơn giản: lập trình viên chỉ cần định nghĩa một interface với các annotation mô tả endpoint đích, OpenFeign tự động sinh implementation tại runtime. Cơ chế này giảm đáng kể lượng code boilerplate so với việc sử dụng `RestTemplate` hoặc `WebClient` thủ công, đồng thời tích hợp sẵn với Eureka để phân giải tên dịch vụ thành địa chỉ thực tế.

Trong hệ thống, OpenFeign được sử dụng cho các tương tác đồng bộ cần phản hồi ngay: `cart-service` gọi `product-service` để xác minh sản phẩm; `order-service` gọi `product-service`, `inventory-service` và `voucher-service` để kiểm tra trước khi tạo đơn; `review-service` gọi `order-service` để xác minh quyền đánh giá. Các Feign client được cấu hình timeout và retry để xử lý tình huống dịch vụ đích tạm thời không khả dụng.

3.4 Hạ tầng dữ liệu

3.4.1 PostgreSQL 17

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ chính của hệ thống, được sử dụng cho 10 trong số 13 business service theo nguyên tắc database-per-service. Trên một instance PostgreSQL duy nhất, mỗi service sở hữu một logical database riêng biệt (ví dụ: `order_db`, `payment_db`, `product_db`), đảm bảo cách ly dữ liệu ở mức schema và cho phép mỗi service quản lý schema độc lập thông qua Flyway migration.

PostgreSQL được chọn thay vì MySQL vì bốn lý do. Thứ nhất, PostgreSQL hỗ trợ tốt hơn các kiểu dữ liệu nâng cao như UUID (được sử dụng làm khóa chính cho hầu hết các bảng trong hệ thống), JSONB cho dữ liệu bán cấu trúc và ARRAY cho các tập giá trị. Thứ hai, PostgreSQL có hệ thống indexing mạnh mẽ hơn với GiST,

GIN và BRIN, đặc biệt hữu ích cho các truy vấn tìm kiếm và lọc phức tạp trong TMĐT. Thứ ba, MVCC (Multi-Version Concurrency Control) của PostgreSQL xử lý tốt hơn các tình huống đọc-ghi đồng thời cao so với cơ chế khóa của MySQL InnoDB. Thứ tư, PostgreSQL có tính tương thích cao với Flyway và Spring Data JPA.

3.4.2 Redis 8

Redis đảm nhiệm ba vai trò quan trọng trong hệ thống, tận dụng mô hình single-threaded và khả năng thao tác trong bộ nhớ với độ trễ dưới mili-giây. Vai trò thứ nhất là **cache phân tán**: product-service sử dụng Redis để cache thông tin sản phẩm, giảm tải cho PostgreSQL và giảm độ trễ phản hồi cho các request đọc danh sách sản phẩm — chiếm phần lớn lưu lượng của hệ thống TMĐT. Vai trò thứ hai là **lưu trữ giỏ hàng**: cart-service lưu toàn bộ giỏ hàng của người dùng trong Redis, với key được tạo từ userId hoặc sessionId, tận dụng thời gian truy cập nhanh của Redis và cơ chế TTL để tự động dọn dẹp giỏ hàng bỏ hoang. Vai trò thứ ba là **rate limiting**: API Gateway sử dụng Redis làm backend cho token bucket rate limiter, lưu trữ và cập nhật số lượng token còn lại cho mỗi client.

So sánh với các lựa chọn thay thế: Memcached là giải pháp cache đơn giản hơn nhưng thiếu các cấu trúc dữ liệu phong phú của Redis (Sorted Set cho ranking, Hash cho object, List cho queue) và không hỗ trợ Lua scripting — yếu tố then chốt cho atomic operation trong flash sale. Hazelcast cung cấp cache phân tán embedded trong JVM, loại bỏ dependency network đến Redis, nhưng tiêu tốn bộ nhớ heap của chính service và phức tạp hơn trong việc đồng bộ trạng thái giữa các instance.

3.4.3 Elasticsearch 8

Elasticsearch được sử dụng làm search engine cho chức năng tìm kiếm toàn văn sản phẩm, triển khai theo mô hình CQRS-lite. Trong khi product-service (với PostgreSQL) đóng vai trò write model — nơi quản trị viên tạo và cập nhật thông tin sản phẩm, search-service (với Elasticsearch) đóng vai trò read model — phục vụ các truy vấn tìm kiếm phức tạp từ phía khách hàng. Hai mô hình được đồng bộ qua Kafka: mỗi khi sản phẩm được tạo, cập nhật hoặc xóa, một sự kiện được phát hành và search-service tiêu thụ để cập nhật chỉ mục Elasticsearch tương ứng.

Elasticsearch được chọn thay vì Apache Solr vì khả năng mở rộng theo chiều ngang tốt hơn, REST API hiện đại và tích hợp tốt với hệ sinh thái Spring Data Elasticsearch. So với PostgreSQL full-text search (`tsvector`), Elasticsearch vượt trội về tốc độ tìm kiếm trên tập dữ liệu lớn, hỗ trợ tìm kiếm mờ (fuzzy search), gợi ý tự động (autocomplete) và phân tích tần suất từ khóa — những tính năng quan

trọng đối với trải nghiệm tìm kiếm sản phẩm trong TMĐT. Tuy nhiên, Elasticsearch tiêu tốn nhiều tài nguyên hơn (RAM và CPU) và cần được cấu hình JVM heap cẩn thận để tránh ảnh hưởng đến các container khác trong môi trường Docker Compose single-host.

3.5 Apache Kafka — Message Broker

Apache Kafka được chọn làm message broker trung tâm, đảm nhiệm toàn bộ giao tiếp bất đồng bộ giữa các dịch vụ trong hệ thống. Kafka vận hành ở chế độ KRaft (KRaft metadata mode) — không phụ thuộc vào Apache ZooKeeper, giảm đáng kể số lượng thành phần hạ tầng cần quản lý và đơn giản hóa việc triển khai. Đây là cải tiến quan trọng của Kafka từ phiên bản 3.x, đặc biệt phù hợp với môi trường triển khai single-host như đề án này.

Kafka được chọn thay vì RabbitMQ vì ba ưu thế chính trong bối cảnh hệ thống TMĐT. Thứ nhất, Kafka được thiết kế cho throughput cao với cơ chế lưu trữ tuần tự trên đĩa (sequential disk I/O) và zero-copy transfer, cho phép xử lý hàng triệu thông điệp mỗi giây — phù hợp với lưu lượng lớn trong các sự kiện flash sale. Thứ hai, Kafka lưu trữ thông điệp bền vững trên đĩa với thời gian lưu trữ có thể cấu hình (retention), cho phép consumer đọc lại thông điệp từ quá khứ — hữu ích cho các cơ chế reconciliation và khôi phục trạng thái. Thứ ba, cơ chế partitioning và consumer group của Kafka cho phép xử lý song song và đảm bảo thứ tự trong cùng một partition — yếu tố quan trọng để tuần tự hóa yêu cầu flash sale.

Tuy nhiên, Kafka cũng có những hạn chế cần cân nhắc. So với RabbitMQ, Kafka có độ trễ end-to-end cao hơn do cơ chế batch và polling, do đó kém phù hợp hơn cho các tương tác yêu cầu phản hồi tức thời. Kafka cũng phức tạp hơn về mặt cấu hình và vận hành, tiêu tốn nhiều tài nguyên hơn. Trong đề án, các tương tác cần phản hồi ngay được xử lý qua REST/OpenFeign, còn Kafka chỉ đảm nhiệm các luồng bất đồng bộ.

3.6 Keycloak — Bảo mật và quản lý định danh

Keycloak 26 được chọn làm Identity Provider (IdP) cho toàn bộ hệ thống, chịu trách nhiệm quản lý người dùng, xác thực và cấp phát token. Keycloak là giải pháp mã nguồn mở, được phát triển bởi Red Hat, hỗ trợ đầy đủ các giao thức chuẩn công nghiệp: OAuth 2.0, OpenID Connect và SAML 2.0. Với Keycloak, hệ thống không cần tự triển khai logic quản lý người dùng, xác thực mật khẩu, quản lý phiên làm việc hay làm mới token — tất cả được Keycloak xử lý một cách chuẩn hóa và an toàn.

Keycloak được lựa chọn thay vì các giải pháp thay thế vì những lý do sau. So với việc tự xây dựng hệ thống xác thực (Spring Security + database người dùng

tự quản), Keycloak cung cấp sẵn các tính năng bảo mật phức tạp đã được kiểm chứng: hỗ trợ đa yếu tố xác thực (MFA), chính sách mật khẩu, brute-force detection, social login (Google, Facebook) và quản trị tập trung qua giao diện web. So với Auth0 hoặc Okta — các giải pháp Identity-as-a-Service thương mại, Keycloak là mã nguồn mở và tự lưu trữ (self-hosted), không phụ thuộc vào bên thứ ba, không giới hạn số lượng người dùng và phù hợp với môi trường học thuật không có ngân sách. So với Apache Shiro hoặc CAS, Keycloak có hệ sinh thái tích hợp Spring Boot/Security phong phú hơn và cộng đồng hỗ trợ lớn hơn.

Trong kiến trúc bảo mật của hệ thống, Keycloak đảm nhiệm ba chức năng chính. Chức năng thứ nhất là **cấp phát token**: sau khi người dùng đăng nhập thành công (qua giao diện đăng nhập của Keycloak hoặc qua API của identity-service), Keycloak cấp một bộ token gồm Access Token (JWT, thời hạn 5 phút), Refresh Token (thời hạn 30 phút) và ID Token. Access Token được ký bằng thuật toán RS256 với private key của Keycloak; public key được công khai tại endpoint JWKS (/realms/ecommerce/protocol/openid-connect/certs) để API Gateway xác minh. Chức năng thứ hai là **làm mới token**: khi Access Token hết hạn, client sử dụng Refresh Token để nhận Access Token mới mà không cần đăng nhập lại. Chức năng thứ ba là **quản trị tập trung**: quản trị viên có thể tạo, sửa, khóa người dùng, quản lý vai trò (ROLE_USER, ROLE_ADMIN) và theo dõi phiên làm việc qua Keycloak Admin Console tại <http://localhost:8180>.

Hạn chế của Keycloak trong bối cảnh đồ án: Keycloak là một ứng dụng Java nặng, tiêu tốn đáng kể tài nguyên (RAM và CPU) trên môi trường single-host. Điều này được quản lý thông qua giới hạn bộ nhớ trong Docker Compose và chấp nhận như một đánh đổi để có được giải pháp bảo mật toàn diện.

3.7 VNPAY — Cổng thanh toán trực tuyến

VNPAY được chọn làm cổng thanh toán cho chức năng thanh toán trực tuyến của hệ thống. Đây là một trong những cổng thanh toán hàng đầu tại Việt Nam, hỗ trợ đa dạng phương thức thanh toán bao gồm thẻ ATM nội địa, thẻ tín dụng quốc tế (Visa, Mastercard, JCB), ví điện tử và QR code. Đối với đồ án tốt nghiệp, VNPAY cung cấp môi trường sandbox miễn phí với đầy đủ API, tài liệu và thẻ test, cho phép kiểm thử toàn bộ luồng thanh toán mà không cần giao dịch thật.

Quy trình thanh toán qua VNPAY được thiết kế theo ba bước. Bước thứ nhất, payment-service tạo một URL thanh toán bằng cách gửi yêu cầu đến VNPAY với các tham số: mã đơn hàng, số tiền, mô tả đơn hàng, địa chỉ IP của khách hàng, ngân hàng (tùy chọn) và ngôn ngữ hiển thị. Tất cả tham số được sắp xếp theo thứ tự alphabet và ký với HMAC-SHA512 sử dụng secret key được VNPAY cấp, tạo

ra một chữ ký số (`vnp_SecureHash`) đảm bảo tính toàn vẹn và chống giả mạo. Bước thứ hai, khách hàng được chuyển hướng đến trang thanh toán của VNPay để nhập thông tin thẻ và xác nhận thanh toán. Bước thứ ba, sau khi thanh toán hoàn tất (thành công hoặc thất bại), VNPay gọi đến Return URL đã đăng ký kèm theo kết quả thanh toán đã được ký, payment-service xác minh chữ ký và cập nhật trạng thái thanh toán. Song song, VNPay cũng gửi IPN (Instant Payment Notification) đến một endpoint riêng để đảm bảo kết quả được ghi nhận ngay cả khi khách hàng đóng trình duyệt trước khi được chuyển hướng về Return URL.

Lựa chọn thay thế cho VNPay bao gồm Momo, ZaloPay và PayPal. Momo và ZaloPay có quy trình tích hợp tương tự nhưng tài liệu sandbox và thử nghiệm ít phong phú hơn cho mục đích học thuật. PayPal phổ biến toàn cầu nhưng không hỗ trợ tốt các ngân hàng nội địa Việt Nam. VNPay được chọn vì tính phổ biến tại Việt Nam, tài liệu tiếng Việt đầy đủ và sandbox dễ tiếp cận.

Hạn chế: phạm vi đồ án chỉ dừng ở môi trường sandbox, chưa kiểm thử với giao dịch thật. Trong môi trường production, cần xử lý thêm các tình huống như hoàn tiền, truy vấn giao dịch và đối soát định kỳ.

3.8 Hệ thống quan sát — Observability Stack

3.8.1 Prometheus và Grafana — Giám sát số liệu

Prometheus được chọn làm hệ thống thu thập và lưu trữ metrics cho toàn bộ hệ thống. Đây là giải pháp giám sát mã nguồn mở, được thiết kế đặc biệt cho môi trường động và microservices với cơ chế pull-based: thay vì chờ service đẩy metrics về, Prometheus chủ động kéo (scrape) dữ liệu từ các endpoint `/actuator/prometheus` của từng service theo chu kỳ cấu hình (15 giây trong đồ án). Cơ chế này giúp Prometheus tự phát hiện service mới hoặc service ngừng hoạt động mà không cần cấu hình thủ công, đồng thời giảm tải cho service so với mô hình push-based.

Prometheus sử dụng ngôn ngữ truy vấn PromQL mạnh mẽ, cho phép tính toán các chỉ số phức tạp như latency percentile (p50, p95, p99), tỷ lệ lỗi theo thời gian và tốc độ tăng trưởng của một chỉ số. Dữ liệu được lưu trữ theo mô hình time-series với khả năng nén hiệu quả, phù hợp cho việc lưu trữ dài hạn trên môi trường single-host với tài nguyên hạn chế.

Grafana đảm nhiệm vai trò trực quan hóa dữ liệu từ Prometheus. Các dashboard được provision tự động thông qua cơ chế file-based provisioning của Grafana, bao gồm `jvm-overview`, `spring-boot-overview` và `ecommerce-saga-overview`. Ba dashboard này tập trung vào các chỉ số JVM, request/latency/error rate của Spring Boot service và trạng thái các luồng Saga cốt lõi. Grafana được

chọn thay vì Kibana vì tích hợp tốt hơn với Prometheus (Kibana chủ yếu dành cho Elasticsearch) và cộng đồng dashboard template phong phú hơn.

3.8.2 Zipkin — Truy vết phân tán

Zipkin được chọn làm hệ thống distributed tracing, cho phép theo dõi hành trình của một request qua nhiều service. Zipkin là giải pháp mã nguồn mở, nhẹ và dễ triển khai — trong đồ án, Zipkin chạy như một container độc lập với storage in-memory, không yêu cầu cơ sở dữ liệu backend phức tạp. Zipkin tuân thủ chuẩn W3C Trace Context, đảm bảo khả năng tương tác với các thư viện tracing khác nhau trong hệ sinh thái.

Mỗi service trong hệ thống được tích hợp Micrometer Tracing (trước đây là Spring Cloud Sleuth) để tự động sinh Trace ID và Span ID cho mỗi request, đồng thời gửi dữ liệu trace về Zipkin server qua HTTP. Khi cần điều tra một request cụ thể — ví dụ một đơn hàng thất bại — kỹ sư có thể tìm kiếm theo Trace ID trên Zipkin UI và xem toàn bộ chuỗi span, thời gian xử lý tại từng service và các tag nghiệp vụ kèm theo. Điều này đặc biệt hữu ích cho việc gỡ lỗi các luồng Saga phức tạp.

Lựa chọn thay thế: Jaeger (Uber) cung cấp hiệu năng cao hơn và hỗ trợ storage backend linh hoạt hơn (Cassandra, Elasticsearch), phù hợp với hệ thống production quy mô lớn. Tuy nhiên, Zipkin nhẹ hơn, dễ triển khai hơn và tích hợp sẵn trong Spring Boot ecosystem thông qua Micrometer — phù hợp hơn với quy mô đồ án.

3.8.3 Mailpit — Kiểm thử email

Mailpit được chọn làm SMTP server cho môi trường phát triển, thay thế cho các dịch vụ email thật như Gmail SMTP hoặc SendGrid. Mailpit bắt giữ tất cả email được gửi từ notification-service và hiển thị chúng qua giao diện web tại <http://localhost:8025>, cho phép kiểm tra nội dung email xác nhận đơn hàng, email hủy đơn và email thông báo flash sale mà không cần gửi email thật ra bên ngoài. Trong môi trường production, Mailpit được thay thế bằng SMTP server thật thông qua cấu hình biến môi trường.

3.9 Next.js, AWS EC2 và CI/CD

3.9.1 Next.js — Storefront và Admin Panel

Next.js được chọn cho tầng giao diện vì hỗ trợ App Router, Server Components, Server Actions và khả năng kết hợp linh hoạt giữa render phía server và tương tác phía client. Trong hệ thống, Next.js phục vụ hai nhóm màn hình: storefront cho khách hàng và Admin Panel cho quản trị viên. Các request từ trình duyệt đi qua lớp BFF/proxy của frontend; access token được lưu trong httpOnly cookie và chỉ được

gắn vào header `Authorization` khi server-side code gọi API Gateway. Cách tổ chức này giảm nguy cơ lộ token ở phía client và phù hợp với mô hình Gateway đóng vai Resource Server.

Admin Panel sử dụng route segment `/admin/*`, được bảo vệ bởi hai lớp: middleware kiểm tra cookie/JWT và role `ROLE_ADMIN`, sau đó layout phía server gọi `getAdminSession` để chặn truy cập khi token thiếu hoặc không đủ quyền. Các mutation như tạo sản phẩm, điều chỉnh tồn kho, cập nhật đơn hàng, quản lý voucher, flash-sale và content được thực hiện bằng Server Actions, nhờ đó token không cần xuất hiện trong JavaScript chạy trên trình duyệt.

3.9.2 AWS EC2, Caddy và GitHub Actions

Môi trường triển khai của đề án là một máy ảo AWS EC2 single-host, phù hợp với giới hạn tài nguyên của đề án và vẫn đủ để kiểm chứng các đặc tính vận hành chính. Caddy được dùng làm reverse proxy vì có khả năng tự động cấp/gia hạn chứng chỉ TLS qua Let's Encrypt và cấu hình đơn giản cho các subdomain dạng `nip.io`. Trên môi trường production-like, Grafana được đổi sang cổng host 3001 để tránh xung đột với frontend ở cổng 3000.

GitHub Actions đảm nhiệm quy trình build và publish Docker image lên GitHub Container Registry (GHCR). File `docker-compose.prod.yml` sử dụng các image `ghcr.io/...` thay cho local build, đồng thời đặt `mem_limit` và `mem_reservation` cho từng container để phù hợp tài nguyên của EC2. Quy trình này chưa phải GitOps/Kubernetes pipeline hoàn chỉnh, nhưng đáp ứng mục tiêu production-like deployment trong phạm vi đề án.

3.10 Docker Compose — Đóng gói và triển khai

Backend, frontend và các thành phần hạ tầng của hệ thống được định nghĩa và điều phối thông qua Docker Compose. Mỗi service được mô tả trong file `docker-compose.yml` với đầy đủ thông tin: image hoặc Dockerfile, cổng, biến môi trường, volume, network, healthcheck, resource limits và mối quan hệ khởi động (`depends_on`).

Docker Compose được chọn thay vì Kubernetes vì ba lý do. Thứ nhất, Docker Compose sử dụng cú pháp YAML đơn giản, dễ đọc và dễ cấu hình. Thứ hai, Docker Compose phù hợp với mô hình triển khai single-host mà đề tài hướng tới, trong khi Kubernetes được thiết kế cho môi trường multi-node cluster và đòi hỏi kiến thức vận hành phức tạp hơn nhiều. Thứ ba, Docker Compose tích hợp sẵn trong Docker Desktop, không yêu cầu cài đặt thêm công cụ, giúp giảm rào cản tiếp cận cho người học.

Hệ thống sử dụng Docker Compose profile để tách biệt cấu hình giữa môi trường phát triển và môi trường production. File `docker-compose.yml` định nghĩa cấu hình cơ sở dùng chung; file `docker-compose.prod.yml` kế thừa và ghi đè các thiết lập cho production như GHCR image, `mem_limit`, `mem_reservation`, cổng Grafana 3001 và CORS allowed origins. Cách tiếp cận này tránh trùng lặp cấu hình và đảm bảo môi trường development không bị ảnh hưởng bởi các ràng buộc của production.

3.11 Công cụ kiểm thử

3.11.1 JUnit 5 và Testcontainers

JUnit 5 là nền tảng kiểm thử chính cho toàn bộ hệ thống, được tích hợp sẵn trong Spring Boot starter test. JUnit 5 mang lại nhiều cải tiến so với JUnit 4: mô hình extension model linh hoạt hơn, hỗ trợ lambda trong assertion, tổ chức test theo class lồng nhau (@Nested) và parameterized test mạnh mẽ hơn. Kết hợp với Mockito, JUnit 5 cho phép mock các dependency bên ngoài (repository, Feign client, Kafka template) để kiểm thử logic nghiệp vụ một cách cô lập.

Testcontainers là thư viện hỗ trợ kiểm thử tích hợp với cơ sở hạ tầng thật: thay vì sử dụng H2 in-memory database — vốn không tương thích hoàn toàn với PostgreSQL, Testcontainers khởi tạo một container PostgreSQL thật trong quá trình chạy test, đảm bảo môi trường kiểm thử giống hệt môi trường production. Testcontainers cũng hỗ trợ khởi tạo container Kafka, Redis và Elasticsearch, mặc dù trong đồ án chủ yếu được sử dụng cho PostgreSQL.

3.11.2 k6 — Kiểm thử hiệu năng

k6 được chọn làm công cụ kiểm thử hiệu năng cho hệ thống. Đây là công cụ mã nguồn mở, sử dụng JavaScript làm ngôn ngữ kịch bản, cho phép viết các kịch bản kiểm thử phức tạp với logic tùy chỉnh (tạo user, lấy token, đặt hàng, kiểm tra kết quả). k6 hỗ trợ nhiều mô hình tải: constant load (duy trì số lượng VU cố định), ramping load (tăng dần số lượng VU) và spike test (tăng đột biến). Kết quả kiểm thử được xuất ra dưới dạng JSON với các chỉ số chi tiết: request rate, latency percentile, error rate, checks pass/fail — các chỉ số này được sử dụng làm bằng chứng định lượng trong Chương 5.

So với JMeter — công cụ kiểm thử hiệu năng truyền thống với giao diện GUI, k6 ưu việt hơn ở khả năng viết kịch bản dưới dạng code (Infrastructure as Code), tích hợp CI/CD dễ dàng hơn và tiêu tốn ít tài nguyên hơn khi chạy. Với yêu cầu mô phỏng 500 VU đồng thời cho flash sale spike test, k6 là lựa chọn phù hợp.

CHƯƠNG 4. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Chương này trình bày toàn bộ kết quả phân tích và thiết kế hệ thống, từ việc xác định yêu cầu, mô hình hóa kiến trúc tổng thể, thiết kế cơ sở dữ liệu, thiết kế API và giao tiếp, đến thiết kế luồng sự kiện, bảo mật, máy trạng thái và các luồng nghiệp vụ chính. Đây là chương có mật độ sơ đồ và bảng biểu cao nhất, phản ánh toàn bộ quá trình chuyển đổi từ yêu cầu nghiệp vụ sang thiết kế kỹ thuật chi tiết. Các quyết định thiết kế trong chương này dựa trên nền tảng lý thuyết đã trình bày ở Chương 2 và được hiện thực hóa bằng các công nghệ đã phân tích ở Chương 3.

4.1 Phân tích yêu cầu hệ thống

4.1.1 Yêu cầu chức năng

Hệ thống được xây dựng để đáp ứng tập hợp đầy đủ các chức năng của một nền tảng thương mại điện tử, phục vụ hai nhóm người dùng chính là khách hàng và quản trị viên. Bảng 4.1 tổng hợp các yêu cầu chức năng được nhóm theo miền nghiệp vụ.

CHƯƠNG 4. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Bảng 4.1: Yêu cầu chức năng theo miền nghiệp vụ

Miền nghiệp vụ	Đối tượng	Yêu cầu chức năng
Định danh & Người dùng	Khách hàng	Đăng ký tài khoản, đăng nhập, làm mới token
	Khách hàng	Quản lý thông tin cá nhân, quản lý địa chỉ giao hàng
Sản phẩm & Danh mục	Khách hàng	Xem danh sách sản phẩm, lọc theo danh mục/thương hiệu, xem chi tiết
	Quản trị viên	Tạo, sửa, xóa sản phẩm, danh mục, thương hiệu
Tìm kiếm	Khách hàng	Tìm kiếm toàn văn theo từ khóa, gợi ý sản phẩm liên quan
Giỏ hàng	Khách hàng	Thêm, sửa số lượng, xóa sản phẩm; hỗ trợ giỏ hàng khách vắng lai
Đơn hàng	Khách hàng	Tạo đơn hàng (COD/VNPAY), xem danh sách đơn hàng, xem chi tiết trạng thái
	Hệ thống	Tự động xử lý luồng Saga: trừ tồn kho, xác nhận thanh toán, hủy đơn quá hạn
Thanh toán	Khách hàng	Thanh toán online qua VNPAY, xem kết quả thanh toán
	Hệ thống	Xử lý IPN callback từ VNPAY, tự động timeout thanh toán quá hạn
Khuyến mãi	Khách hàng	Xem danh sách voucher khả dụng, áp dụng voucher khi đặt hàng
	Quản trị viên	Tạo, sửa, xóa voucher; cấu hình điều kiện sử dụng
Flash Sale	Khách hàng	Xem danh sách chiến dịch, mua hàng flash sale với số lượng giới hạn
	Quản trị viên	Tạo, kích hoạt, kết thúc chiến dịch; cấu hình sản phẩm và số lượng
Đánh giá	Khách hàng	Tạo, sửa, xóa đánh giá sản phẩm (chỉ sau khi mua hàng thành công)
Nội dung	Quản trị viên	Quản lý banner, bài viết blog
Thông báo	Hệ thống	Gửi email xác nhận đơn hàng, hủy đơn, thông báo flash sale

4.1.2 Yêu cầu phi chức năng

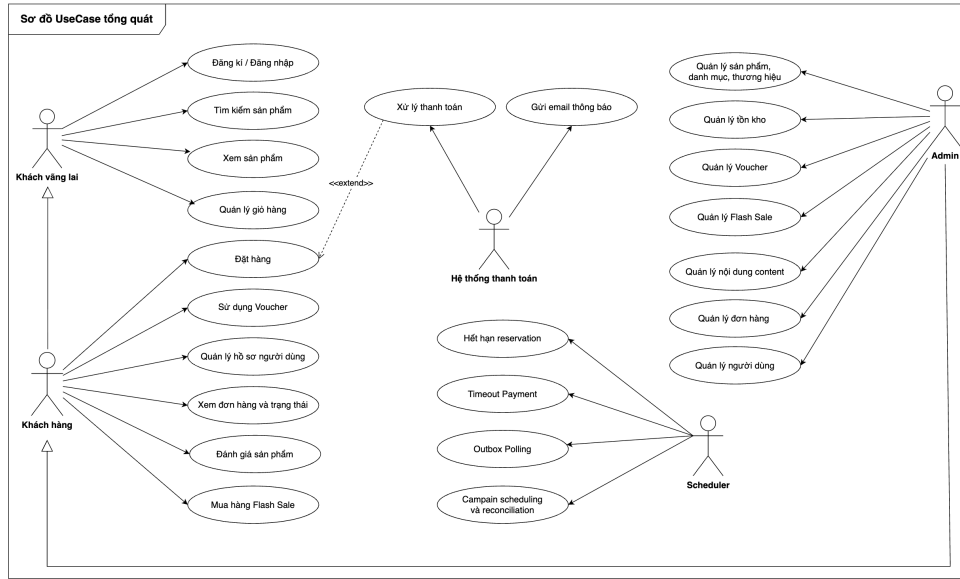
Bên cạnh các yêu cầu chức năng, hệ thống phải đáp ứng các yêu cầu phi chức năng then chốt của một hệ thống phân tán, được tổng hợp trong Bảng 4.2.

Bảng 4.2: Yêu cầu phi chức năng và chiến lược đáp ứng

Yêu cầu	Mô tả	Chiến lược thiết kế
Khả năng mở rộng	Hệ thống có thể mở rộng độc lập từng dịch vụ theo tải	Phân rã 13 service; database per service
Tính sẵn sàng	Hệ thống tiếp tục hoạt động khi một service gặp sự cố	Circuit Breaker; stateless service; graceful degradation
Tính nhất quán	Dữ liệu nhất quán cuối cùng giữa các service	Saga Orchestration; Outbox + Idempotent Consumer
Bảo mật	Xác thực và phân quyền cho mọi API endpoint	Keycloak OAuth2/OIDC; JWT RS256; Gateway Trusted Subsystem
Hiệu năng	Đáp ứng tải cao trong flash sale; phản hồi nhanh cho catalog	Redis cache; Redis Lua atomic; k6 load test
Khả năng quan sát	Giám sát và truy vết request end-to-end	Prometheus + Grafana; Zipkin; structured logging
Khả năng triển khai	Dễ dàng khởi tạo toàn bộ hệ thống	Docker Compose; healthcheck; Flyway migration tự động

4.2 Mô hình hóa yêu cầu — Biểu đồ Use Case

Hệ thống có bốn tác nhân chính tương tác theo các vai trò và mục đích khác nhau. Khách hàng chưa đăng nhập (Guest) có thể duyệt sản phẩm, tìm kiếm và thao tác với giỏ hàng tạm thời. Khách hàng đã đăng nhập (Customer) có toàn quyền thực hiện các nghiệp vụ mua sắm: đặt hàng, thanh toán, tham gia flash sale và đánh giá sản phẩm. Quản trị viên (Admin) quản lý toàn bộ danh mục, sản phẩm, tồn kho, khuyến mãi, nội dung và chiến dịch flash sale. Hệ thống bên ngoài (External System) bao gồm VNPAY xử lý thanh toán và SMTP Server gửi email thông báo. Ngoài ra, các tác vụ nền (Scheduler) tự động xử lý các công việc định kỳ như hết hạn đặt chỗ tồn kho và timeout thanh toán.

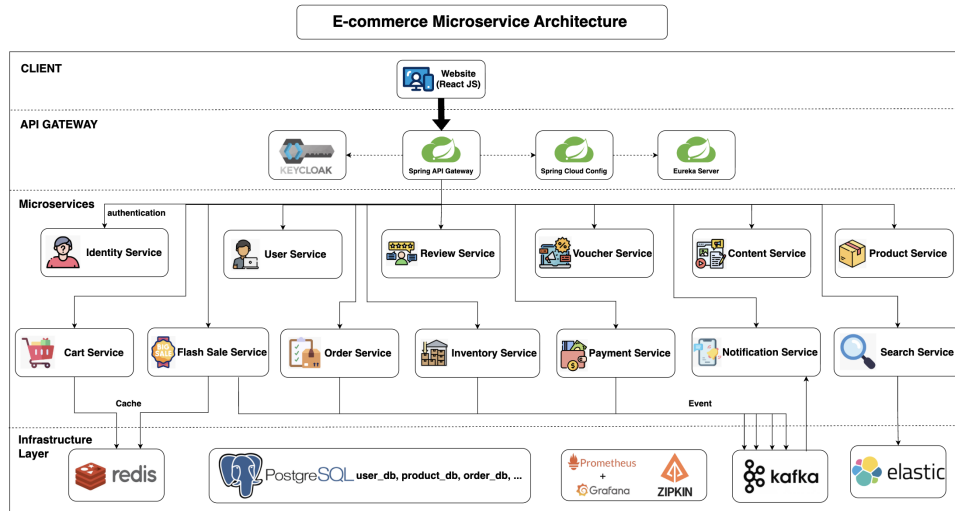


Hình 4.1: Biểu đồ Use Case tổng thể hệ thống

4.3 Kiến trúc tổng thể

Hệ thống được tổ chức thành bốn tầng kiến trúc rõ ràng, mỗi tầng có trách nhiệm và phạm vi ảnh hưởng riêng biệt. Tầng Client bao gồm trình duyệt web, ứng dụng di động hoặc các công cụ kiểm thử API như Postman — đây là nơi phát sinh yêu cầu vào hệ thống. Tầng Edge và Spring Cloud chứa API Gateway (cổng vào duy nhất), Eureka Server (đăng ký và khám phá dịch vụ), Config Server (quản lý cấu hình tập trung) và Keycloak (xác thực và cấp token). Tầng Business Services gồm 13 dịch vụ nghiệp vụ, mỗi dịch vụ đảm nhiệm một miền chức năng độc lập và sở hữu cơ sở dữ liệu riêng. Tầng Infrastructure chứa các hệ thống hỗ trợ: PostgreSQL, Redis, Kafka, Elasticsearch, Mailpit, Prometheus, Grafana và Zipkin.

Giao tiếp giữa các tầng tuân theo nguyên tắc: Client chỉ giao tiếp với tầng Edge (qua API Gateway); các dịch vụ trong tầng Business giao tiếp với nhau qua REST/OpenFeign hoặc bất đồng bộ qua Kafka; mọi truy cập đến cơ sở dữ liệu đều thông qua dịch vụ sở hữu dữ liệu đó — không có truy cập trực tiếp từ dịch vụ khác.



Hình 4.2: Sơ đồ thành phần bốn tầng của hệ thống

4.4 Phân rã miền nghiệp vụ và sở hữu dịch vụ

Áp dụng nguyên tắc Domain-Driven Design, hệ thống được phân rã thành 13 Bounded Context, mỗi context tương ứng với một microservice. Bảng 4.3 mô tả chi tiết trách nhiệm, dữ liệu sở hữu và phương thức giao tiếp của từng dịch vụ. Nguyên tắc cốt lõi là mỗi dịch vụ sở hữu toàn bộ logic nghiệp vụ và dữ liệu trong phạm vi Bounded Context của mình; mọi truy cập dữ liệu chéo phải thông qua API công khai hoặc sự kiện được công bố trên Kafka.

Bảng 4.3: Sở hữu dịch vụ và giao tiếp

Dịch vụ	Cổng	Bounded Context	Dữ liệu sở hữu	Giao tiếp
identity-service	8081	Định danh & Đăng ký	Keycloak users	Kafka (pub)
user-service	8082	Hồ sơ người dùng	PostgreSQL (user_db)	Kafka (sub)
product-service	8083	Sản phẩm & Danh mục	PostgreSQL (product_db), Redis cache	REST, Kafka (pub)
inventory-service	8084	Tồn kho & Đặt chỗ	PostgreSQL (inventory_db)	Kafka (pub/sub)
cart-service	8085	Giỏ hàng	Redis (cart keys)	REST (Feign)
order-service	8086	Đơn hàng (Saga orchestrator)	PostgreSQL (order_db), Outbox	REST, Kafka (pub/sub)
payment-service	8087	Thanh toán & VNPAY	PostgreSQL (payment_db), Outbox	REST, Kafka (pub/sub)
voucher-service	8088	Khuyến mãi	PostgreSQL (voucher_db)	REST
notification-service	8089	Thông báo & Email	PostgreSQL (notification_db)	Kafka (sub)
review-service	8090	Đánh giá sản phẩm	PostgreSQL (review_db)	REST (Feign)
search-service	8091	Tìm kiếm toàn văn	Elasticsearch indices	Kafka (sub)
content-service	8092	Nội dung (Banner/Blog)	PostgreSQL (content_db)	REST
flash-sale-service	8093	Flash Sale	PostgreSQL (flash_sale_db), Redis stock	REST, Kafka (pub/sub)

4.5 Thiết kế cơ sở dữ liệu

4.5.1 Mô hình quan hệ — PostgreSQL

Mỗi business service sử dụng PostgreSQL sở hữu một database riêng với schema được quản lý bởi Flyway migration. Dưới đây là thiết kế cho các database cốt lõi. Các bảng sử dụng UUID làm khóa chính, đảm bảo tính duy nhất toàn cục và tránh phụ thuộc vào sequence tự tăng của database — điều quan trọng trong môi trường phân tán nơi dữ liệu có thể được tạo từ nhiều service khác nhau.

Database đơn hàng — `order_db`

Đây là database quan trọng nhất, lưu trữ toàn bộ vòng đời của đơn hàng cùng với cơ chế Outbox và Idempotency. Bảng `orders` lưu thông tin đơn hàng với các trường chính: mã đơn, mã khách hàng, phương thức thanh toán, trạng thái, tổng tiền, mã voucher (nếu có) và thời điểm hết hạn đặt chỗ. Bảng `order_items` lưu chi tiết từng mặt hàng trong đơn: mã sản phẩm, số lượng và đơn giá tại thời điểm

đặt. Bảng `outbox` lưu các sự kiện cần phát hành lên Kafka, với các trường: mã sự kiện, loại aggregate, mã aggregate, loại sự kiện, payload JSON, cờ `processed` và thời điểm xử lý. Bảng `processed_events` lưu mã các sự kiện đã được xử lý, phục vụ cơ chế Idempotent Consumer.

Database thanh toán — `payment_db`

Bảng `payments` lưu thông tin thanh toán: mã thanh toán, mã đơn hàng, phương thức, số tiền, trạng thái và thời điểm hết hạn. Bảng `payment_outbox` lưu sự kiện thanh toán ở trạng thái `PENDING`, số lần thử, lỗi cuối cùng và thời điểm phát hành, đảm bảo khả năng phát hành sự kiện thanh toán một cách tin cậy.

Database sản phẩm — `product_db`

Bảng `products` lưu thông tin sản phẩm (tên, mô tả, giá, SKU, trạng thái hoạt động), liên kết với `categories` (danh mục phân cấp) và `brands` (thương hiệu).

Database tồn kho — `inventory_db`

Bảng `inventory` lưu hai giá trị chính cho từng SKU: `quantity` là tổng số lượng còn trong kho và `reserved_quantity` là số lượng đang được đặt chỗ. Số lượng khả dụng được suy ra bằng `quantity-reserved_quantity`. Khi nhận sự kiện `order-created`, `inventory-service` tăng `reserved_quantity`; khi nhận `order-confirmed`, `service` giảm đồng thời `quantity` và `reserved_quantity`; khi nhận `order-cancelled`, `service` giảm `reserved_quantity` để giải phóng đặt chỗ.

Các database còn lại

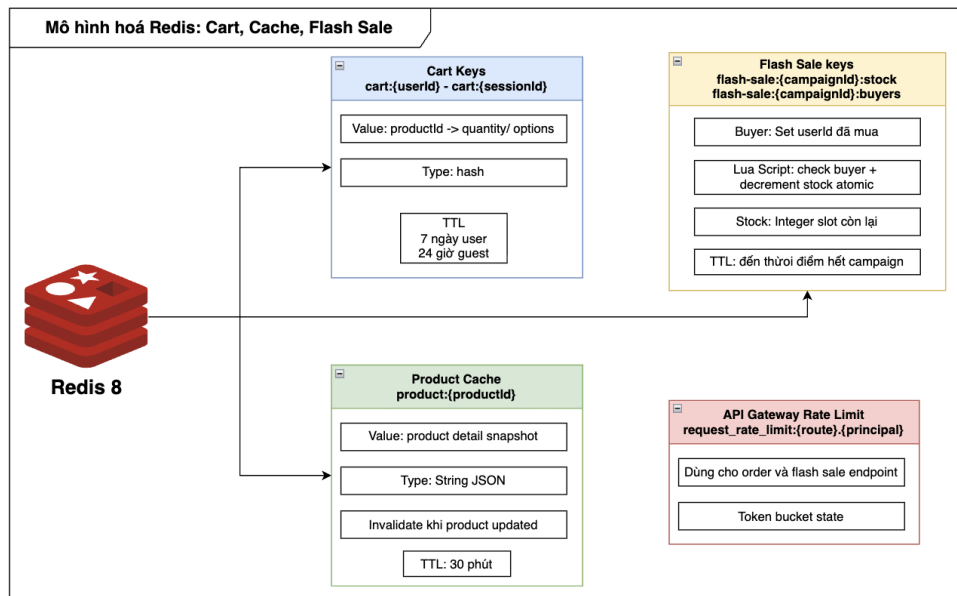
Các database `user_db`, `voucher_db`, `notification_db`, `review_db`, `content_db` và `flash_sale_db` lưu trữ dữ liệu nghiệp vụ tương ứng với mô hình quan hệ chuẩn hóa, mỗi database do một service duy nhất quản lý.

4.5.2 Mô hình tài liệu — Elasticsearch

`Search-service` sử dụng Elasticsearch để lưu trữ và truy vấn thông tin sản phẩm phục vụ tìm kiếm toàn văn. Mỗi sản phẩm được lưu dưới dạng một document trong index `products` với cấu trúc được tối ưu cho tìm kiếm: trường `name` và `description` được phân tích toàn văn (text analyzer với stemming tiếng Việt nếu có), trường `category` và `brand` được lưu dưới dạng keyword để lọc chính xác, trường `price` lưu dưới dạng số để lọc theo khoảng giá. Document được đồng bộ từ PostgreSQL của `product-service` thông qua Kafka event mỗi khi có thay đổi, đảm bảo eventual consistency giữa hai mô hình.

4.5.3 Mô hình khóa — Redis

Redis được sử dụng cho ba mục đích với cấu trúc khóa được thiết kế rõ ràng. Đối với giỏ hàng, khóa có định dạng `cart:{userId}` hoặc `cart:{sessionId}` lưu trữ danh sách sản phẩm dưới dạng Hash, với TTL 7 ngày cho người dùng đã đăng nhập và 24 giờ cho khách vắng lai. Đối với cache sản phẩm, khóa `product:{productId}` lưu thông tin sản phẩm dưới dạng JSON string với TTL 30 phút, được tự động invalidate khi sản phẩm được cập nhật. Đối với flash sale, khóa `flash-sale:{campaignId}:stock` lưu số lượng slot còn lại dưới dạng integer, được khởi tạo khi chiến dịch được kích hoạt và tự động hết hạn theo TTL bằng với thời gian còn lại của chiến dịch; khóa `flash-sale:{campaignId}:buyers` lưu tập hợp (Set) mã người dùng đã mua thành công để chống trùng lặp.



Hình 4.3: Mô hình khóa Redis: Cart, Cache, Flash Sale

4.6 Thiết kế API và giao tiếp đồng bộ

4.6.1 Định tuyến API Gateway

API Gateway định tuyến yêu cầu từ client đến các dịch vụ backend dựa trên đường dẫn. Ngoài chức năng định tuyến, Gateway còn áp dụng các chính sách bảo mật và giới hạn tốc độ khác nhau cho từng nhóm endpoint. Bảng 4.4 tổng hợp các route chính.

Bảng 4.4: Định tuyến API Gateway

Đường dẫn	Dịch vụ đích	Xác thực	Rate Limit
/api/auth/**	identity-service	Không (public)	10 req/s, burst 20
/api/products/**	product-service	Tùy chọn (GET public)	Không giới hạn
/api/search/**	search-service	Tùy chọn	Không giới hạn
/api/cart/**	cart-service	Tùy chọn (guest session)	Không giới hạn
/api/users/**	user-service	Bắt buộc	Không giới hạn
/api/orders/**	order-service	Bắt buộc	10 req/s, burst 20
/api/payments/**	payment-service	Bắt buộc (trừ VNPAY IPN)	Không giới hạn
/api/vouchers/**	voucher-service	Bắt buộc (admin cho CRUD)	Không giới hạn
/api/reviews/**	review-service	Bắt buộc (tạo), tùy chọn (xem)	Không giới hạn
/api/flash-sales/*/purchase	flash-sale-service	Bắt buộc	3 req/s, burst 5
/api/flash-sales/**	flash-sale-service	Bắt buộc (admin cho CRUD)	Không giới hạn
/api/content/**	content-service	Tùy chọn (GET public)	Không giới hạn
/api/notifications/**	notification-service	Bắt buộc	Không giới hạn

4.6.2 Giao tiếp Feign giữa các dịch vụ

Các tương tác đồng bộ giữa các dịch vụ được thực hiện thông qua OpenFeign client. Cart-service gọi product-service để xác minh tính hợp lệ của sản phẩm khi thêm vào giỏ hàng. Order-service — với vai trò Saga orchestrator — gọi product-service để kiểm tra giá và trạng thái sản phẩm, gọi inventory-service để kiểm tra tồn kho khả dụng trước khi tạo đơn, và gọi voucher-service để xác minh và đặt chỗ voucher. Review-service gọi order-service để kiểm tra người dùng đã có đơn hàng đã xác nhận chứa sản phẩm cần đánh giá hay chưa. Tất cả Feign client được cấu hình với timeout 5 giây và retry 1 lần, kết hợp với Circuit Breaker để ngăn cascading failure.

4.7 Thiết kế luồng sự kiện — Kafka Topics

Giao tiếp bất đồng bộ giữa các dịch vụ được thực hiện thông qua 13 Kafka topic, được tổ chức thành 11 nhóm nghiệp vụ. Mỗi topic có producer và consumer được xác định rõ ràng, cùng với cấu trúc payload cụ thể. Kafka được cấu hình với cơ chế tự động tạo topic (`auto.create.topics.enable=true`) cho hầu hết các topic, ngoại trừ `flash-sale-order-requested` được tạo thủ công với 3

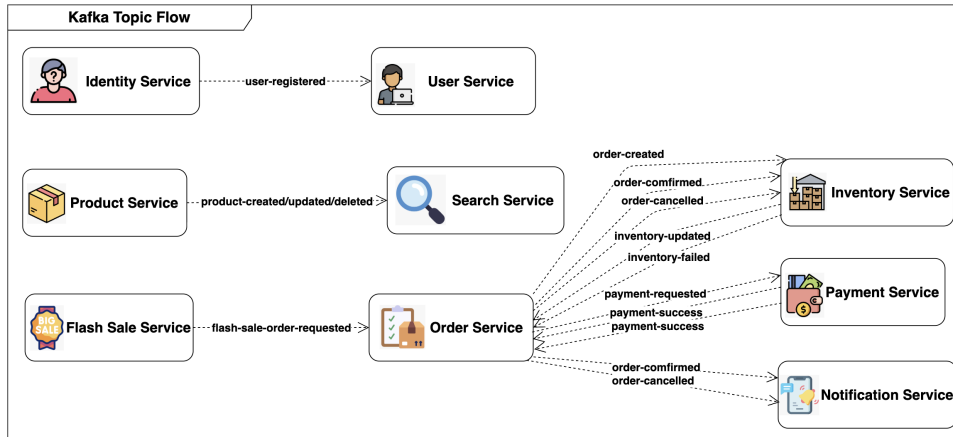
CHƯƠNG 4. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

partition để kiểm soát thứ tự xử lý.

Bảng 4.5 mô tả toàn bộ các topic, luồng sự kiện và vai trò của từng dịch vụ tham gia.

Bảng 4.5: Bản đồ Kafka Topics

Topic	Producer	Consumer	Mục đích
user-registered	identity-service	user-service	Tạo hồ sơ người dùng sau đăng ký
product-created	product-service	search-service	Index sản phẩm mới vào Elasticsearch
product-updated	product-service	search-service	Cập nhật index khi sửa sản phẩm
product-deleted	product-service	search-service	Xóa index khi xóa sản phẩm
order-created	order-service	inventory-service	Yêu cầu đặt chỗ tồn kho
inventory-updated	inventory-service	order-service	Xác nhận đặt chỗ thành công
inventory-failed	inventory-service	order-service	Thông báo không đủ tồn kho
payment-requested	order-service	payment-service	Yêu cầu xử lý thanh toán COD
payment-success	payment-service	order-service	Thanh toán thành công, xác nhận đơn
payment-failed	payment-service	order-service	Thanh toán thất bại, hủy đơn
order-confirmed	order-service	inventory-service, notification-service	Xác nhận tồn kho, gửi email
order-cancelled	order-service	inventory-service, voucher-service, notification-service	Hoàn tồn kho, hoàn voucher, gửi email
flash-sale-order-requested	flash-sale-service	order-service	Tạo đơn hàng flash sale



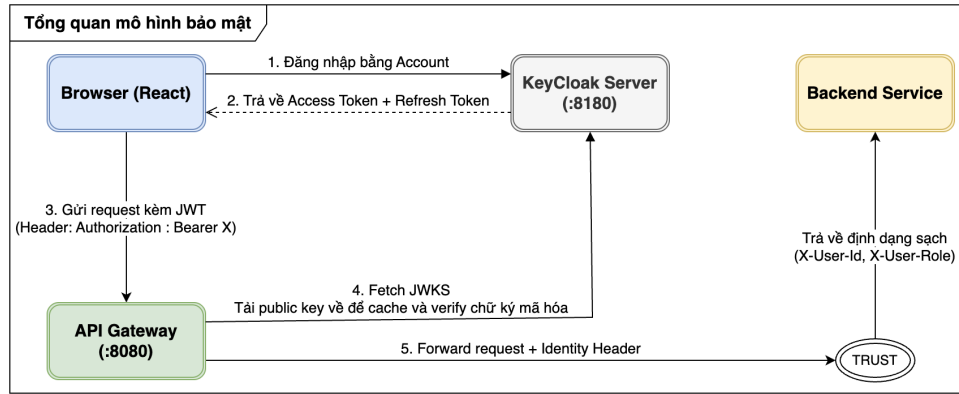
Hình 4.4: Sơ đồ luồng sự kiện Kafka giữa các dịch vụ

4.8 Thiết kế bảo mật

Hệ thống áp dụng mô hình bảo mật tập trung với API Gateway là điểm thực thi duy nhất. Luồng xác thực diễn ra qua bốn bước. Bước thứ nhất, client gửi thông tin đăng nhập đến Keycloak (qua identity-service hoặc trực tiếp) và nhận được bộ token JWT. Bước thứ hai, client gửi kèm Access Token trong header `Authorization: Bearer <token>` cho mọi request đến API Gateway. Bước thứ ba, Gateway xác minh chữ ký JWT sử dụng public key lấy từ endpoint JWKS của Keycloak, kiểm tra thời hạn và các claims cần thiết. Bước thứ tư, sau khi xác minh thành công, Gateway trích xuất thông tin định danh từ JWT (userId, roles, email) và chèn vào ba header `X-User-Id`, `X-User-Roles`, `X-User-Email` trước khi chuyển tiếp request đến dịch vụ backend.

Các dịch vụ backend không thực hiện lại việc xác minh JWT mà hoàn toàn tin tưởng vào các header do Gateway cung cấp (Trusted Subsystem Pattern). Điều kiện để mô hình này an toàn là các dịch vụ backend chỉ lắng nghe trên mạng nội bộ của Docker, không có cổng nào được mở ra bên ngoài. Phân quyền được thực hiện dựa trên vai trò: các route quản trị như `/api/orders/admin/**`, `/api/users/admin/**`, `/api/inventory/admin/**`, `/api/reviews/admin/**` và các mutation `product/voucher/flash-sale/content` yêu cầu `ROLE_ADMIN`; các endpoint nghiệp vụ người dùng yêu cầu token hợp lệ hoặc được mở public tùy chức năng.

Các endpoint đặc biệt nhạy cảm được bảo vệ bổ sung bằng Redis token bucket rate limiter. Endpoint đặt hàng (`POST /api/orders`) được giới hạn 10 request/giây với khả năng burst 20 request để ngăn chặn đặt hàng tự động. Endpoint mua flash sale (`POST /api/flash-sales/*/purchase`) được giới hạn 3 request/giây với burst 5 request, đảm bảo công bằng giữa những người tham gia.



Hình 4.5: Luồng xác thực và phân quyền qua API Gateway

4.9 Thiết kế máy trạng thái

4.9.1 Máy trạng thái đơn hàng

Đơn hàng trong hệ thống trải qua một vòng đời được kiểm soát chặt chẽ với bốn trạng thái chính. Trạng thái PENDING là trạng thái khởi tạo, ngay sau khi order-service tạo đơn hàng và ghi sự kiện `order-created` vào outbox. Trạng thái STOCK_RESERVED đạt được khi inventory-service xác nhận đã đặt chỗ tồn kho thành công. Từ đây, luồng xử lý rẽ nhánh tùy theo phương thức thanh toán: với COD, đơn hàng được xác nhận ngay lập tức; với VNPAY, đơn hàng chờ khách hàng thanh toán trong vòng 30 phút. Trạng thái CONFIRMED là trạng thái kết thúc thành công, đạt được khi thanh toán COD được xác nhận hoặc VNPAY callback thành công. Trạng thái CANCELLED là trạng thái kết thúc thất bại, xảy ra khi tồn kho không đủ, thanh toán thất bại hoặc hết thời gian chờ thanh toán.

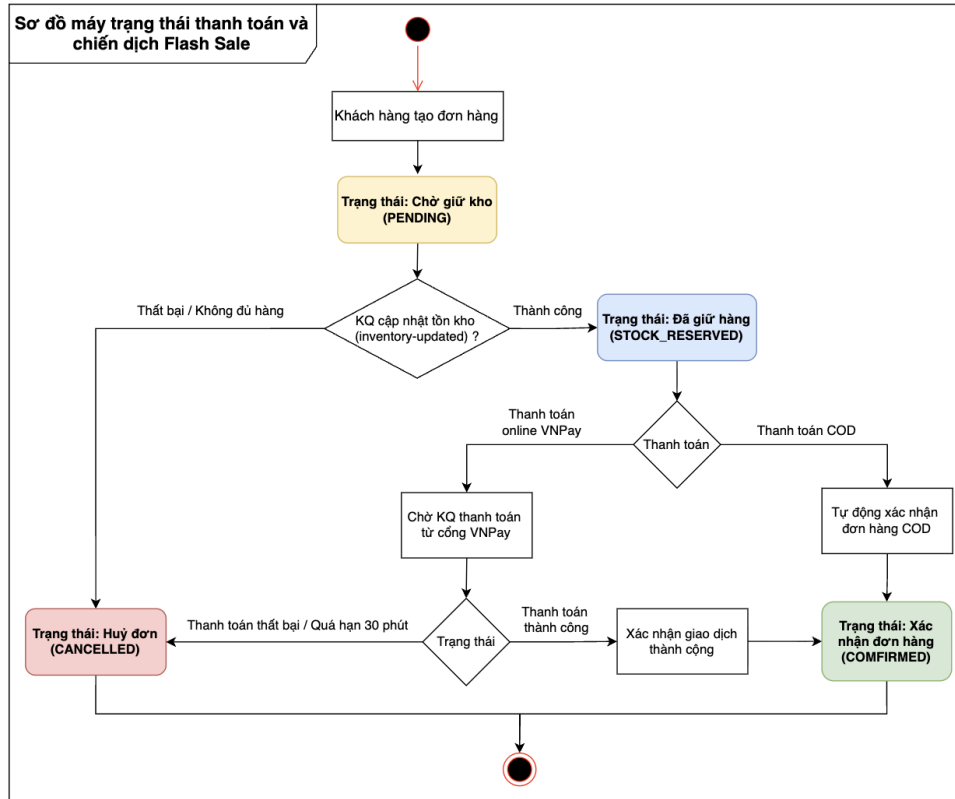
4.9.2 Máy trạng thái thanh toán

Thanh toán VNPAY có bốn trạng thái. PENDING — URL thanh toán đã được tạo, khách hàng đang trong quá trình thanh toán. COMPLETED — VNPAY xác nhận thanh toán thành công qua IPN callback hoặc Return URL. FAILED — thanh toán thất bại do khách hàng hủy, thẻ không hợp lệ hoặc số dư không đủ. TIMEOUT — quá thời gian chờ 30 phút, được phát hiện và xử lý bởi PaymentTimeoutScheduler. Một ràng buộc quan trọng là mỗi đơn hàng chỉ có thể có tối đa một thanh toán ở trạng thái PENDING tại mỗi thời điểm, được đảm bảo bằng partial unique index trên `order_id` khi `status = 'PENDING'`.

4.9.3 Máy trạng thái chiến dịch Flash Sale

Chiến dịch flash sale vận hành qua bốn trạng thái. SCHEDULED — chiến dịch đã được tạo và lên lịch, đang chờ đến thời điểm bắt đầu. ACTIVE — chiến dịch đang diễn ra, người dùng có thể tham gia mua hàng; ở trạng thái này, CampaignScheduler đã khởi tạo Redis stock key và TTL. ENDED — chiến dịch đã kết thúc theo thời gian, không nhận thêm yêu cầu mua mới. CANCELLED — chiến dịch bị

quản trị viên hủy trước hoặc trong thời gian diễn ra. Quá trình chuyển trạng thái từ SCHEDULED sang ACTIVE được thực hiện tự động bởi CampaignScheduler (chu kỳ 5 giây) dựa trên thời gian hệ thống. Việc chuyển sang ENDED cũng được CampaignScheduler xử lý khi thời gian kết thúc đã qua; khi hủy chiến dịch, service xóa các Redis key liên quan để không tiếp nhận mua mới.



Hình 4.6: Sơ đồ máy trạng thái thanh toán và chiến dịch Flash Sale

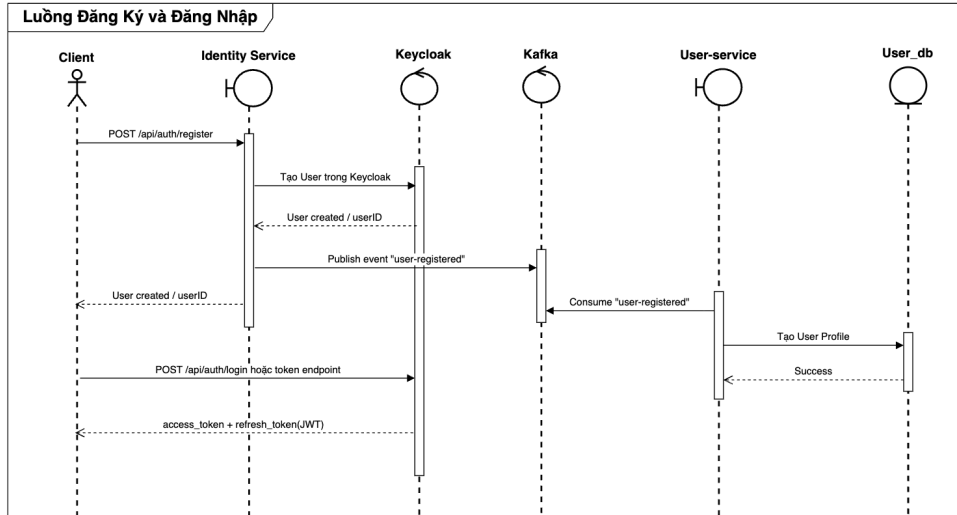
4.10 Thiết kế luồng nghiệp vụ chính

4.10.1 Luồng đăng ký người dùng

Khi người dùng gửi yêu cầu đăng ký qua `POST/api/auth/register`, identity-service tiếp nhận và gọi Keycloak Admin API để tạo tài khoản mới trong realm `ecommerce`. Sau khi tạo thành công, identity-service phát hành sự kiện `user-registered` lên Kafka, chứa thông tin: `userId`, `email`, `username` và `timestamp`. User-service tiêu thụ sự kiện này và tạo bản ghi User profile tương ứng trong `user_db`, bao gồm các trường mặc định như họ tên (từ email), trạng thái hoạt động và thời điểm tạo. Luồng này minh họa mẫu giao tiếp bất đồng bộ: identity-service không cần biết user-service tồn tại, và user-service có thể được triển khai hoặc thay đổi độc lập.

4.10.2 Luồng đặt hàng COD — Saga Orchestration

Đây là luồng phức tạp nhất, thể hiện đầy đủ Saga Orchestration Pattern với order-service đóng vai trò orchestrator. Luồng diễn ra qua các bước sau:



Hình 4.7: Biểu đồ tuần tự luồng đăng kí và đăng nhập người dùng

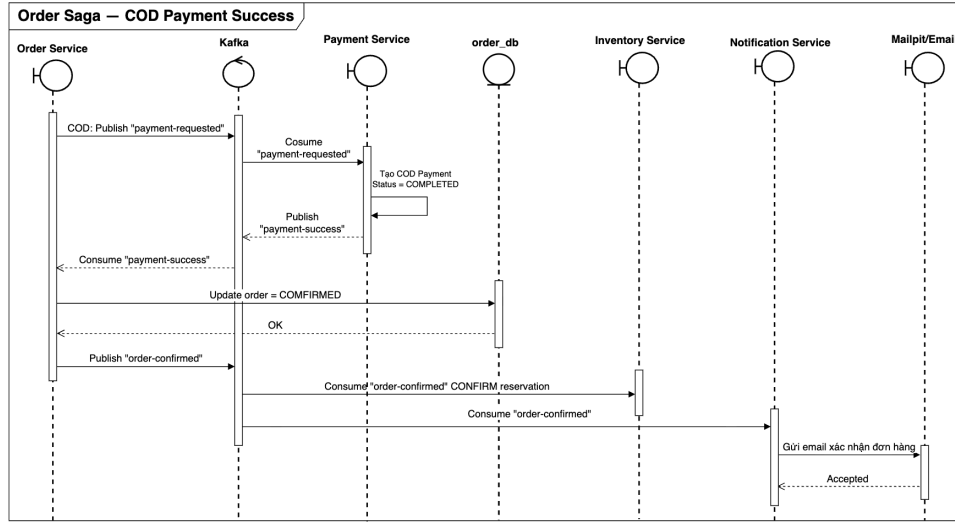
Bước 1 — Tạo đơn hàng. Khách hàng gửi yêu cầu `POST/api/orders` với payload chứa danh sách sản phẩm (`productId`, `quantity`), phương thức thanh toán COD, địa chỉ giao hàng và mã voucher (nếu có). Order-service thực hiện ba lời gọi Feign đồng bộ: gọi product-service để xác minh sản phẩm tồn tại và đang hoạt động, lấy đơn giá hiện tại; gọi inventory-service để kiểm tra số lượng tồn kho khả dụng cho từng sản phẩm; gọi voucher-service để xác minh và đặt chỗ voucher (nếu có). Nếu tất cả đều hợp lệ, order-service tạo bản ghi đơn hàng với trạng thái PENDING và đồng thời ghi sự kiện `order-created` vào bảng `outbox` trong cùng một giao dịch.

Bước 2 — Đặt chỗ tồn kho. OutboxPoller (chu kỳ 1 giây) của order-service phát hành sự kiện `order-created` lên Kafka. Inventory-service tiêu thụ sự kiện, thực hiện đặt chỗ cho từng sản phẩm trong đơn hàng bằng cách tăng `reserved_quantity` trong khi `quantity` chưa đổi. Nếu tất cả sản phẩm đều đủ số lượng, inventory-service phát hành `inventory-updated` lên Kafka. Nếu bất kỳ sản phẩm nào không đủ, phát hành `inventory-failed`.

Bước 3 — Xác nhận đơn COD. Order-service tiêu thụ `inventory-updated`, chuyển trạng thái đơn hàng sang `STOCK_RESERVED`. Với phương thức COD, quá trình thanh toán được xử lý tự động: order-service phát hành `order-confirmed` ngay lập tức. Inventory-service tiêu thụ `order-confirmed`, thực hiện confirm reservation (trừ hẵn số lượng tồn kho). Notification-service tiêu thụ `order-confirmed`, gửi email xác nhận đơn hàng đến khách hàng. Đơn hàng đạt trạng thái cuối cùng `CONFIRMED`.

Bước 4 — Xử lý thất bại. Nếu inventory-service phát hành `inventory-failed`, order-service chuyển trạng thái đơn hàng sang `CANCELLED` với lý do không đủ tồn kho, phát hành `order-cancelled`. Inventory-service và voucher-

service tiêu thụ `order-cancelled` để hoàn trả đặt chỗ và voucher. Notification-service gửi email thông báo hủy đơn.



Hình 4.8: Biểu đồ tuần tự luồng đặt hàng COD — Saga Orchestration

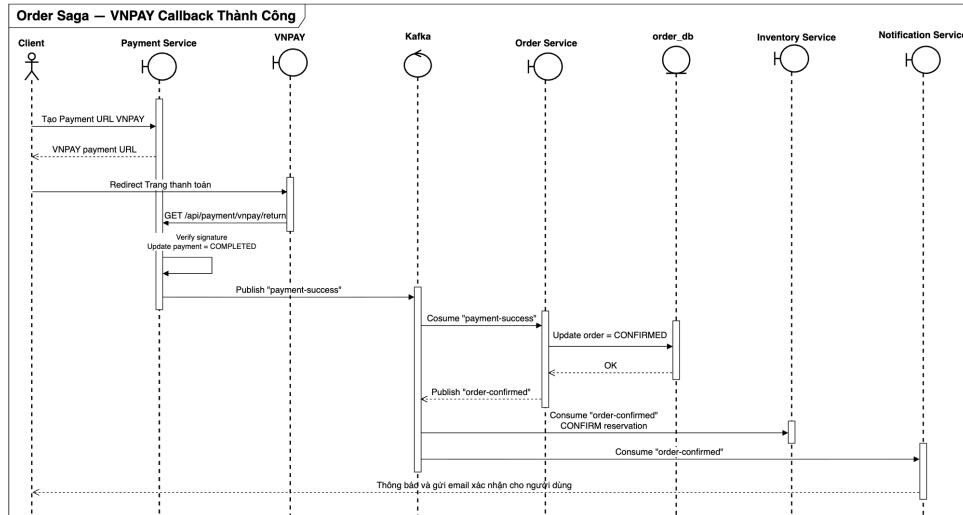
4.10.3 Luồng đặt hàng VNPAY

Luồng VNPAY mở rộng luồng COD với các bước bổ sung cho thanh toán trực tuyến. Sau khi đơn hàng đặt trạng thái `STOCK_RESERVED`, order-service thiết lập `reservation_expired_at=NOW()+30` phút. Khách hàng gọi `POST/api/payments/vnpay/create` để tạo URL thanh toán. Payment-service sinh URL với chữ ký HMAC-SHA512 và trả về cho client. Khách hàng được chuyển hướng đến trang thanh toán VNPAY.

Khi VNPAY gọi IPN callback với kết quả thanh toán, payment-service xác minh chữ ký và cập nhật trạng thái. Nếu thành công, payment-service phát hành `payment-success` lên Kafka; order-service tiêu thụ và chuyển đơn hàng sang `CONFIRMED`. Nếu thất bại, phát hành `payment-failed` và đơn hàng bị hủy với cơ chế hoàn trả tương tự. Nếu sau 30 phút không có callback, `ReservationExpiryScheduler` (order-service) và `PaymentTimeoutScheduler` (payment-service) cùng hoạt động từ hai phía để đảm bảo đơn hàng và thanh toán đều được chuyển sang trạng thái kết thúc phù hợp.

4.10.4 Luồng Flash Sale

Luồng flash sale được thiết kế đặc biệt để xử lý tình huống đồng thời cao với nhiều lớp bảo vệ. Khi khách hàng gọi `POST/api/flash-sales/{campaignId}/purchase`, flash-sale-service thực hiện tuần tự các bước kiểm tra. Đầu tiên, kiểm tra thời gian hiệu lực của chiến dịch (phải ở trạng thái `ACTIVE` và trong khoảng thời gian cho phép). Tiếp theo, thực thi Redis Lua script nguyên tử để kiểm tra buyer set và giảm số lượng slot: script kiểm tra khóa `flash-sale:{campaignId}:buyers` để chống mua trùng, sau đó kiểm tra khóa



Hình 4.9: Biểu đồ tuần tự luồng đặt hàng VNPAY với xử lý IPN callback

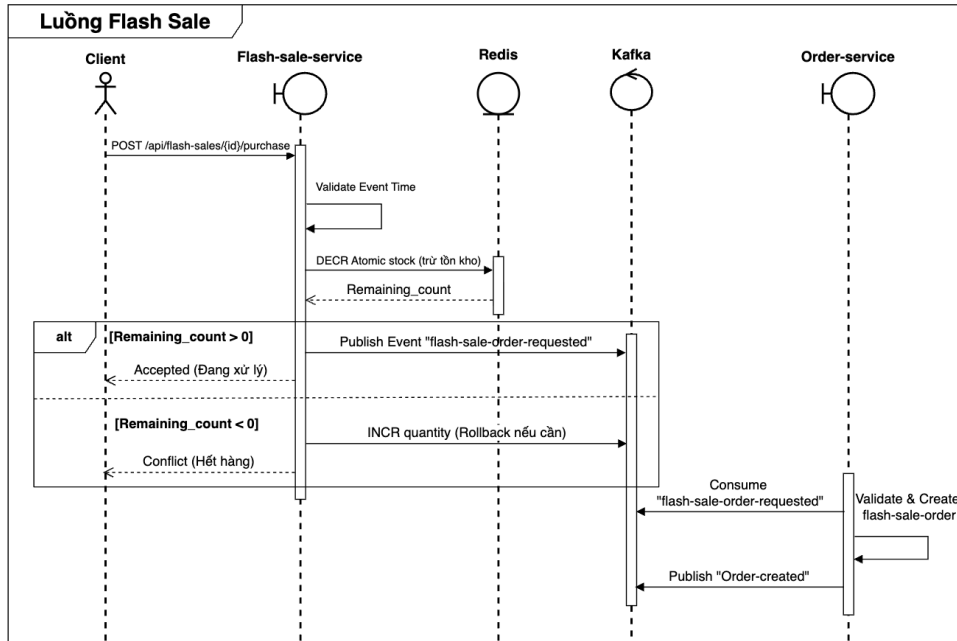
`flash-sale:{campaignId}:stock`; nếu stock lớn hơn 0 thì giảm 1 và thêm user vào buyer set, ngược lại trả về thất bại. Nếu Redis trả về thất bại, service phản hồi HTTP 409 Conflict — sản phẩm đã bán hết hoặc người dùng đã mua trước đó.

Nếu Redis trả về thành công, `flash-sale-service` tăng `soldCount` trong database và phát hành sự kiện `flash-sale-order-requested` lên Kafka. Order-service tạo đơn hàng flash sale với giá đã giảm theo chiến dịch và xử lý tương tự luồng COD. Nếu bước tạo đơn thất bại, `flash-sale-service` dùng compensation script để hoàn lại slot Redis và loại người dùng khỏi buyer set. Reconciliation-Scheduler định kỳ đối chiếu `soldCount` với số đơn flash-sale thực tế trong order-service để phát hiện và bù trừ slot mồi.

CampaignScheduler (chu kỳ 5 giây, với Redis distributed lock) đảm nhiệm ba nhiệm vụ nền: kích hoạt chiến dịch đến hạn (chuyển SCHEDULED sang ACTIVE, seed Redis stock), khôi phục khóa Redis bị mất (đọc `soldCount` từ database để tính lại stock còn lại), và kết thúc chiến dịch hết hạn (chuyển ACTIVE sang ENDED).

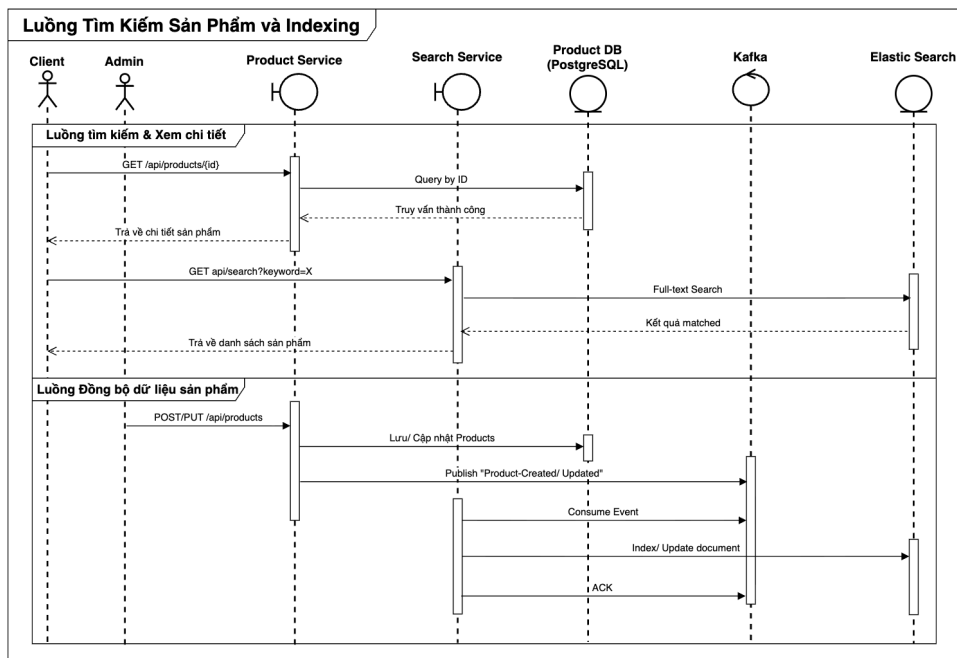
4.10.5 Luồng tìm kiếm sản phẩm (CQRS-lite)

Luồng tìm kiếm minh họa áp dụng CQRS-lite với hai đường dữ liệu độc lập. Đường ghi (Command): khi quản trị viên tạo hoặc cập nhật sản phẩm qua `product-service`, dữ liệu được lưu vào PostgreSQL (`product_db`). Đồng thời, `product-service` phát hành sự kiện `product-created` hoặc `product-updated` trực tiếp lên Kafka. Search-service tiêu thụ sự kiện và cập nhật document tương ứng trong Elasticsearch index. Đường đọc (Query): khi khách hàng tìm kiếm sản phẩm, request được gửi trực tiếp đến `search-service`, truy vấn Elasticsearch và trả về kết quả đã được tối ưu cho tìm kiếm toàn văn, không thông qua PostgreSQL. Hai đường



Hình 4.10: Biểu đồ tuần tự luồng mua hàng Flash Sale với cơ chế ba lớp bảo vệ

dữ liệu hoạt động độc lập, chấp nhận độ trễ đồng bộ nhỏ (thường dưới 1 giây), đảm bảo eventual consistency.



Hình 4.11: Biểu đồ tuần tự luồng tìm kiếm sản phẩm theo mô hình CQRS-lite

4.11 Thiết kế tác vụ nền — Scheduled Jobs

Hệ thống có sáu tác vụ nền (scheduled jobs) chạy định kỳ để xử lý các tình huống mà cơ chế event-driven không bao phủ được, đặc biệt là các tình huống liên quan đến timeout và phục hồi. Bảng 4.6 tổng hợp toàn bộ các tác vụ.

Bảng 4.6: Tác vụ nền trong hệ thống

Tác vụ	Dịch vụ	Chu kỳ	Nhiệm vụ
ReservationExpiry Scheduler	order-service	60 giây	Hủy đơn VNPAY quá hạn 30 phút, giải phóng voucher
PaymentTimeout Scheduler	payment-service	60 giây	Chuyển payment PENDING quá hạn sang TIMEOUT
OutboxPoller (Order)	order-service	1 giây	Phát hành sự kiện chưa gửi từ bảng outbox
OutboxPoller (Payment)	payment-service	1 giây	Phát hành sự kiện chưa gửi từ bảng payment_outbox
Campaign Scheduler	flash-sale-service	5 giây	Kích hoạt campaign, khôi phục Redis key, kết thúc campaign hết hạn
Reconciliation Scheduler	flash-sale-service	300 giây	Đối chiếu soldCount với số đơn flash-sale thực tế và bù slot mở còi

Hai scheduler xử lý hết hạn reservation và timeout payment hoạt động song song để xử lý cùng một tình huống từ hai phía, tạo cơ chế dự phòng chéo: nếu một scheduler gặp sự cố hoặc bỏ sót, scheduler còn lại vẫn đảm bảo tài nguyên (tồn kho, voucher) được giải phóng. OutboxPoller với chu kỳ 1 giây đảm bảo độ trễ phát hành sự kiện thấp trong điều kiện bình thường, đồng thời phục hồi các sự kiện bị bỏ lỡ sau khi service khởi động lại. CampaignScheduler sử dụng Redis distributed lock (SETNX với TTL) để đảm bảo chỉ một instance của flash-sale-service thực thi tại mỗi thời điểm, tránh xung đột trong môi trường nhiều replica.

CHƯƠNG 5. CÀI ĐẶT, TRIỂN KHAI, KIỂM THỬ VÀ ĐÁNH GIÁ

Chương này trình bày quá trình hiện thực hóa thiết kế đã phân tích ở Chương 4. Do báo cáo được giới hạn trong năm chương, nội dung cài đặt, triển khai, kiểm thử và đánh giá được tổ chức chung trong một chương. Cách tổ chức này giúp thể hiện mạch đầy đủ từ thiết kế đến hệ thống chạy được, đồng thời vẫn tách rõ phần hiện thực và phần kiểm chứng bằng các mục riêng biệt.

5.1 Môi trường phát triển và cấu trúc dự án

Hệ thống được phát triển theo mô hình monorepo để thuận tiện cho việc quản lý dependency, build, kiểm thử và chạy toàn bộ stack trong phạm vi đồ án. Backend sử dụng Java 21, Spring Boot 3.5.13, Spring Cloud 2025.0.0 và Maven Wrapper. Frontend sử dụng Next.js App Router, TypeScript và Server Actions. Toàn bộ môi trường runtime được chuẩn hóa bằng Docker Compose.

Bảng 5.1: Môi trường và công cụ phát triển

Nhóm	Công cụ/Phiên bản	Vai trò trong đồ án
Ngôn ngữ back-end	Java 21	Phát triển 13 business service và 3 infrastructure service
Framework back-end	Spring Boot 3.5.13, Spring Cloud 2025.0.0	REST API, Gateway, Eureka, Config, OpenFeign, Kafka, Security, Actuator
Build backend	Maven Wrapper	Build multi-module, quản lý dependency và plugin tập trung
Frontend	Next.js App Router, TypeScript	Storefront, Admin Panel, BFF/proxy, Server Actions
Container	Docker Compose	Khởi động local stack và production-like stack trên EC2
Kiểm thử	JUnit 5, Testcontainers, k6, Bash script	Unit, integration, smoke, performance và resilience test
Triển khai	AWS EC2, Caddy, GitHub Actions, GHCR	Production-like single-host deployment và build/push image

Về cấu trúc mã nguồn, repository gồm module `common`, 13 business service như `identity`, `user`, `product`, `inventory`, `cart`, `order`, `payment`, `voucher`, `notification`, `review`, `search`, `content` và `flash-sale`; ba service hạ tầng `api-gateway`, `discovery-server`, `config-server`; và thư mục `frontend`. Mỗi service backend có `Dockerfile`, cấu hình riêng và nếu có trạng thái thì có Flyway migration quản lý schema.

5.2 Cài đặt các thành phần backend cốt lõi

5.2.1 Spring Cloud layer

Ba service hạ tầng tạo thành lớp điều phối cho toàn bộ backend. `api-gateway` là điểm vào duy nhất cho client, chịu trách nhiệm định tuyến, xác thực JWT, gán trusted headers và rate limiting. `discovery-server` sử dụng Eureka để các service đăng ký và phát hiện lẫn nhau theo tên logic thay vì hard-code địa chỉ. `config-server` cung cấp cấu hình tập trung từ classpath, giúp các service lấy cấu hình theo profile khi khởi động.

5.2.2 Database per Service và Flyway

Các service stateful sở hữu database logic riêng trên PostgreSQL. Cách tổ chức này đảm bảo nguyên tắc Database per Service: service khác không đọc/ghi trực tiếp vào database không thuộc quyền sở hữu của mình. Schema được quản lý bằng Flyway migration và Hibernate chạy với chế độ validate, nhờ đó sai lệch giữa entity và schema được phát hiện sớm khi service khởi động.

Bảng 5.2: Map service với lớp lưu trữ chính

Service	Lưu trữ chính	Ghi chú
identity-service	Keycloak	Tạo user trong realm, phát sự kiện đăng ký
user-service	PostgreSQL	Hồ sơ người dùng, địa chỉ, dữ liệu admin user
product-service	PostgreSQL, Redis	Catalog, category, brand, cache product list/detail
inventory-service	PostgreSQL	Tồn kho, reserved quantity, stock movement
cart-service	Redis	Giỏ hàng user/guest với TTL
order-service	PostgreSQL, Kafka	Order, order item, outbox, processed events
payment-service	PostgreSQL, Kafka	Payment, payment outbox, processed events
search-service	Elasticsearch	Read model phục vụ full-text search
flash-sale-service	PostgreSQL, Redis, Kafka	Campaign, Redis Lua reservation, reconciliation

5.2.3 Saga Orchestration, Outbox và Idempotent Consumer

Luồng đặt hàng là phần thể hiện rõ nhất các mẫu thiết kế phân tán. `order-service` đóng vai trò orchestrator: tạo đơn, đặt chỗ voucher, phát sự kiện `order-created`, nhận kết quả từ `inventory/payment` và quyết định xác nhận hoặc hủy đơn. Để tránh lỗi dual-write, `order-service` ghi sự kiện vào bảng `outbox` trong cùng transaction với dữ liệu đơn hàng. `OutboxPoller` quét 1 giây/lần, lấy batch chưa xử lý bằng `FOR UPDATE SKIP LOCKED`, gửi Kafka và đánh dấu `processed=true`.

Ở phía consumer, các service như order, inventory và payment sử dụng bảng `processed_events` để lưu `eventId` đã xử lý. Nếu Kafka gửi lại cùng một event, consumer bỏ qua thay vì thực thi nghiệp vụ lần hai. Cách làm này chấp nhận at-least-once delivery ở tầng message broker nhưng đạt hiệu ứng exactly-once processing ở tầng ứng dụng.

5.2.4 Thanh toán VNPAY

`payment-service` tích hợp VNPAY sandbox theo ba bước chính. Khi người dùng chọn thanh toán online, service tạo payment ở trạng thái `PENDING`, sinh URL thanh toán và ký tham số bằng HMAC-SHA512. Khi VNPAY gọi IPN hoặc người dùng quay lại Return URL, service xác minh chữ ký, kiểm tra số tiền, cập nhật trạng thái payment và ghi sự kiện vào `payment_outbox`. `PaymentEventOutboxPoller` phát sự kiện `payment-success` hoặc `payment-failed` để `order-service` hoàn tất Saga. Nếu payment quá hạn, `PaymentTimeoutScheduler` chuyển trạng thái sang `TIMEOUT`.

5.2.5 Flash sale high-concurrency

`flash-sale-service` xử lý mua flash sale bằng Redis Lua script để thao tác kiểm tra buyer set, kiểm tra stock và giảm slot trong một lệnh nguyên tử. Redis key chính có dạng `flash-sale:{campaignId}:stock` và `flash-sale:{campaignId}:buyers`, được đặt TTL theo thời điểm kết thúc chiến dịch. Sau khi giữ slot thành công, service phát event `flash-sale-order-requested`; nếu tạo đơn thất bại, compensation script hoàn lại slot và xóa buyer khỏi set.

Hai scheduler hỗ trợ flash-sale vận hành ổn định. `CampaignScheduler` chạy 5 giây/lần, dùng Redis distributed lock để chỉ một instance kích hoạt chiến dịch đến hạn, khôi phục Redis stock key bị mất và kết thúc campaign hết hạn. `ReconciliationScheduler` chạy 300 giây/lần, đối chiếu `soldCount` trong `flash-sale-service` với số order flash-sale thực tế ở `order-service`, từ đó phát hiện và bù slot mờ côi khi cần.

5.3 Cài đặt bảo mật và phân quyền

Bảo mật được thực thi tập trung tại API Gateway. Client gửi access token dạng `Authorization:Bearer<token>` tới Gateway; Gateway xác minh chữ ký RS256 qua JWKS của Keycloak, kiểm tra thời hạn token, trích xuất `userId/email/roles` rồi gắn vào ba trusted headers: `X-User-Id`, `X-User-Email`, `X-User-Roles`. Backend service chỉ tin headers do Gateway đưa vào trong mạng nội bộ Docker, không mở cổng trực tiếp ra ngoài.

Bảng 5.3: Chính sách phân quyền tại API Gateway

Nhóm endpoint	Quyền yêu cầu	Ghi chú
GET/api/products/**, GET/api/search/**, GET/api/content/**	Public	Cho phép người dùng duyệt catalog, search và nội dung CMS
/api/cart/**	Public hoặc token	Hỗ trợ guest cart bằng session id và user cart bằng token
POST/api/orders, user profile, review create	Authenticated	Yêu cầu token hợp lệ và trusted headers
/api/orders/admin/**, /api/users/admin/**, /api/inventory/admin/**, /api/reviews/admin/**	ROLE_ADMIN	Endpoint quản trị bổ sung trong Phase 14
Product/voucher/flash-sale/content mutations	ROLE_ADMIN	POST/PUT/DELETE được bảo vệ theo method và path
POST/api/payments/vnpay/ipn	Public có HMAC	Public vì do VNPAY gọi, nhưng bắt buộc xác minh chữ ký

Ngoài xác thực và RBAC, Gateway áp dụng Redis token bucket rate limiter cho các endpoint nhạy cảm. Luồng đặt hàng được giới hạn 10 request/giây, burst 20; luồng mua flash sale được giới hạn 3 request/giây, burst 5. Rate limiter không thay thế kiểm soát nghiệp vụ nhưng giúp giảm tải và hạn chế automation trong các luồng có rủi ro cao.

5.4 Cài đặt frontend Storefront và Admin Panel

Frontend được xây dựng trong thư mục `frontend/` bằng Next.js App Router. Ứng dụng có hai khu vực chính: storefront cho khách hàng và Admin Panel cho quản trị viên. Storefront hỗ trợ duyệt sản phẩm, xem chi tiết, tìm kiếm, giỏ hàng, checkout, xem đơn hàng, thanh toán VNPAY và các màn hình flash-sale, content, review phục vụ demo. Admin Panel nằm dưới route `/admin/`, dùng chung code-base và container với storefront.

Bảng 5.4: Các module chính của Admin Panel

Module	Route frontend	Endpoint backend chính
Dashboard	/admin/dashb oard	/api/orders/admin/analytics/*, /api/inventory/low-stock
Products	/admin/produ cts	/api/products
Inventory	/admin/inven tory	/api/inventory/admin, /api/invento ry/stock-in, /api/inventory/stock -out
Orders	/admin/order s	/api/orders/admin
Users	/admin/users	/api/users/admin
Vouchers	/admin/vouch ers	/api/vouchers
Flash sales	/admin/flash -sales	/api/flash-sales
Content/Reviews	/admin/conte nt/*, /admin /reviews	/api/content/*, /api/reviews/admin

Các mutation trong Admin Panel được thực hiện bằng Server Actions. Action gọi `adminFetch` hoặc `adminMutate`, phía server đọc access token từ `httpOnly` cookie và gọi API Gateway. Sau khi mutation thành công, action `revalidatePath` để làm mới dữ liệu và redirect về trang danh sách. Thiết kế này giúp token không xuất hiện trong JavaScript phía trình duyệt, đồng thời vẫn tận dụng được cơ chế caching/revalidation của Next.js.

Giới hạn của frontend trong phạm vi đề án được ghi nhận rõ: ảnh sản phẩm/banner nhập bằng URL thay vì upload file thật; review-service mới hỗ trợ list/delete, chưa có workflow approve/reject đầy đủ; chức năng ban/unban user chưa tích hợp Keycloak Admin API; dashboard analytics tổng hợp trực tiếp từ order-service, phù hợp demo nhưng chưa phải kiến trúc analytics quy mô lớn.

5.5 Docker Compose và vận hành local

Môi trường local dùng `docker-compose.yml` để khởi động toàn bộ backend và hạ tầng. Các container được đặt chung network `ecommerce-network`; các stateful service dùng Docker volume để giữ dữ liệu giữa các lần restart. Healthcheck được khai báo cho PostgreSQL, Redis, Kafka, Elasticsearch, Keycloak, Eureka và các service Spring Boot. Nhờ đó Compose khởi động theo thứ tự hợp lý thay vì chỉ dựa vào thứ tự khai báo.

Các endpoint vận hành local quan trọng gồm API Gateway `http://localhost:8080`, Eureka `http://localhost:8761`, Keycloak `http://lo`

calhost:8180, Prometheus <http://localhost:9090>, Zipkin <http://localhost:9411>, Mailpit <http://localhost:8025> và Elasticsearch <http://localhost:9200>. Grafana mặc định dùng cổng 3000 trong local backend stack; khi chạy frontend cùng lúc cần remap hoặc dùng cấu hình production nơi Grafana được đưa sang cổng 3001.

5.6 Triển khai production-like trên AWS EC2

Môi trường triển khai thực nghiệm sử dụng một máy ảo AWS EC2 single-host tại khu vực ap-southeast-1. Đây không phải kiến trúc high availability, nhưng phù hợp với phạm vi DATN: đủ để chứng minh hệ thống có thể build image, chạy container, cấu hình reverse proxy/HTTPS, giám sát runtime và thực hiện các bài kiểm thử hiệu năng/resilience trên môi trường gần thực tế.

Bảng 5.5: Thành phần triển khai production-like

Thành phần	Vai trò
AWS EC2	Máy chủ single-host chạy Docker Compose production stack
Docker Compose production override	Dùng GHCR image, cấu hình memory limit/reservation, remap Grafana 3001
Caddy	Reverse proxy, HTTPS tự động qua Let's Encrypt và nlp.io
GitHub Actions	Build/push backend và frontend image lên GHCR, hỗ trợ deploy lên EC2
GHCR	Registry lưu Docker image theo service

File `docker-compose.prod.yml` không build image trực tiếp trên EC2 mà tham chiếu image theo dạng `ghcr.io/{owner}/{service}:{tag}`. Cách này giảm thời gian deploy trên máy chủ, tách build khỏi runtime và giúp cùng một image có thể được kéo lại khi cần rollback. Production override cũng đặt `mem_limit` và `mem_reservation` cho từng container, giúp stack ổn định hơn trong giới hạn tài nguyên của EC2. Caddy định tuyến các subdomain `app.*`, `api.*`, `keycloak.*` và `grafana.*`; Grafana dùng host port 3001 để không xung đột với frontend.

5.7 Chiến lược kiểm thử

Kiểm thử được tổ chức theo nhiều tầng. Tầng thấp nhất là unit test và integration test cho logic nghiệp vụ quan trọng. Tầng tiếp theo là backend readiness smoke test, chạy theo các suite tách nhỏ để tránh quá tải tài nguyên và giúp khoanh vùng lỗi nhanh. Tầng cao hơn là kiểm thử hiệu năng bằng k6 trên môi trường AWS EC2 và kiểm thử resilience bằng cách chủ động dừng service/hạ tầng rồi quan sát khả năng tự phục hồi.

Bảng 5.6: Kết quả backend readiness smoke test

Suite	Kết quả	Artifact
Maven	PASS, 5 pass, 0 fail	01-maven-PASS.md
Infra	PASS, 27 pass, 0 fail	02-infra-PASS.md
Auth/user	PASS, 30 pass, 0 fail	03-auth-user-PASS-after-fix.md
Catalog/search	PASS, 49 pass, 0 fail	04-catalog-search-PASS.md
Cart/voucher	PASS, 57 pass, 0 fail	05-cart-voucher-PASS-after-fix.md
Order COD/review	PASS, 67 pass, 0 fail	06-order-cod-review-PASS.md
VNPAY	PASS, 55 pass, 0 fail	07-vnpay-PASS.md
Flash-sale	PASS, 62 pass, 0 fail	08-flash-sale-PASS-after-fix.md
Security	PASS, 76 pass, 0 fail, 1 warn	09-security-PASS-trimmed-memory.md

Kết quả tổng hợp cho thấy 9/9 split suites đạt. Các lỗi phát hiện trong quá trình chuẩn bị kiểm thử, như healthcheck bị 401, guest cart session id bị reject, lỗi serializer cache sản phẩm và cấu hình Metaspace cho flash-sale test, đã được sửa trước khi ghi nhận kết quả final.

5.8 Kết quả kiểm thử hiệu năng

Các bài kiểm thử hiệu năng được thực hiện trên môi trường AWS EC2 chạy Docker Compose production-like stack. Load generator là máy cá nhân chạy k6. Dataset thực nghiệm gồm 100 sản phẩm, 500 user/token và một flash-sale campaign có 100 slot. Nguyên tắc đưa số liệu vào báo cáo là mọi metric phải đối chiếu được với raw artifact trong thư mục `.test/results`.

Bảng 5.7: Tổng hợp kết quả kiểm thử hiệu năng bằng k6

Kịch bản	Thời lượng	Max VU	Requests	p95	Lỗi
Catalog soak	34m03s	200	220,956	61.68 ms	0.00%
Checkout stress	7m00s	50	13,734	160.92 ms	0.00%
Flash-sale spike	1m30s	500	78,448	1.04 s over-all	0 duplicate buyer

Catalog soak mô phỏng người dùng duyệt danh sách sản phẩm trong 34 phút 03 giây với tải tối đa 200 VU. Kịch bản tạo 55,239 iterations và 220,956 HTTP

requests, p95 đạt 61.68 ms, p99 đạt 85.48 ms, error rate 0.00%. Kết quả này cho thấy read path catalog/search/cache đáp ứng tốt trên single-host trong phạm vi dataset đồ án.

Checkout stress mô phỏng tạo đơn hàng với tải tối đa 50 VU trong 7 phút. Kịch bản ghi nhận 6,817 iterations, 13,734 HTTP requests, order success check 100.00%, HTTP failure rate 0.00% và p95 latency 160.92 ms. Đây là luồng nặng hơn catalog vì đi qua product, inventory, voucher, order, Kafka và notification, nhưng vẫn duy trì độ trễ phù hợp với mục tiêu thực nghiệm.

Flash-sale spike mô phỏng 500 VU trong 90 giây tranh mua 100 slot. Kết quả có đúng 100 purchase thành công trên 100 slot, soldCount của campaign bằng 100, số đơn confirmed cho campaign bằng 100 và không có duplicate buyer. Bài test ghi nhận 36,134 phản hồi sold-out; p95 tổng thể là 1.04 giây, còn p95 của nhóm expected responses là 335.55 ms. Điều này chứng minh trong kịch bản thực nghiệm đã chạy, Redis Lua atomic reservation kết hợp buyer set, compensation và reconciliation không gây over-sell.

5.9 Kết quả kiểm thử khả năng chịu lỗi

Để đánh giá resilience, bốn kịch bản được thực hiện bằng cách chủ động gây lỗi vào service hoặc hạ tầng. Mục tiêu không phải chứng minh hệ thống có high availability hoàn chỉnh, mà là kiểm chứng các cơ chế đã thiết kế: circuit breaker/-fallback, outbox replay, graceful degradation và Saga compensation.

Bảng 5.8: Tổng hợp kết quả kiểm thử khả năng chịu lỗi

Kịch bản	Kết quả chính	Trạng thái
Kill order-service	Trong 30 giây downtime, order-created đạt 47.15%, HTTP failure 25.94%; sau restart gateway/order health trả 200	Completed
Kill Kafka	Khi Kafka down, order vẫn được tạo, outbox row ở trạng thái pending; sau Kafka restart, event được process và order kết thúc CONFIRMED	PASS
Kill Redis	Product list vẫn trả 200 trong Redis outage; cart phụ thuộc Redis degrade 500 trong outage và recover 200 sau restart	PASS
Inventory failure compensation	Order 4170422f-fc2d-49f8-aa5f-c7cd1d79266a chuyển CANCELLED, inventory còn quantity=0, reserved_quantity=0	PASS

Kịch bản Kill Kafka là bằng chứng trực tiếp cho Transactional Outbox: service nghiệp vụ vẫn ghi được dữ liệu và ghi event pending vào database khi broker không khả dụng; sau khi broker khởi động lại, poller phát lại event và Saga tiếp tục đến trạng thái cuối. Kịch bản Kill Redis cho thấy hệ thống có phân biệt read path có

fallback (product list) với chức năng phụ thuộc trực tiếp Redis (cart/rate-limit/flash-sale). Cách ghi nhận này trung thực với bản chất hệ thống: một số luồng có graceful degradation, một số luồng phải fail fast để tránh sai dữ liệu.

5.10 Đánh giá tổng thể

5.10.1 Đối chiếu với mục tiêu đề tài

Bảng 5.9: Đối chiếu kết quả với mục tiêu ban đầu

Mục tiêu	Kết quả đạt được	Đánh giá
Phân rã hệ thống theo microservices	13 business service, 3 Spring Cloud service, database/cache/search tách theo ownership	Đạt
Áp dụng distributed patterns	Saga Orchestration, Transactional Outbox, Idempotent Consumer, CQRS-lite, scheduler/reconciliation	Đạt
Bảo mật tập trung	Keycloak, JWT validation tại Gateway, trusted headers, RBAC, rate limiter	Đạt
Thanh toán trực tuyến	VNPAY sandbox, HMAC-SHA512, Return URL, IPN, timeout scheduler	Đạt
Flash sale chống over-sell	Redis Lua, buyer set, compensation, reconciliation; spike test 500 VU xác nhận 100/100 slot và 0 duplicate buyer	Đạt
Triển khai và quan sát	Docker Compose local/AWS EC2, Caddy HTTPS, GitHub Actions/GHCR, Prometheus/Grafana/Zipkin	Đạt trong phạm vi DATN
Giao diện demo hệ thống hoàn chỉnh	Next.js storefront và Admin Panel với các module quản trị chính	Đạt mức demo DATN

5.10.2 Hạn chế

Hệ thống đã đạt mục tiêu của đề án nhưng vẫn có các giới hạn cần ghi nhận. Thứ nhất, môi trường triển khai là single-host EC2 nên chưa có high availability; nếu EC2 gặp sự cố, toàn bộ hệ thống bị ảnh hưởng. Thứ hai, CI/CD hiện đáp ứng build/push/deploy cơ bản, chưa có canary/blue-green deployment, security scan bắt buộc hay quản lý secret chuyên dụng. Thứ ba, frontend/Admin Panel phục vụ tốt demo DATN nhưng chưa phải sản phẩm thương mại hoàn chỉnh: chưa có upload file qua object storage, workflow kiểm duyệt review đầy đủ, ban/unban user qua Keycloak và analytics service riêng. Thứ tư, event schema đang dùng module common, phù hợp monorepo đề án nhưng production lớn nên cân nhắc schema registry hoặc contract testing. Thứ năm, hệ thống có metrics/tracing nhưng chưa có log aggregation và alerting production-grade.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Kết luận

Đề tài đã hoàn thành việc phân tích, thiết kế, cài đặt, triển khai và kiểm chứng một hệ thống thương mại điện tử theo kiến trúc microservices với Java 21, Spring Boot 3.5 và Spring Cloud 2025. Hệ thống gồm 13 business microservice, ba Spring Cloud infrastructure service, các thành phần hạ tầng dữ liệu/messaging/observability, cùng Next.js storefront và Admin Panel phục vụ demo các luồng nghiệp vụ chính.

Về kiến trúc, hệ thống áp dụng Domain-Driven Design và Bounded Context để phân rã bài toán thành các service độc lập theo miền nghiệp vụ. Giao tiếp đồng bộ được thực hiện qua REST/OpenFeign, trong khi giao tiếp bất đồng bộ sử dụng Apache Kafka. Các mẫu thiết kế phân tán quan trọng đã được hiện thực và kiểm chứng gồm Saga Orchestration cho luồng đặt hàng, Transactional Outbox cho order/payment event, Idempotent Consumer để chống xử lý trùng lặp, CQRS-lite cho tìm kiếm sản phẩm bằng Elasticsearch, scheduler/reconciliation cho các tình huống timeout và tự phục hồi.

Về tính năng, hệ thống đáp ứng các nghiệp vụ cốt lõi của một nền tảng thương mại điện tử: đăng ký và xác thực người dùng qua Keycloak, quản lý sản phẩm/danh mục/thương hiệu, giỏ hàng, đặt hàng COD và VNPAY sandbox, voucher, tồn kho, thông báo email, đánh giá sản phẩm, tìm kiếm toàn văn, nội dung CMS, flash-sale và quản trị hệ thống. Next.js storefront hỗ trợ các luồng người mua, còn Admin Panel cung cấp các module quản trị dashboard, products, inventory, orders, users, vouchers, flash-sales, content và reviews.

Điểm nổi bật nhất về kỹ thuật là luồng đặt hàng và flash-sale. Luồng đặt hàng thể hiện đầy đủ Saga Orchestration, Outbox và Idempotent Consumer để đạt tính nhất quán cuối cùng giữa order, inventory, voucher, payment và notification. Luồng flash-sale sử dụng Redis Lua atomic script, buyer set, compensation và ReconciliationScheduler để chống over-sell. Spike test 500 VU cho kết quả đúng 100 purchase thành công trên 100 slot, 100 đơn confirmed và 0 duplicate buyer, cho thấy thiết kế đáp ứng mục tiêu thực nghiệm của đồ án.

Về vận hành, hệ thống được đóng gói bằng Docker Compose và triển khai production-like trên AWS EC2 single-host. Hệ thống sử dụng Caddy reverse proxy, HTTPS và GitHub Actions/GHCR cho quy trình build/push image. Observability stack gồm Prometheus, Grafana và Zipkin giúp theo dõi metrics, trace và debug

các luồng nghiệp vụ end-to-end. Kết quả kiểm thử cho thấy backend readiness đạt 9/9 split suites; catalog soak đạt p95 61.68 ms với 0.00% lỗi; checkout stress đạt p95 160.92 ms với order success 100.00%; các kịch bản Kill Kafka, Kill Redis và inventory failure compensation đều đạt trong phạm vi thực nghiệm.

Hướng phát triển

Bên cạnh kết quả đạt được, hệ thống vẫn còn một số hướng phát triển quan trọng nếu tiếp tục nâng cấp sau phạm vi đề án.

Nâng cấp hạ tầng triển khai. Hướng phát triển tự nhiên là chuyển từ Docker Compose single-host sang Kubernetes multi-node để có high availability, autoscaling, rolling update và self-healing tốt hơn. Khi đó có thể bổ sung Service Mesh như Istio hoặc Linkerd để quản lý mTLS, traffic splitting, retry policy và fault injection ở tầng hạ tầng.

Hoàn thiện CI/CD và vận hành production. Pipeline hiện đã đáp ứng build/push image và deploy EC2 ở mức production-like. Giai đoạn tiếp theo nên bổ sung automated test gate, security scanning bằng Trivy/Snyk, quản lý secret chuyên dụng, deployment đa môi trường, Alertmanager/Grafana Alerting và log aggregation tập trung bằng ELK hoặc Loki.

Hoàn thiện frontend và nghiệp vụ quản trị. Storefront và Admin Panel đã đủ cho demo DATN, nhưng để tiến gần sản phẩm thương mại cần bổ sung object storage cho upload ảnh/file, workflow kiểm duyệt review, ban/unban user qua Keycloak Admin API, dashboard analytics chuyên dụng, tối ưu mobile UX và các nghiệp vụ sau bán như đổi trả, hoàn tiền, vận chuyển.

Nâng cao hợp đồng tích hợp và chất lượng kiểm thử. Module `common` đang đóng vai trò shared kernel cho DTO/event schema, phù hợp monorepo đề án nhưng có thể tạo coupling khi hệ thống mở rộng. Hướng phát triển là bổ sung schema registry hoặc contract testing cho Kafka/REST API, thiết lập ngưỡng coverage với JaCoCo, thêm mutation test cho nghiệp vụ quan trọng và kiểm thử security theo OWASP Top 10.

Tối ưu hiệu năng và dữ liệu. Với quy mô lớn hơn, hệ thống có thể tách PostgreSQL thành cluster hoặc managed database, dùng read replica cho các read-heavy service, Redis Cluster cho cache/flash-sale, storage bền vững cho Zipkin, và analytics/read model riêng cho báo cáo doanh thu. Các hướng như Event Sourcing hoặc gRPC nội bộ chỉ nên cân nhắc khi nhu cầu audit hoặc latency thực sự vượt quá khả năng của thiết kế hiện tại.

PHỤ LỤC

A. HƯỚNG DẪN CÀI ĐẶT VÀ VẬN HÀNH

Phụ lục này cung cấp các lệnh vận hành chính dùng để biên dịch, khởi động, kiểm tra và dừng hệ thống trong môi trường local và môi trường AWS EC2 dạng production-like.

A.1. Yêu cầu hệ thống

- **Java:** OpenJDK 21.
- **Docker:** Docker Desktop 27+ hoặc Docker Engine kèm Docker Compose plugin.
- **Maven:** Không cần cài riêng vì dự án dùng Maven Wrapper `./mvnw`.
- **Node.js:** Node.js 20+ cho frontend Next.js.
- **RAM:** Tối thiểu 12GB khả dụng cho Docker, khuyến nghị 16GB.
- **Disk:** Tối thiểu 10GB cho Docker images, volumes và build artifacts.

A.2. Biên dịch backend

Từ thư mục gốc của dự án, thực thi:

```
./mvnw clean package -DskipTests
```

Lệnh này biên dịch toàn bộ 17 module Maven theo thứ tự phụ thuộc và đóng gói các service thành file JAR thực thi. Khi cần chạy test Maven đầy đủ, có thể bỏ tham số `-DskipTests` hoặc chạy theo từng module/suite để giảm tải tài nguyên.

A.3. Chạy backend và hạ tầng local

```
docker compose up -d --build
```

Lệnh trên khởi động PostgreSQL, Redis, Kafka, Elasticsearch, Keycloak, Mailpit, Prometheus, Grafana, Zipkin, ba Spring Cloud service và 13 business service. Các lần chạy đầu tiên có thể mất vài phút do cần build image và tải dependency.

A.4. Chạy frontend local

Frontend nằm trong thư mục `frontend/`. Khi cần chạy storefront hoặc Admin Panel ở chế độ development:

```
cd frontend
npm install
npm run dev
```

Mặc định Next.js dùng `http://localhost:3000`. Nếu Grafana local cũng đang dùng cổng 3000, cần đổi cổng một trong hai thành phần hoặc dùng cấu hình

production-like nơi Grafana được remap sang host port 3001.

A.5. Kiểm tra trạng thái local

```
# Xem trạng thái container
```

```
docker compose ps
```

```
# Xem log Gateway hoặc order-service
```

```
docker compose logs -f api-gateway
```

```
docker compose logs -f order-service
```

```
# Kiểm tra health Gateway
```

```
curl http://localhost:8080/actuator/health/readiness
```

```
# Kiểm tra Eureka registry
```

```
curl -u eureka:eureka http://localhost:8761/eureka/apps
```

A.6. Các URL local quan trọng

Bảng A.1: Các URL truy cập hệ thống trên môi trường local

Thành phần	URL	Ghi chú
Frontend	http://localhost:3000	Storefront/Admin Panel khi chạy Next.js dev
API Gateway	http://localhost:8080	Cổng vào backend
Swagger UI	http://localhost:8080/swagger-ui.html	Tài liệu API qua Gateway
Eureka Dashboard	http://localhost:8761	user: eureka, pass: eureka
Keycloak Admin	http://localhost:8180	Quản trị realm
Grafana	http://localhost:3000 hoặc 3001	Phụ thuộc cấu hình chạy frontend/-Grafana
Zipkin	http://localhost:9411	Distributed tracing
Prometheus	http://localhost:9090	Metrics
Mailpit	http://localhost:8025	Kiểm tra email
Elasticsearch	http://localhost:9200	Search engine

A.7. Chạy smoke test backend

```
.test/scripts/smoke-backend.sh --suite maven
```

```
.test/scripts/smoke-backend.sh --suite infra
.test/scripts/smoke-backend.sh --suite auth-user
.test/scripts/smoke-backend.sh --suite catalog-search
.test/scripts/smoke-backend.sh --suite cart-voucher
.test/scripts/smoke-backend.sh --suite order-cod-review
.test/scripts/smoke-backend.sh --suite vnpay
.test/scripts/smoke-backend.sh --suite flash-sale
.test/scripts/smoke-backend.sh --suite security
```

Kết quả final của các suite đã được lưu trong thư mục `.test/results` và được tổng hợp ở Chương 5.

A.8. Chạy kiểm thử hiệu năng bằng k6

Ví dụ chạy catalog soak hoặc checkout stress với gateway production-like:

```
k6 run -e BASE_URL=http://13.213.118.96:8080 .test/load/catalog-browse.js
k6 run -e BASE_URL=http://13.213.118.96:8080 .test/load/checkout-stress.js
```

Khi đưa số liệu vào báo cáo, chỉ dùng các artifact raw đã lưu trong `.test/results` để tránh ghi số liệu không đối chiếu được.

A.9. Vận hành EC2 production-like

Khi cần bật EC2 để demo:

```
source aws/config.env
aws ec2 start-instances --instance-ids $INSTANCE_ID
aws ec2 wait instance-running --instance-ids $INSTANCE_ID

# Doi 2-3 phut cho services khoi dong
ssh -i $SSH_KEY_PATH ubuntu@$ELASTIC_IP "docker compose ps"
```

Khi demo xong, nên tắt instance để tiết kiệm chi phí/credit:

```
source aws/config.env
aws ec2 stop-instances --instance-ids $INSTANCE_ID
```

A.10. Dừng hệ thống local

```
# Dừng và giữ dữ liệu
docker compose down
```

```
# Dừng và xóa toàn bộ dữ liệu volume
docker compose down -v
```

B. ĐẶC TẢ USE CASE

Phụ lục này cung cấp đặc tả chi tiết cho các use case nghiệp vụ chính của hệ thống. Mỗi use case được mô tả theo cấu trúc: tên use case, tác nhân chính, điều kiện tiên quyết, luồng sự kiện chính, luồng sự kiện thay thế và điều kiện hậu kiểm.

B.1. Đăng ký tài khoản

Tên use case	Đăng ký tài khoản người dùng
Tác nhân	Khách hàng (Guest)
Điều kiện tiên quyết	Người dùng chưa có tài khoản trong hệ thống
Luồng chính	1. Người dùng gửi yêu cầu đăng ký với email, mật khẩu và thông tin cá nhân 2. Identity-service gọi Keycloak Admin API tạo user 3. Identity-service phát hành sự kiện <code>user-registered</code> lên Kafka 4. User-service tiêu thụ sự kiện, tạo hồ sơ người dùng 5. Hệ thống trả về thông báo đăng ký thành công
Luồng thay thế	A1. Email đã tồn tại: trả về lỗi 409 Conflict A2. Mật khẩu không đạt yêu cầu: trả về lỗi 400 Bad Request
Hậu kiểm	User tồn tại trong Keycloak, hồ sơ user được tạo trong <code>user_db</code>

B.2. Đặt hàng COD

Tên use case	Đặt hàng thanh toán khi nhận hàng (COD)
Tác nhân	Khách hàng đã đăng nhập
Điều kiện tiên quyết	Người dùng đã đăng nhập, giỏ hàng không rỗng, sản phẩm còn tồn kho
Luồng chính	1. Khách hàng gửi yêu cầu tạo đơn hàng (paymentMethod=COD, danh sách sản phẩm, địa chỉ) 2. Order-service kiểm tra sản phẩm, tồn kho, voucher qua Feign 3. Order-service tạo Order (PENDING) và OutboxEvent trong cùng transaction 4. Inventory-service đặt chỗ tồn kho khi nhận order-created 5. Order-service xác nhận đơn hàng (CONFIRMED) khi nhận inventory-updated 6. Inventory-service xác nhận trừ tồn kho thực tế 7. Notification-service gửi email xác nhận đơn hàng
Luồng thay thế	A1. Tồn kho không đủ: Inventory phát hành inventory-failed, Order chuyển CANCELLED A2. Voucher không hợp lệ: Order-service trả lỗi, không tạo đơn
Hậu kiểm	Order status = CONFIRMED, tồn kho đã bị trừ, email đã được gửi

B.3. Thanh toán VNPAY

Tên use case	Thanh toán trực tuyến qua VNPAY
Tác nhân	Khách hàng đã đăng nhập, Hệ thống VNPAY
Điều kiện tiên quyết	Đơn hàng đã được tạo với paymentMethod=VNPAY, trạng thái STOCK_RESERVED
Luồng chính	1. Khách hàng yêu cầu tạo URL thanh toán 2. Payment-service tạo payment (PENDING), sinh URL với chữ ký HMAC-SHA512 3. Khách hàng được chuyển hướng đến trang thanh toán VNPAY 4. Khách hàng nhập thông tin thẻ và xác nhận thanh toán 5. VNPAY gọi IPN callback với kết quả thành công 6. Payment-service xác minh chữ ký, cập nhật payment COMPLETED 7. Order-service nhận payment-success, chuyển order CONFIRMED 8. Inventory xác nhận tồn kho, Notification gửi email
Luồng thay thế	A1. Thanh toán thất bại: VNPAY trả về mã lỗi, payment chuyển FAILED, order CANCELLED A2. Quá thời hạn 30 phút: Scheduler tự động hủy đơn và giải phóng tồn kho A3. Chữ ký không hợp lệ: Payment-service từ chối, không cập nhật trạng thái
Hậu kiểm	Payment status = COMPLETED, Order status = CONFIRMED

B.4. Tham gia Flash Sale

Tên use case	Mua hàng trong sự kiện Flash Sale
Tác nhân	Khách hàng đã đăng nhập
Điều kiện tiên quyết	Chiến dịch đang ở trạng thái ACTIVE, trong thời gian diễn ra, còn tồn kho
Luồng chính	1. Khách hàng gửi yêu cầu mua hàng <code>POST/api/flash-sales/{id}/purchase</code> 2. Flash-sale-service kiểm tra thời gian hiệu lực của chiến dịch 3. Thực thi Redis Lua script: kiểm tra stock, kiểm tra chưa mua, giảm stock 4. Nếu Redis trả về thành công, phát hành <code>flash-sale-order-requested</code> lên Kafka 5. Order-service tạo đơn hàng với giá flash sale 6. Xử lý Saga tương tự luồng COD
Luồng thay thế	- A1. Hết hàng (Redis stock = 0): trả về HTTP 409 Conflict - A2. Đã mua trước đó: Redis buyers set phát hiện, trả về HTTP 409 - A3. Chiến dịch chưa bắt đầu/đã kết thúc: trả về HTTP 400 - A4. Tạo đơn thất bại sau khi giữ slot: compensation hoàn Redis stock và buyer set
Hậu kiểm	Đơn hàng flash sale được tạo, không vượt quá số lượng, không trùng người mua

B.5. Tìm kiếm sản phẩm

Tên use case	Tìm kiếm sản phẩm toàn văn
Tác nhân	Khách hàng (Guest hoặc đã đăng nhập)
Điều kiện tiên quyết	Elasticsearch đã được đồng bộ dữ liệu sản phẩm
Luồng chính	1. Người dùng gửi yêu cầu GET /api/search?keyword=X&category=Y&minPrice=Z 2. Search-service xây dựng Boolean Query: full-text match + term filter + range filter 3. Truy vấn Elasticsearch index products 4. Trả về danh sách sản phẩm phù hợp kèm phân trang
Luồng thay thế	A1. Không có kết quả: trả về danh sách rỗng A2. Elasticsearch không khả dụng: trả về lỗi 503 Service Unavailable
Hậu kiểm	Kết quả tìm kiếm được trả về với thời gian phản hồi dưới 100ms